4.01-hpi-unos-modelling-full

July 5, 2024

[1]:
```
%load_ext autoreload
%autoreload 2

from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
from prism.PRiSM_functions import (
    normalise,
    train_mlp_batched,
    modelStats,
    extract_weights,
    partialResponses,
    prPlots
)
from prism.prlasso import prLASSO
from prism.prnomogram import prnomogram_refactor
from prism.maskedmlp import (
    mlpmask_pytorch,
    generate_mask,
    get_model_weights_with_biases
)
from prism.save_models import (
    save_mlp,
    save_lasso,
    save_prn
)

import pandas as pd
import numpy as np
import torch
import pickle
from datetime import datetime
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
```

```
2024-07-05 13:50:07.214 | INFO     |
```

```
prism.config:<module>:11 - PROJ_ROOT path is:

C:\Users\Henry Pigot\Documents\MEGAsync\2024 post doc\explainable survival

prediction model\prism_github
```

0.1 Parameters

[2]:
```
device = 'cpu'
method = 'dirac'
SAVE_MODELS = True

seed = 257

np.random.seed(seed)
torch.manual_seed(seed)
```

[2]:
```
<torch._C.Generator at 0x26420441ef0>
```

[3]:
```
# Print gpu hardware info
if torch.cuda.is_available():
    print("Number of CUDA devices:", torch.cuda.device_count())

    for i in range(torch.cuda.device_count()):
        print("CUDA Device #{}: {}".format(i, torch.cuda.get_device_name(i)))

    for i in range(torch.cuda.device_count()):
        print("Total memory of CUDA Device #{}: {:.2f} GB".format(
            i, torch.cuda.get_device_properties(i).total_memory / 1e9))

    for i in range(torch.cuda.device_count()):
        device_properties = torch.cuda.get_device_properties(i)
        print("CUDA Capability of Device #{}: {}.{}".format(
            i, device_properties.major, device_properties.minor))

    print("Default data type for CUDA tensors:", torch.get_default_dtype())
    print(f"CUDA version: {torch.version.cuda}")
else:
    print("CUDA is not available.")
```

```
CUDA is not available.
```

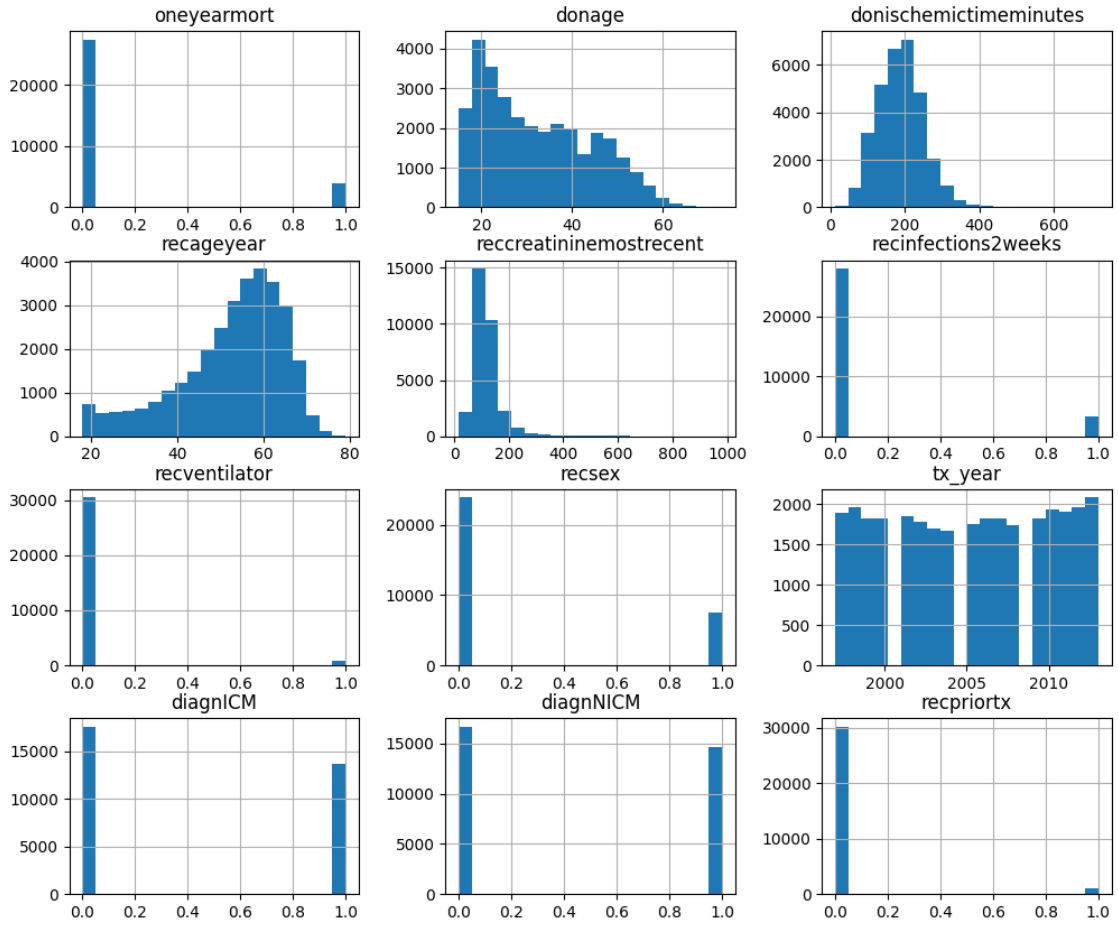
0.2 Import data

Preprocessed and split into train, test, and validate.

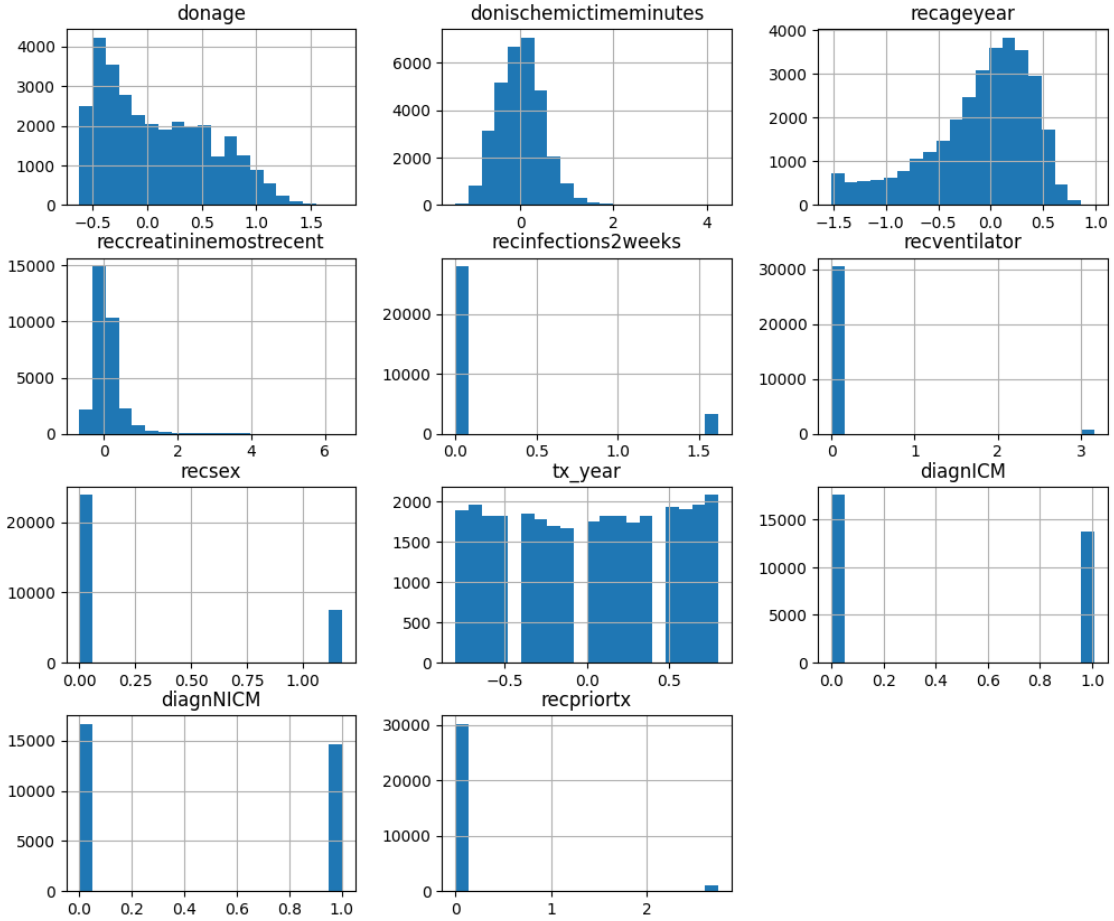
Target: `oneyearmort`

[4]:
```
data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
↪csv'))
```

```
data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
 ↪csv'))
data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

data_train_test = pd.concat([data_train,data_test]) # used for axis annotation␣
 ↪in some plotting functions

# drop id column
data_train.drop('trr_id_code',axis=1,inplace=True)
data_test.drop('trr_id_code',axis=1,inplace=True)
data_val.drop('trr_id_code',axis=1,inplace=True)

target_col = 'oneyearmort'

data_train.hist(bins=20,figsize=(12,10));
```



[5]:
```
x_train0 = data_train.drop(target_col,axis=1)
y_train = data_train[target_col]

x_test0 = data_test.drop(target_col,axis=1)
y_test = data_test[target_col]

x_val0 = data_val.drop(target_col,axis=1)
y_val = data_val[target_col]

[x_train,x_test] = normalise(x_train0,x_test0)
x_val = normalise(x_val0)

# check result
x_train.hist(bins=20,figsize=(12,10));
```



0.3 Train MLP

[6]:
```
mlp_params = {
    'n_hidden': 10,
    'weight_decay': 1e-5,
    'lr': 0.001,
    'patience': 50,
    'tolerance': 0.0001,
    'batch_size': 1024,
    'device': device,
    'seed': seed
}

mlp_batched = train_mlp_batched(x_train, y_train, x_test, y_test, **mlp_params)
```

```
Epoch 0: Train loss 0.6690, Val loss 0.6166
Epoch 1: Train loss 0.5749, Val loss 0.5303
Epoch 2: Train loss 0.5062, Val loss 0.4680
Epoch 3: Train loss 0.4573, Val loss 0.4224
Epoch 4: Train loss 0.4240, Val loss 0.3906
Epoch 5: Train loss 0.4014, Val loss 0.3693
Epoch 6: Train loss 0.3869, Val loss 0.3563
Epoch 7: Train loss 0.3782, Val loss 0.3485
Epoch 8: Train loss 0.3726, Val loss 0.3440
Epoch 9: Train loss 0.3694, Val loss 0.3416
Epoch 10: Train loss 0.3667, Val loss 0.3402
Epoch 11: Train loss 0.3653, Val loss 0.3395
Epoch 12: Train loss 0.3644, Val loss 0.3393
Epoch 13: Train loss 0.3640, Val loss 0.3392
Epoch 14: Train loss 0.3631, Val loss 0.3391
Epoch 15: Train loss 0.3621, Val loss 0.3391
Epoch 16: Train loss 0.3624, Val loss 0.3392
Epoch 17: Train loss 0.3617, Val loss 0.3392
Epoch 18: Train loss 0.3613, Val loss 0.3393
Epoch 19: Train loss 0.3611, Val loss 0.3392
Epoch 20: Train loss 0.3611, Val loss 0.3393
Epoch 21: Train loss 0.3608, Val loss 0.3394
Epoch 22: Train loss 0.3601, Val loss 0.3392
Epoch 23: Train loss 0.3604, Val loss 0.3392
Epoch 24: Train loss 0.3604, Val loss 0.3392
Epoch 25: Train loss 0.3601, Val loss 0.3390
Epoch 26: Train loss 0.3598, Val loss 0.3390
Epoch 27: Train loss 0.3593, Val loss 0.3389
Epoch 28: Train loss 0.3598, Val loss 0.3391
Epoch 29: Train loss 0.3586, Val loss 0.3388
Epoch 30: Train loss 0.3588, Val loss 0.3389
Epoch 31: Train loss 0.3587, Val loss 0.3388
Epoch 32: Train loss 0.3584, Val loss 0.3389
```

```
Epoch 33: Train loss 0.3585, Val loss 0.3388
Epoch 34: Train loss 0.3584, Val loss 0.3388
Epoch 35: Train loss 0.3588, Val loss 0.3387
Epoch 36: Train loss 0.3583, Val loss 0.3386
Epoch 37: Train loss 0.3581, Val loss 0.3386
Epoch 38: Train loss 0.3583, Val loss 0.3385
Epoch 39: Train loss 0.3576, Val loss 0.3384
Epoch 40: Train loss 0.3577, Val loss 0.3384
Epoch 41: Train loss 0.3574, Val loss 0.3385
Epoch 42: Train loss 0.3572, Val loss 0.3384
Epoch 43: Train loss 0.3567, Val loss 0.3383
Epoch 44: Train loss 0.3573, Val loss 0.3385
Epoch 45: Train loss 0.3568, Val loss 0.3383
Epoch 46: Train loss 0.3569, Val loss 0.3384
Epoch 47: Train loss 0.3573, Val loss 0.3383
Epoch 48: Train loss 0.3570, Val loss 0.3380
Epoch 49: Train loss 0.3573, Val loss 0.3382
Epoch 50: Train loss 0.3567, Val loss 0.3381
Epoch 51: Train loss 0.3571, Val loss 0.3381
Epoch 52: Train loss 0.3568, Val loss 0.3379
Epoch 53: Train loss 0.3562, Val loss 0.3378
Epoch 54: Train loss 0.3567, Val loss 0.3377
Epoch 55: Train loss 0.3564, Val loss 0.3379
Epoch 56: Train loss 0.3566, Val loss 0.3380
Epoch 57: Train loss 0.3558, Val loss 0.3378
Epoch 58: Train loss 0.3564, Val loss 0.3379
Epoch 59: Train loss 0.3559, Val loss 0.3376
Epoch 60: Train loss 0.3550, Val loss 0.3377
Epoch 61: Train loss 0.3555, Val loss 0.3378
Epoch 62: Train loss 0.3561, Val loss 0.3378
Epoch 63: Train loss 0.3547, Val loss 0.3377
Epoch 64: Train loss 0.3551, Val loss 0.3377
Epoch 65: Train loss 0.3556, Val loss 0.3378
Epoch 66: Train loss 0.3553, Val loss 0.3375
Epoch 67: Train loss 0.3555, Val loss 0.3377
Epoch 68: Train loss 0.3552, Val loss 0.3376
Epoch 69: Train loss 0.3547, Val loss 0.3377
Epoch 70: Train loss 0.3555, Val loss 0.3376
Epoch 71: Train loss 0.3549, Val loss 0.3378
Epoch 72: Train loss 0.3546, Val loss 0.3376
Epoch 73: Train loss 0.3546, Val loss 0.3376
Epoch 74: Train loss 0.3550, Val loss 0.3375
Epoch 75: Train loss 0.3541, Val loss 0.3375
Epoch 76: Train loss 0.3547, Val loss 0.3378
Epoch 77: Train loss 0.3543, Val loss 0.3376
Epoch 78: Train loss 0.3545, Val loss 0.3375
Epoch 79: Train loss 0.3542, Val loss 0.3374
Epoch 80: Train loss 0.3552, Val loss 0.3375
```

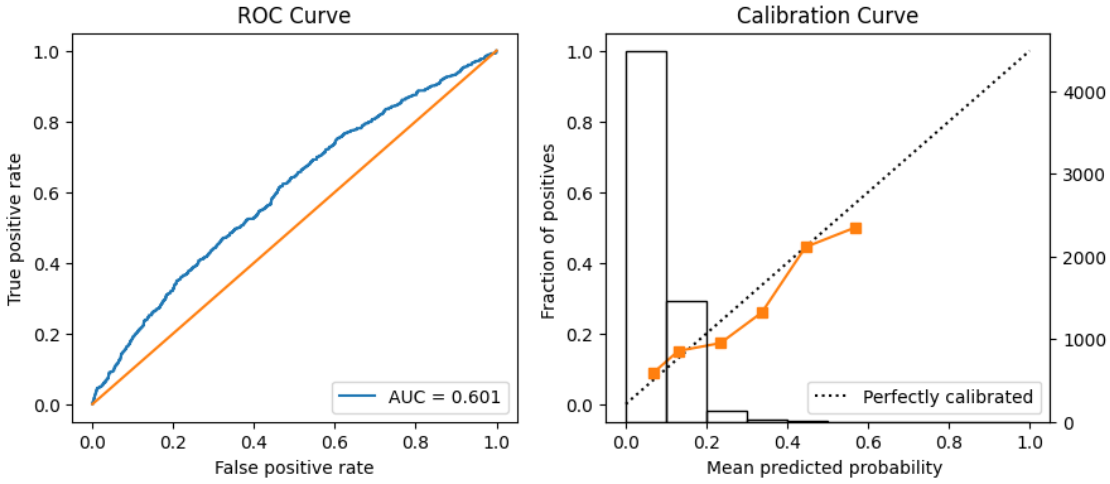
```
Epoch 81: Train loss 0.3544, Val loss 0.3374
Epoch 82: Train loss 0.3538, Val loss 0.3376
Epoch 83: Train loss 0.3546, Val loss 0.3375
Epoch 84: Train loss 0.3540, Val loss 0.3374
Epoch 85: Train loss 0.3552, Val loss 0.3374
Epoch 86: Train loss 0.3548, Val loss 0.3376
Epoch 87: Train loss 0.3542, Val loss 0.3373
Epoch 88: Train loss 0.3540, Val loss 0.3376
Epoch 89: Train loss 0.3542, Val loss 0.3375
Epoch 90: Train loss 0.3542, Val loss 0.3376
Epoch 91: Train loss 0.3540, Val loss 0.3375
Epoch 92: Train loss 0.3543, Val loss 0.3375
Epoch 93: Train loss 0.3537, Val loss 0.3374
Epoch 94: Train loss 0.3539, Val loss 0.3377
Epoch 95: Train loss 0.3543, Val loss 0.3375
Epoch 96: Train loss 0.3540, Val loss 0.3374
Epoch 97: Train loss 0.3537, Val loss 0.3377
Epoch 98: Train loss 0.3535, Val loss 0.3374
Epoch 99: Train loss 0.3537, Val loss 0.3374
Epoch 100: Train loss 0.3534, Val loss 0.3375
Epoch 101: Train loss 0.3542, Val loss 0.3376
Epoch 102: Train loss 0.3539, Val loss 0.3376
Epoch 103: Train loss 0.3540, Val loss 0.3377
Epoch 104: Train loss 0.3532, Val loss 0.3373
Epoch 105: Train loss 0.3537, Val loss 0.3378
Epoch 106: Train loss 0.3537, Val loss 0.3375
Epoch 107: Train loss 0.3535, Val loss 0.3374
Epoch 108: Train loss 0.3535, Val loss 0.3377
Epoch 109: Train loss 0.3533, Val loss 0.3377
Epoch 110: Train loss 0.3535, Val loss 0.3377
Epoch 111: Train loss 0.3536, Val loss 0.3376
Epoch 112: Train loss 0.3539, Val loss 0.3379
Epoch 113: Train loss 0.3533, Val loss 0.3376
Epoch 114: Train loss 0.3539, Val loss 0.3377
Epoch 115: Train loss 0.3536, Val loss 0.3379
Epoch 116: Train loss 0.3543, Val loss 0.3379
Epoch 117: Train loss 0.3534, Val loss 0.3376
Epoch 118: Train loss 0.3534, Val loss 0.3376
Epoch 119: Train loss 0.3545, Val loss 0.3378
Epoch 120: Train loss 0.3538, Val loss 0.3379
Epoch 121: Train loss 0.3538, Val loss 0.3374
Epoch 122: Train loss 0.3535, Val loss 0.3374
Epoch 123: Train loss 0.3536, Val loss 0.3377
Epoch 124: Train loss 0.3527, Val loss 0.3378
Epoch 125: Train loss 0.3535, Val loss 0.3378
Epoch 126: Train loss 0.3532, Val loss 0.3377
Epoch 127: Train loss 0.3538, Val loss 0.3380
Epoch 128: Train loss 0.3536, Val loss 0.3378
```

```
Epoch 129: Train loss 0.3537, Val loss 0.3378
Epoch 130: Train loss 0.3527, Val loss 0.3378
Epoch 131: Train loss 0.3532, Val loss 0.3376
Epoch 132: Train loss 0.3534, Val loss 0.3377
Epoch 133: Train loss 0.3533, Val loss 0.3378
Epoch 134: Train loss 0.3529, Val loss 0.3377
Epoch 135: Train loss 0.3537, Val loss 0.3379
Epoch 136: Train loss 0.3535, Val loss 0.3380
Stopping early at epoch 87
```

0.4 Evaluate MLP

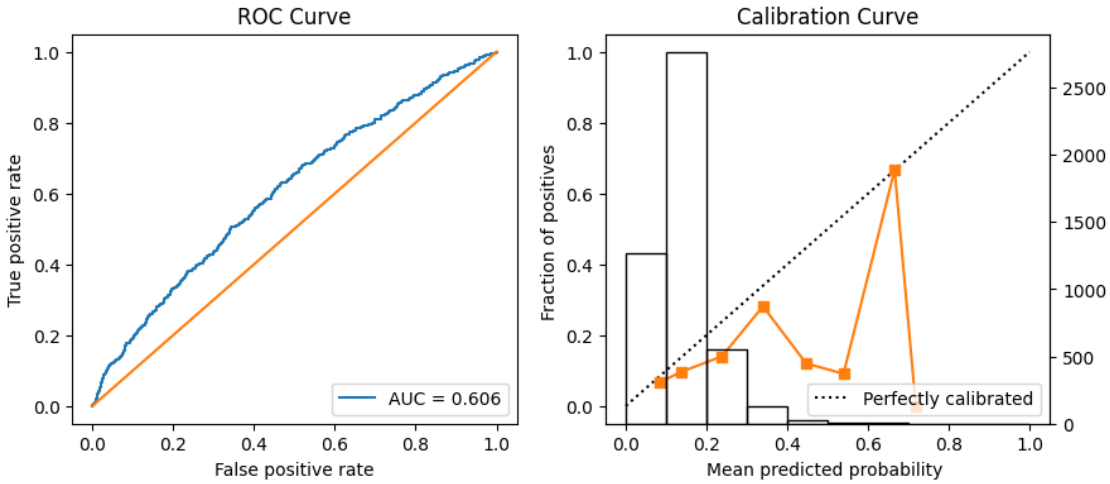
[7]:
```
x_test_tensor = torch.tensor(x_test.values, dtype=torch.float32,device=device)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32)
x_val_tensor = torch.tensor(x_val.values, dtype=torch.float32,device=device)
y_val_tensor = torch.tensor(y_val.values, dtype=torch.float32)
```

[8]:
```
# Get model inference
mlp_batched.eval()
with torch.no_grad():
    y_test_blackbox = mlp_batched(x_test_tensor).to('cpu').numpy()
    y_val_blackbox = mlp_batched(x_val_tensor).to('cpu').numpy()

mlp_metrics_test = modelStats(y_test_blackbox, y_test, y_train, ROC=True,␣
 ↪mdlCalibration=True, metricNames=True, auc_ci=True)
modelStats(y_val_blackbox, y_val, y_train, ROC=True, mdlCalibration=True,␣
 ↪metricNames=True, auc_ci=True)
```




```
-------- Metrics --------
prevalence: 0.123
```

```
sensitivity: 0.241
specificity: 0.864
accuracy: 0.797
ppv: 0.175
auc score: 0.601
auc lower ci: 0.577
auc upper ci: 0.626
-------------------------
```




```
-------- Metrics --------
prevalence: 0.123
sensitivity: 0.667
specificity: 0.489
accuracy: 0.507
ppv: 0.126
auc score: 0.606
auc lower ci: 0.581
auc upper ci: 0.636
-------------------------
```

[8]:
```
{'prevalence': 0.123,
 'sensitivity': 0.667,
 'specificity': 0.489,
 'accuracy': 0.507,
 'ppv': 0.126,
 'auc score': 0.606,
 'auc lower ci': '0.581',
 'auc upper ci': '0.636'}
```

0.5 Save MLP

[9]:
```
if SAVE_MODELS:
    save_mlp(mlp_batched, mlp_params, mlp_metrics_test, MODELS_DIR)
```

```
MLP model saved as mlp_model_20240705_135534
```

0.6 MLP LASSO

[10]:
```
blackbox_weights = extract_weights(mlp_batched)

F_covariates_train, F_covariates_test, bivariate_inputs =␣
 ↪partialResponses(x_train, x_test, blackbox_weights,␣
 ↪method=method,device=device)
```

```
Univariate Responses:
Column 0:
Column 1:
Column 2:
Column 3:
Column 4:
Column 5:
Column 6:
Column 7:
Column 8:
Column 9:
Column 10:
Bivariate Responses:
Columns 0 & 1:
Columns 0 & 2:
Columns 0 & 3:
Columns 0 & 4:
Columns 0 & 5:
Columns 0 & 6:
Columns 0 & 7:
Columns 0 & 8:
Columns 0 & 9:
Columns 0 & 10:
Columns 1 & 2:
Columns 1 & 3:
Columns 1 & 4:
Columns 1 & 5:
Columns 1 & 6:
Columns 1 & 7:
Columns 1 & 8:
Columns 1 & 9:
Columns 1 & 10:
Columns 2 & 3:
Columns 2 & 4:
```

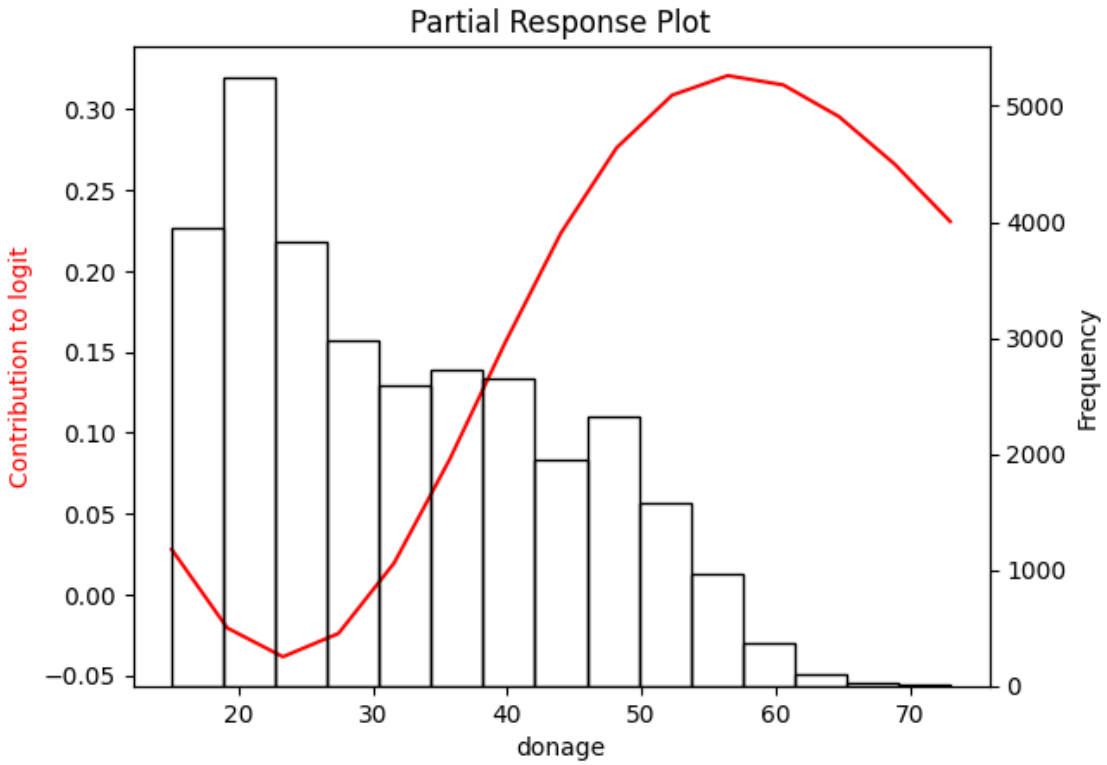
```
Columns 2 & 5:
Columns 2 & 6:
Columns 2 & 7:
Columns 2 & 8:
Columns 2 & 9:
Columns 2 & 10:
Columns 3 & 4:
Columns 3 & 5:
Columns 3 & 6:
Columns 3 & 7:
Columns 3 & 8:
Columns 3 & 9:
Columns 3 & 10:
Columns 4 & 5:
Columns 4 & 6:
Columns 4 & 7:
Columns 4 & 8:
Columns 4 & 9:
Columns 4 & 10:
Columns 5 & 6:
Columns 5 & 7:
Columns 5 & 8:
Columns 5 & 9:
Columns 5 & 10:
Columns 6 & 7:
Columns 6 & 8:
Columns 6 & 9:
Columns 6 & 10:
Columns 7 & 8:
Columns 7 & 9:
Columns 7 & 10:
Columns 8 & 9:
Columns 8 & 10:
Columns 9 & 10:
Residual
```

Plot all univariate partial responses, for initial eval of MLP training results.

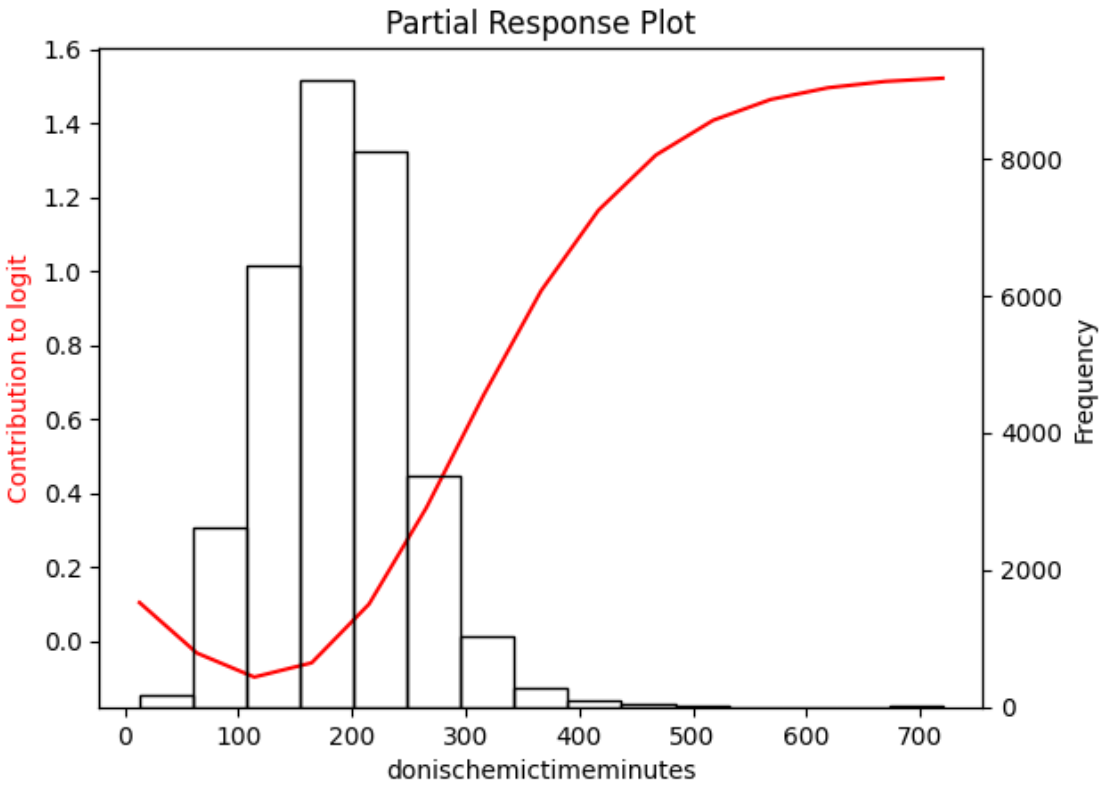
[11]:
```
betas_univariate = np.zeros([F_covariates_train.shape[1],1])
betas_univariate[:x_train.shape[1],0] = 1
```

[12]:
```
prPlots(betas_univariate, 0, x_train0, x_train, data_train_test,␣
↪blackbox_weights, bivariate_inputs, n_steps = 15, sd_scale=2, method=method,␣
↪device=device)
```

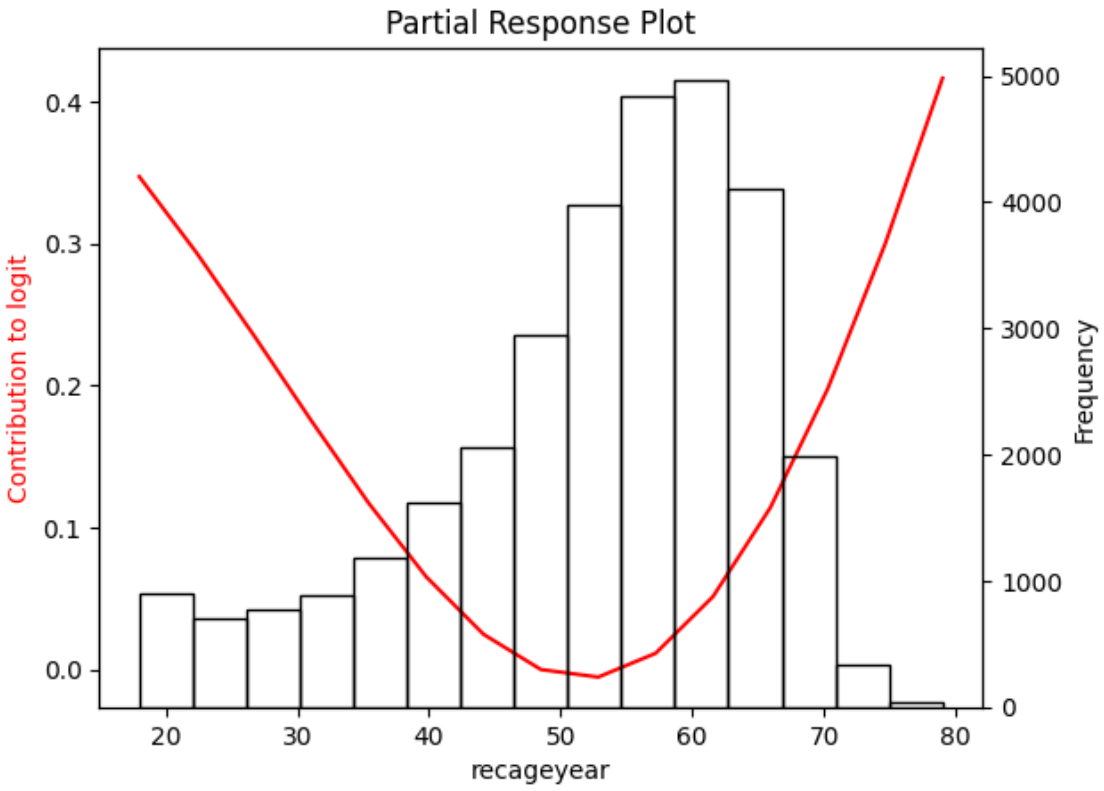
```
0 - univ
```


Partial Response Plot
Contribution to logit
Frequency
donage
0.30
0.25
0.20
0.15
0.10
0.05
0.00
−0.05
5000
4000
3000
2000
1000
0
20
30
40
50
60
70

```
1 - univ
```


Partial Response Plot
Contribution to logit
Frequency
donischemictimeminutes
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
8000
6000
4000
2000
0
0
100
200
300
400
500
600
700

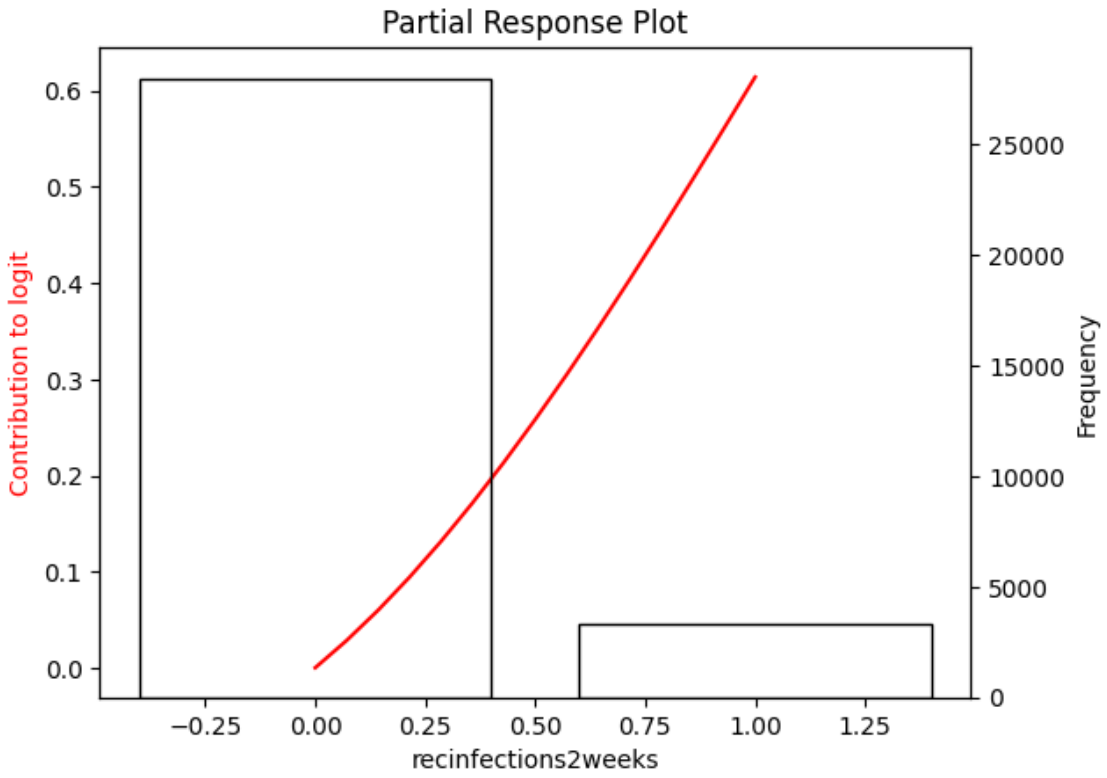
2 - univ


Partial Response Plot
Contribution to logit
Frequency
recageyear
0.0
0.1
0.2
0.3
0.4
0
1000
2000
3000
4000
5000
20
30
40
50
60
70
80

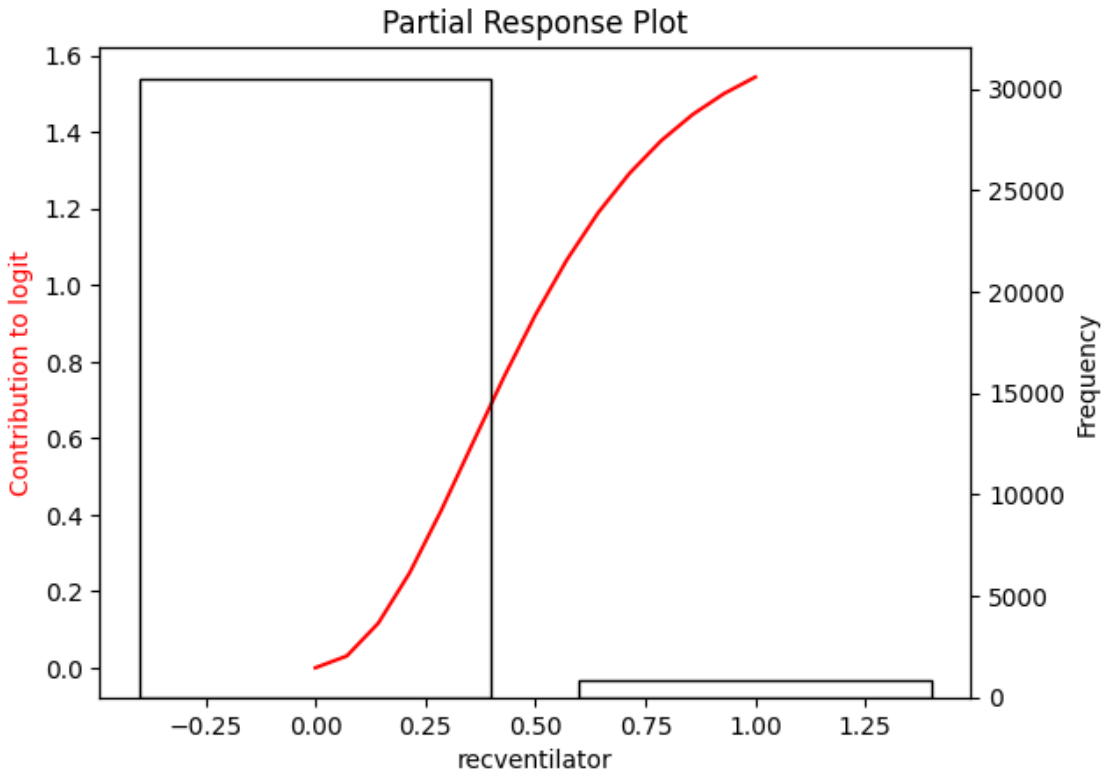
3 - univ


Partial Response Plot
Contribution to logit
Frequency
reccreatininemostrecent
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
17500
15000
12500
10000
7500
5000
2500
0
0
200
400
600
800
1000

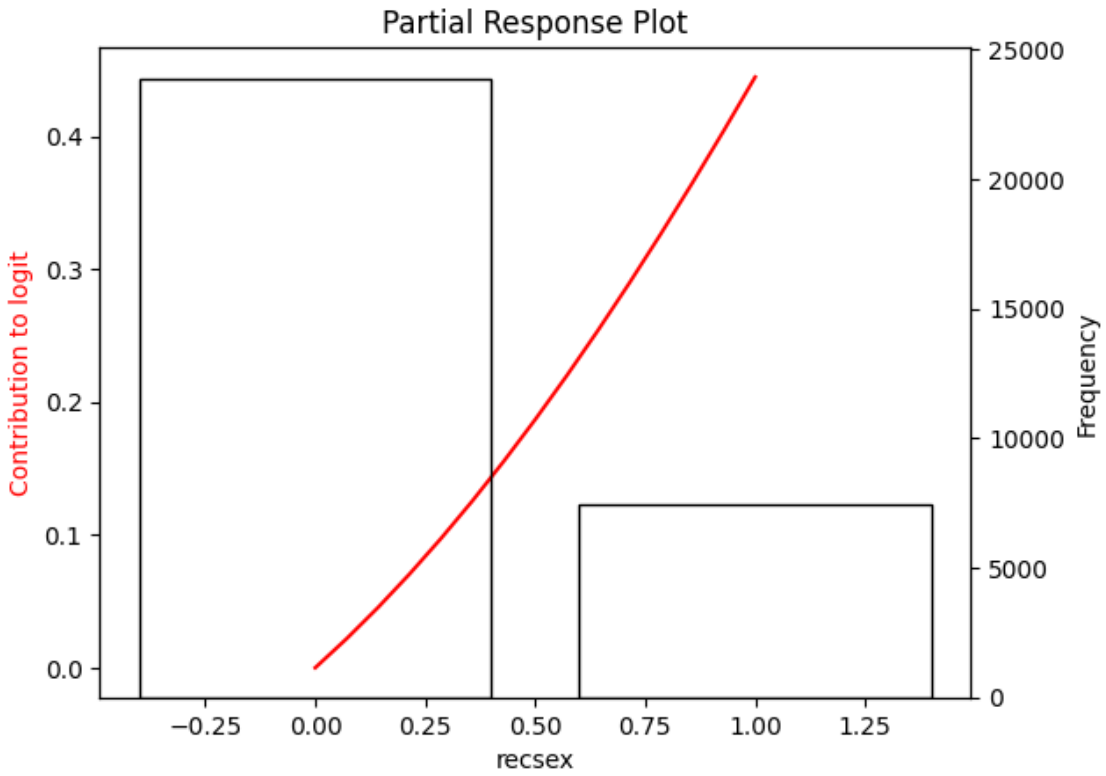
```
4 - univ
```


Partial Response Plot
Contribution to logit
Frequency
recinfections2weeks
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

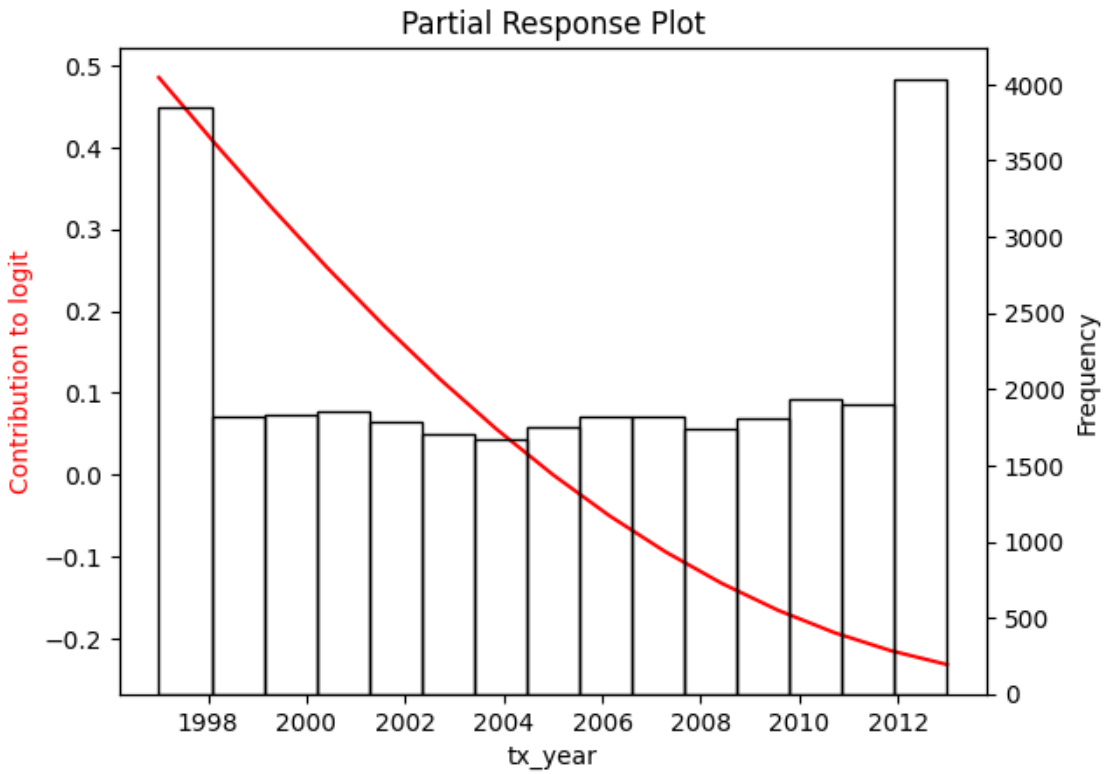
```
5 - univ
```


Partial Response Plot
Contribution to logit
Frequency
recventilator
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
30000
25000
20000
15000
10000
5000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

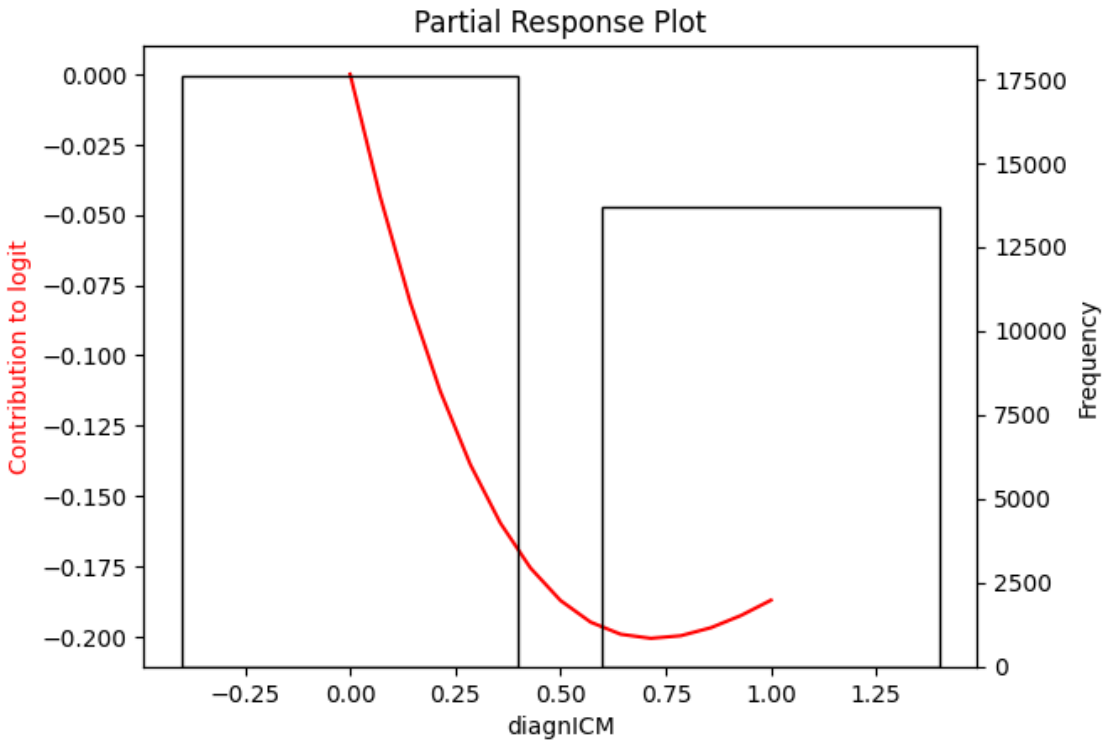
6 - univ


Partial Response Plot
Contribution to logit
Frequency
recsex
0.0
0.1
0.2
0.3
0.4
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

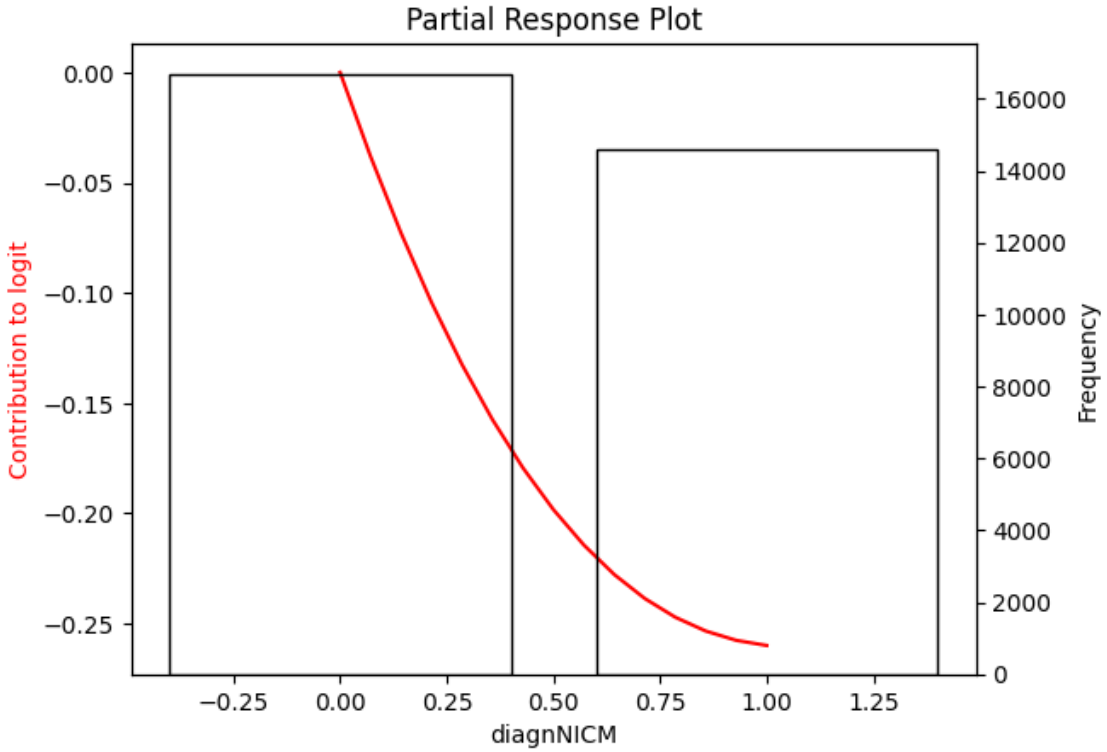
```
7 - univ
```


Partial Response Plot
Contribution to logit
Frequency
tx_year
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
4000
3500
3000
2500
2000
1500
1000
500
0
1998
2000
2002
2004
2006
2008
2010
2012

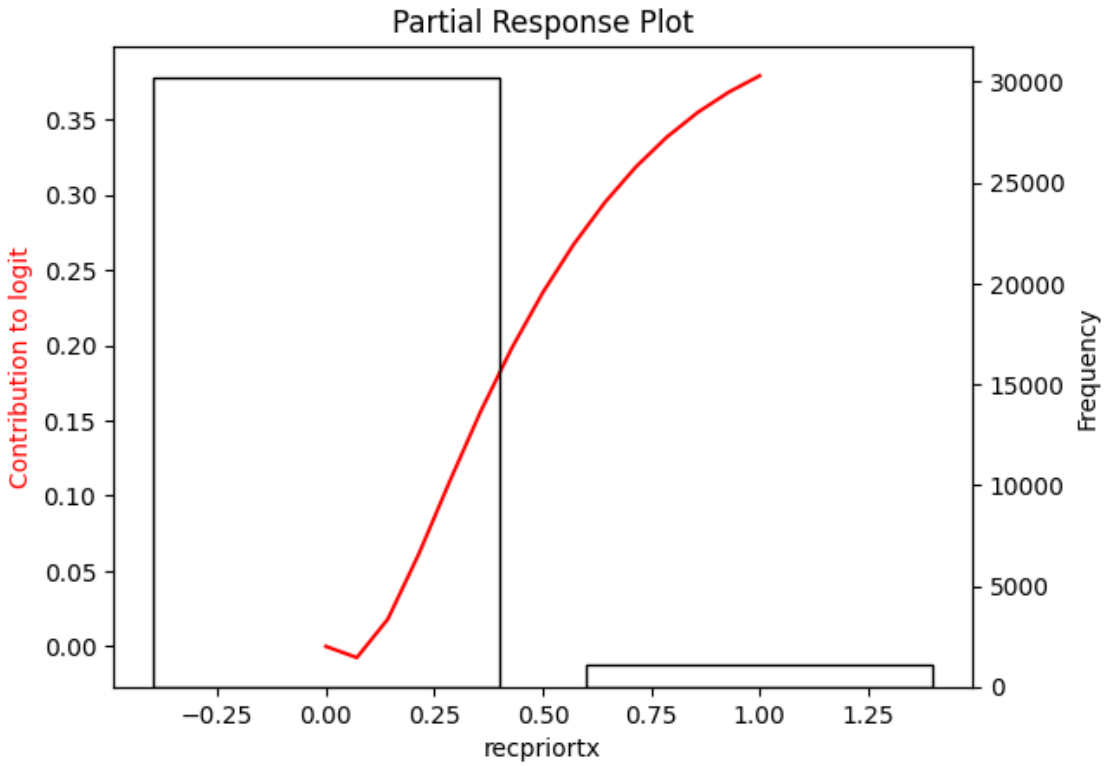
8 - univ


Partial Response Plot
Contribution to logit
Frequency
diagnICM
0.000
−0.025
−0.050
−0.075
−0.100
−0.125
−0.150
−0.175
−0.200
17500
15000
12500
10000
7500
5000
2500
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

9 - univ


Partial Response Plot
Contribution to logit
Frequency
diagnNICM
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
16000
14000
12000
10000
8000
6000
4000
2000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

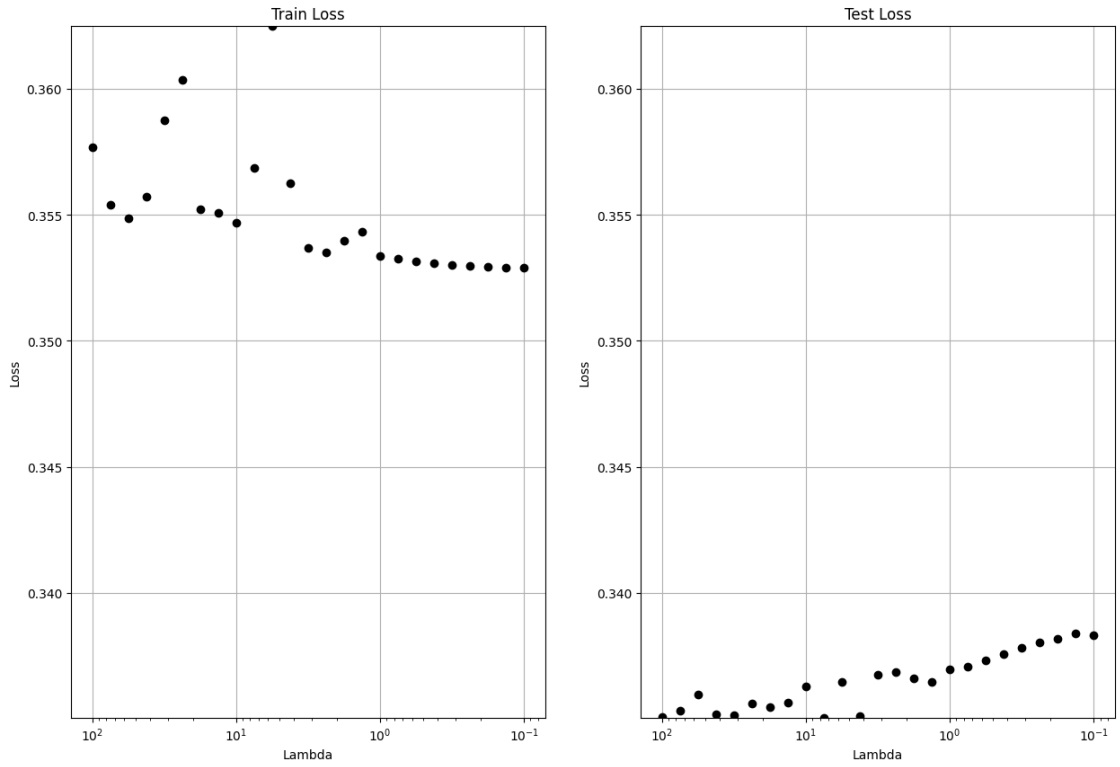
```
10 - univ
```



0.7 Run LASSO on MLP Partial Responses

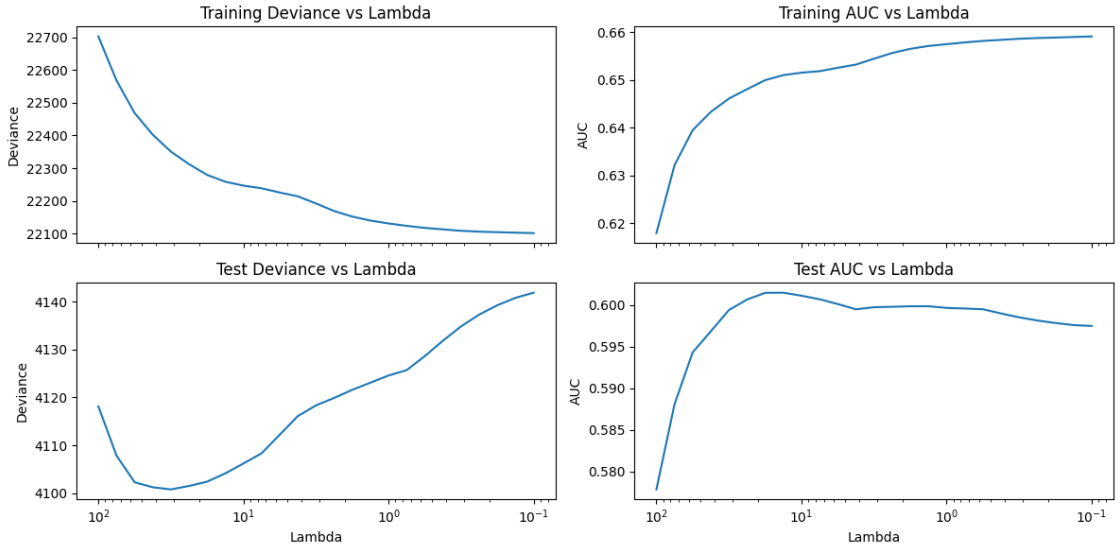
Note losses shown are normalized to loss at epoch 0, so not for comparing between hyperparameter sets.

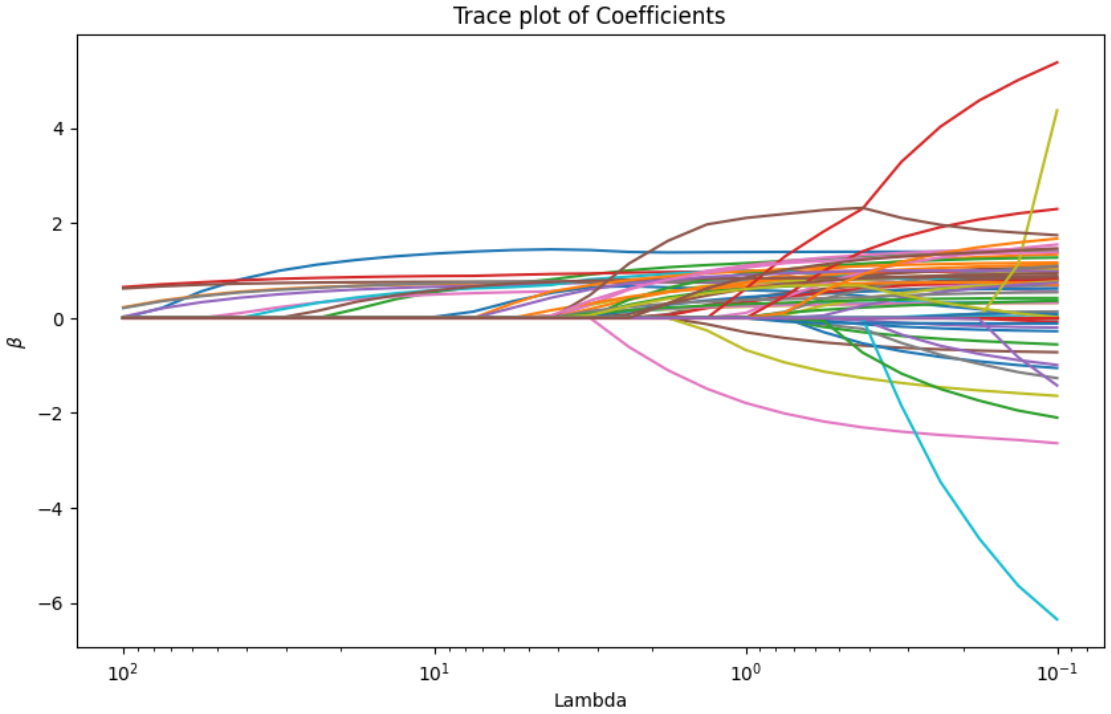
[13]:
```
num_lambdas = 25
verbose = True
lambdas, betas, trainAUC, testAUC, trainDev, testDev, glmPred =␣
  ↪prLASSO(F_covariates_train, F_covariates_test, y_train, y_test,␣
  ↪num_lambdas=num_lambdas, log_min_lambda=-1, verbose=verbose)
```




```
23 - Lambda 0.13335: Test AUC 0.5976, n_univ: 10, n_biv: 46
 0 - Lambda 100.00000: Train AUC 0.6179, Train Deviance 22702.7540, Test AUC
0.5778, Test Deviance 4118.1219, n_univ: 4, n_biv: 0
    -> Within 1 SD of maximum Test AUC
 1 - Lambda 74.98942: Train AUC 0.6322, Train Deviance 22568.0476, Test AUC
0.5881, Test Deviance 4107.8366, n_univ: 6, n_biv: 0
 2 - Lambda 56.23413: Train AUC 0.6395, Train Deviance 22468.4754, Test AUC
0.5943, Test Deviance 4102.2627, n_univ: 6, n_biv: 0
 3 - Lambda 42.16965: Train AUC 0.6433, Train Deviance 22402.3386, Test AUC
0.5968, Test Deviance 4101.2217, n_univ: 7, n_biv: 0
 4 - Lambda 31.62278: Train AUC 0.6461, Train Deviance 22350.5526, Test AUC
0.5994, Test Deviance 4100.7713, n_univ: 8, n_biv: 0
    -> Minimum Test Deviance at this lambda
 5 - Lambda 23.71374: Train AUC 0.6481, Train Deviance 22312.3397, Test AUC
0.6006, Test Deviance 4101.4939, n_univ: 8, n_biv: 1
 6 - Lambda 17.78279: Train AUC 0.6500, Train Deviance 22279.3315, Test AUC
0.6014, Test Deviance 4102.4011, n_univ: 9, n_biv: 1
 7 - Lambda 13.33521: Train AUC 0.6510, Train Deviance 22258.7204, Test AUC
0.6015, Test Deviance 4104.1231, n_univ: 9, n_biv: 1
    -> Maximum Test AUC at this lambda
 8 - Lambda 10.00000: Train AUC 0.6515, Train Deviance 22246.7457, Test AUC
```

```
0.6011, Test Deviance 4106.1941, n_univ: 9, n_biv: 1
 9 - Lambda 7.49894: Train AUC 0.6518, Train Deviance 22238.6558, Test AUC
0.6007, Test Deviance 4108.3147, n_univ: 10, n_biv: 1
10 - Lambda 5.62341: Train AUC 0.6526, Train Deviance 22225.6833, Test AUC
0.6001, Test Deviance 4112.2080, n_univ: 10, n_biv: 3
11 - Lambda 4.21697: Train AUC 0.6532, Train Deviance 22213.9767, Test AUC
0.5995, Test Deviance 4116.0973, n_univ: 10, n_biv: 4
    -> Within 1 SD of minimum Test Deviance
12 - Lambda 3.16228: Train AUC 0.6545, Train Deviance 22192.2278, Test AUC
0.5997, Test Deviance 4118.3271, n_univ: 11, n_biv: 10
13 - Lambda 2.37137: Train AUC 0.6556, Train Deviance 22169.0350, Test AUC
0.5998, Test Deviance 4119.8841, n_univ: 11, n_biv: 18
14 - Lambda 1.77828: Train AUC 0.6565, Train Deviance 22152.1313, Test AUC
0.5998, Test Deviance 4121.5825, n_univ: 11, n_biv: 21
15 - Lambda 1.33352: Train AUC 0.6571, Train Deviance 22140.0345, Test AUC
0.5998, Test Deviance 4123.0677, n_univ: 11, n_biv: 26
16 - Lambda 1.00000: Train AUC 0.6575, Train Deviance 22131.1267, Test AUC
0.5996, Test Deviance 4124.5715, n_univ: 11, n_biv: 28
17 - Lambda 0.74989: Train AUC 0.6579, Train Deviance 22123.7973, Test AUC
0.5996, Test Deviance 4125.6891, n_univ: 11, n_biv: 32
18 - Lambda 0.56234: Train AUC 0.6582, Train Deviance 22117.5850, Test AUC
0.5995, Test Deviance 4128.6313, n_univ: 11, n_biv: 37
19 - Lambda 0.42170: Train AUC 0.6584, Train Deviance 22113.0706, Test AUC
0.5990, Test Deviance 4131.8468, n_univ: 11, n_biv: 40
20 - Lambda 0.31623: Train AUC 0.6586, Train Deviance 22108.6833, Test AUC
0.5985, Test Deviance 4134.8363, n_univ: 11, n_biv: 43
21 - Lambda 0.23714: Train AUC 0.6588, Train Deviance 22106.0253, Test AUC
0.5981, Test Deviance 4137.2948, n_univ: 11, n_biv: 43
22 - Lambda 0.17783: Train AUC 0.6589, Train Deviance 22104.4897, Test AUC
0.5978, Test Deviance 4139.2495, n_univ: 11, n_biv: 43
23 - Lambda 0.13335: Train AUC 0.6590, Train Deviance 22103.0129, Test AUC
0.5976, Test Deviance 4140.7972, n_univ: 10, n_biv: 46
24 - Lambda 0.10000: Train AUC 0.6591, Train Deviance 22101.5875, Test AUC
0.5975, Test Deviance 4141.8601, n_univ: 10, n_biv: 46
```





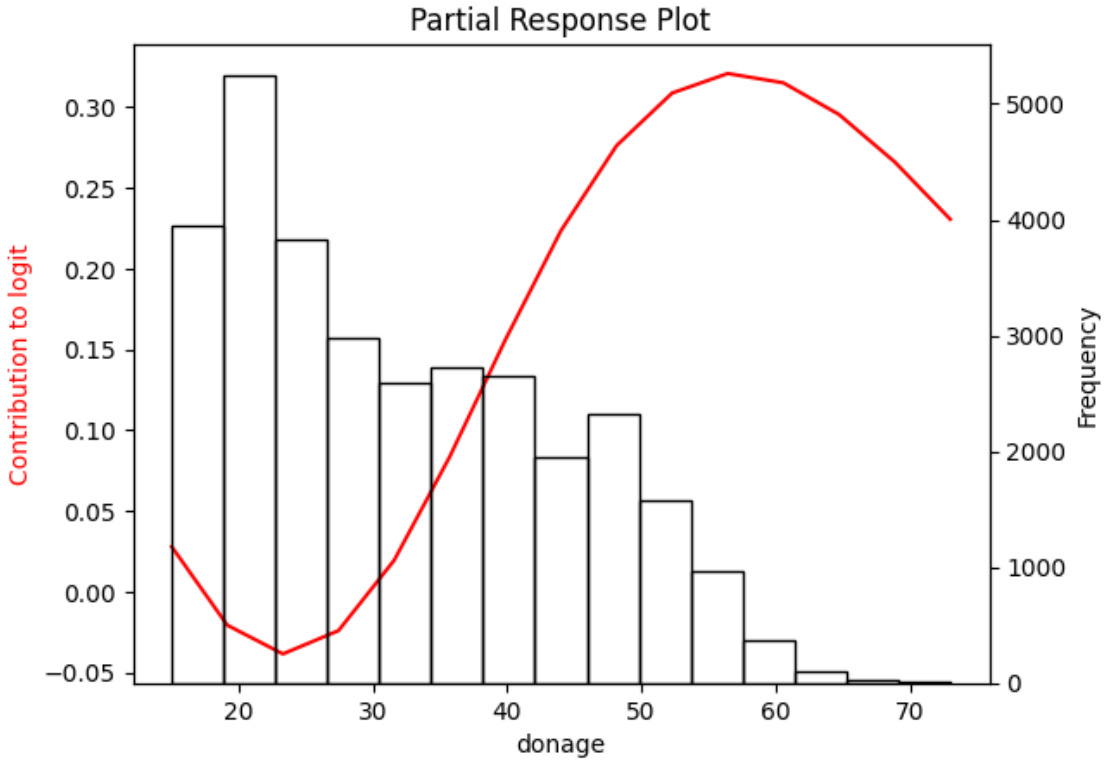
[14]:
```
# userLambda = selectLambda(lambdas, betas, testAUC, testDev,␣
↪x_train,bivariate_inputs)
userLambda = 7
lambdas[userLambda]
```

[14]: 13.33521432163324

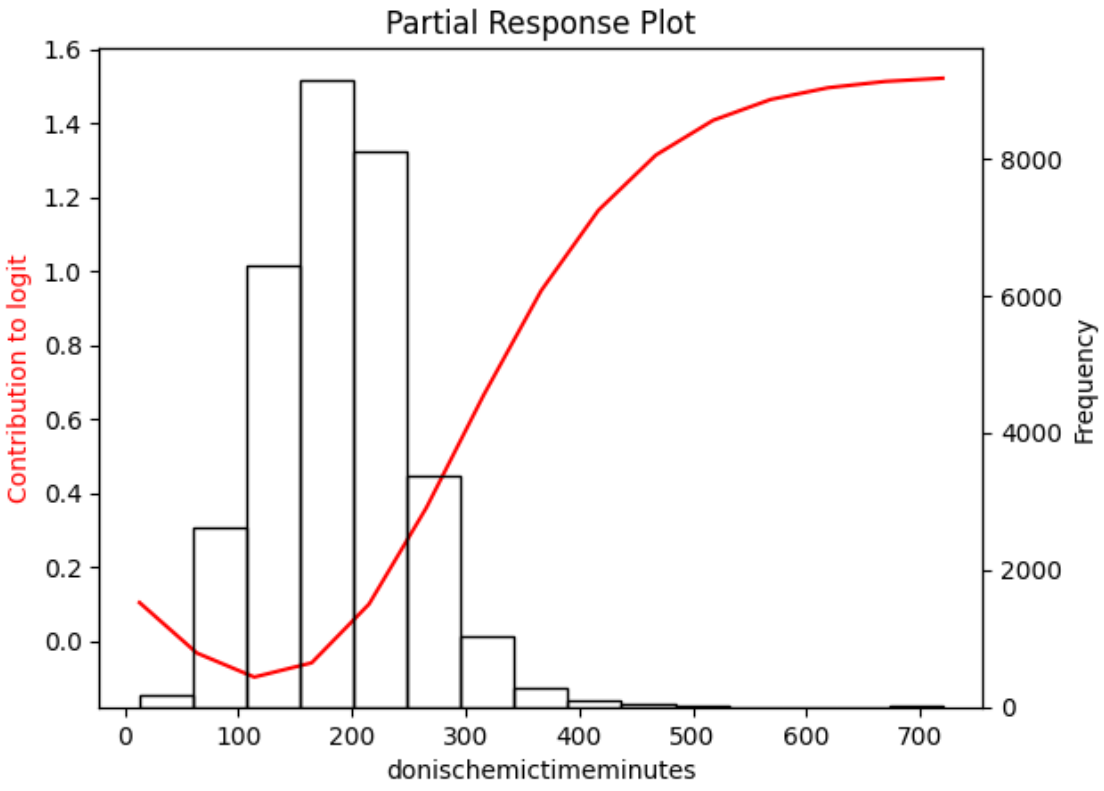
0.8 Plots of the MLP partial responses for the selected lambda

[15]:
```
prPlots(betas, userLambda, x_train0, x_train, data_train_test,␣
↪blackbox_weights, bivariate_inputs, n_steps = 15, sd_scale=2, method=method,␣
↪device=device)
```

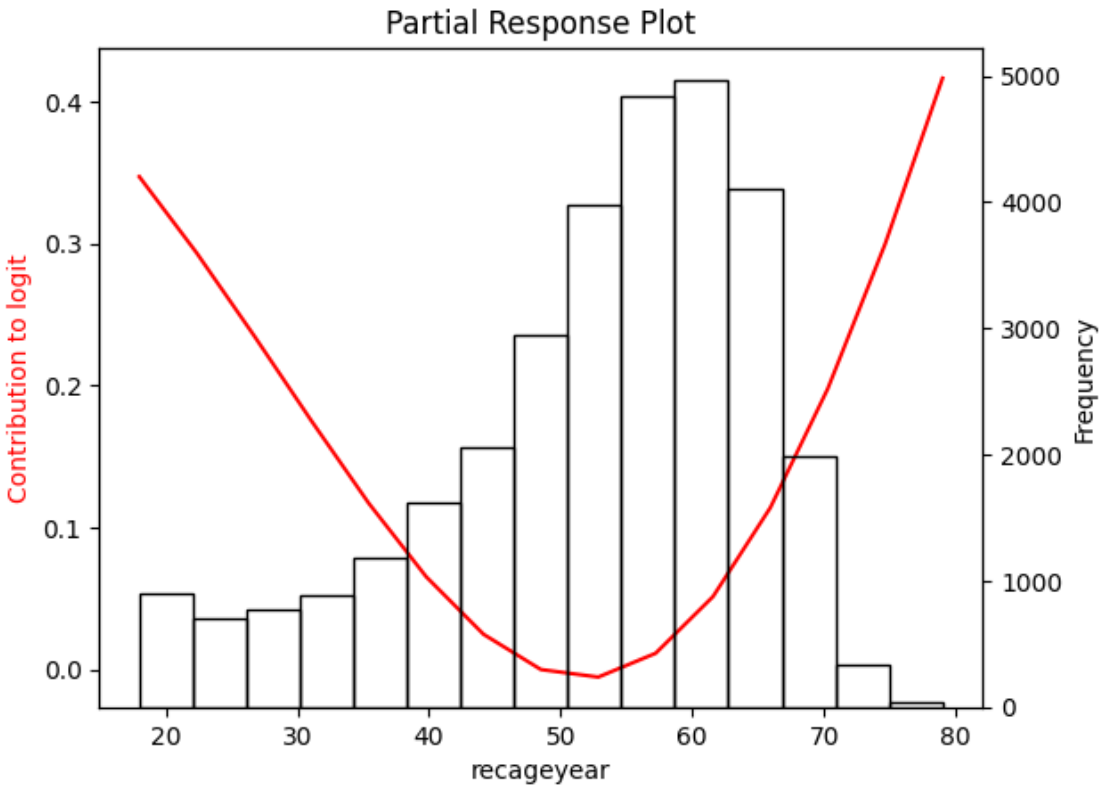
```
0 - univ
```




```
1 - univ
```


Partial Response Plot
Contribution to logit
Frequency
donischemictimeminutes
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
8000
6000
4000
2000
0
0
100
200
300
400
500
600
700

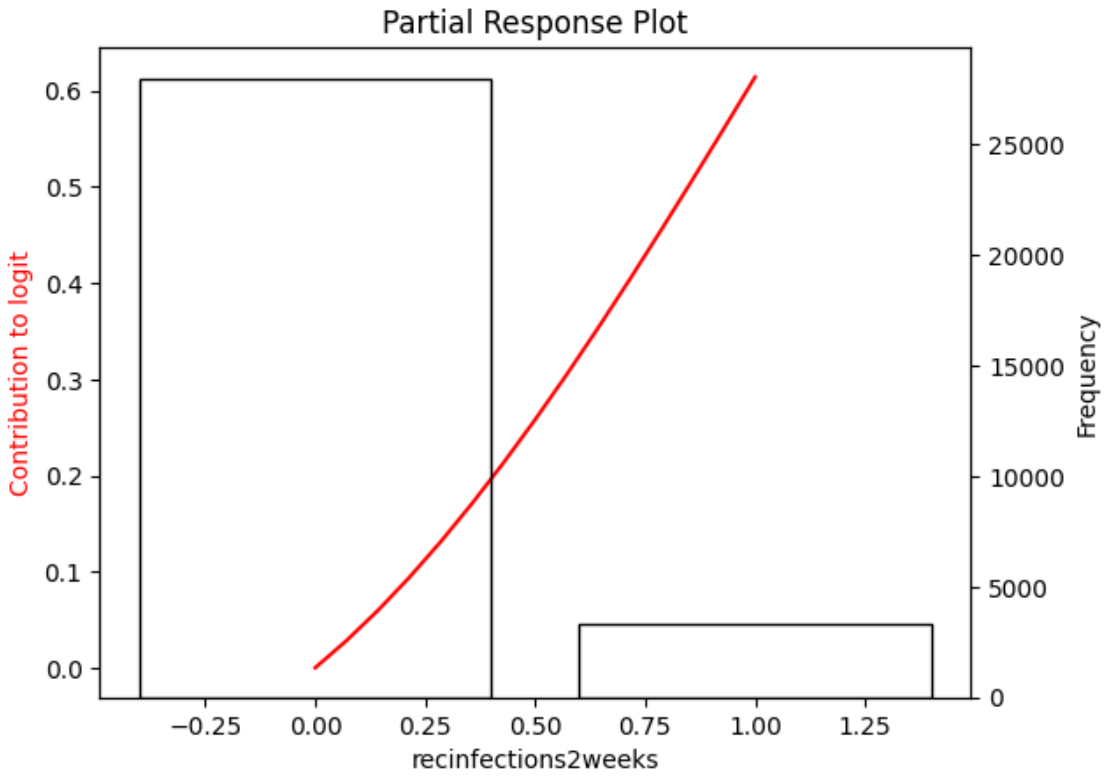
2 - univ


Partial Response Plot
Contribution to logit
Frequency
recageyear
0.0
0.1
0.2
0.3
0.4
0
1000
2000
3000
4000
5000
20
30
40
50
60
70
80

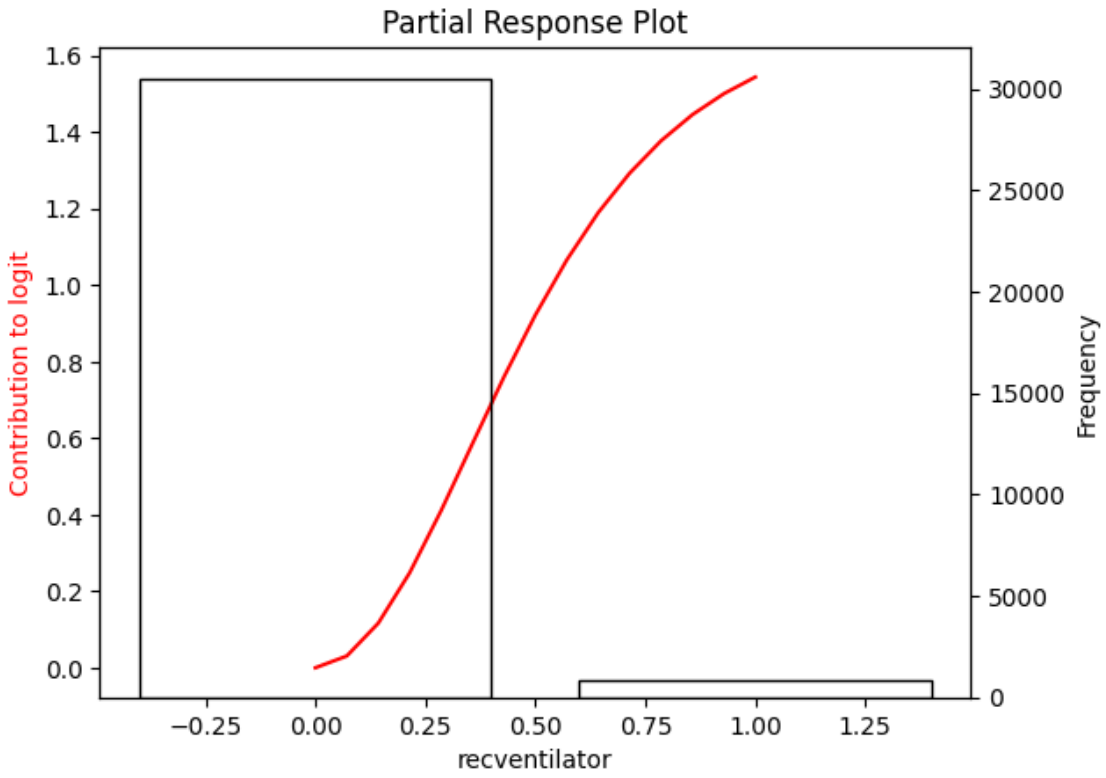
3 - univ


Partial Response Plot
Contribution to logit
Frequency
reccreatininemostrecent
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
17500
15000
12500
10000
7500
5000
2500
0
0
200
400
600
800
1000

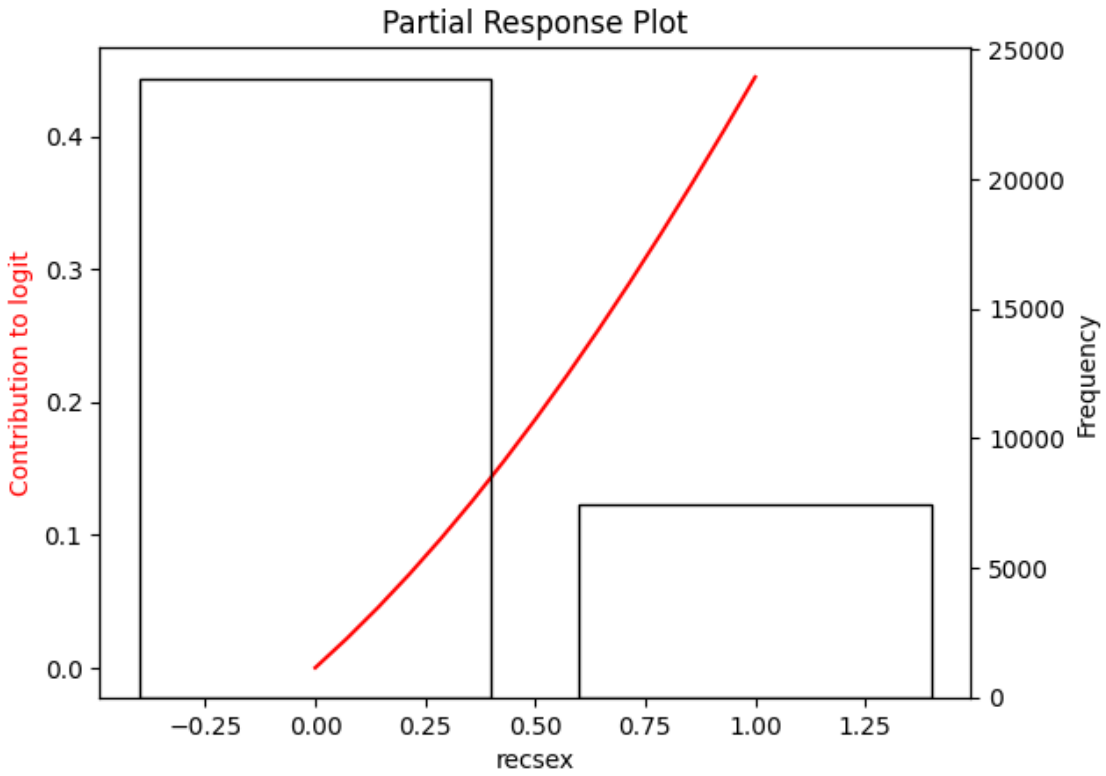
4 - univ


Partial Response Plot
Contribution to logit
Frequency
recinfections2weeks
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

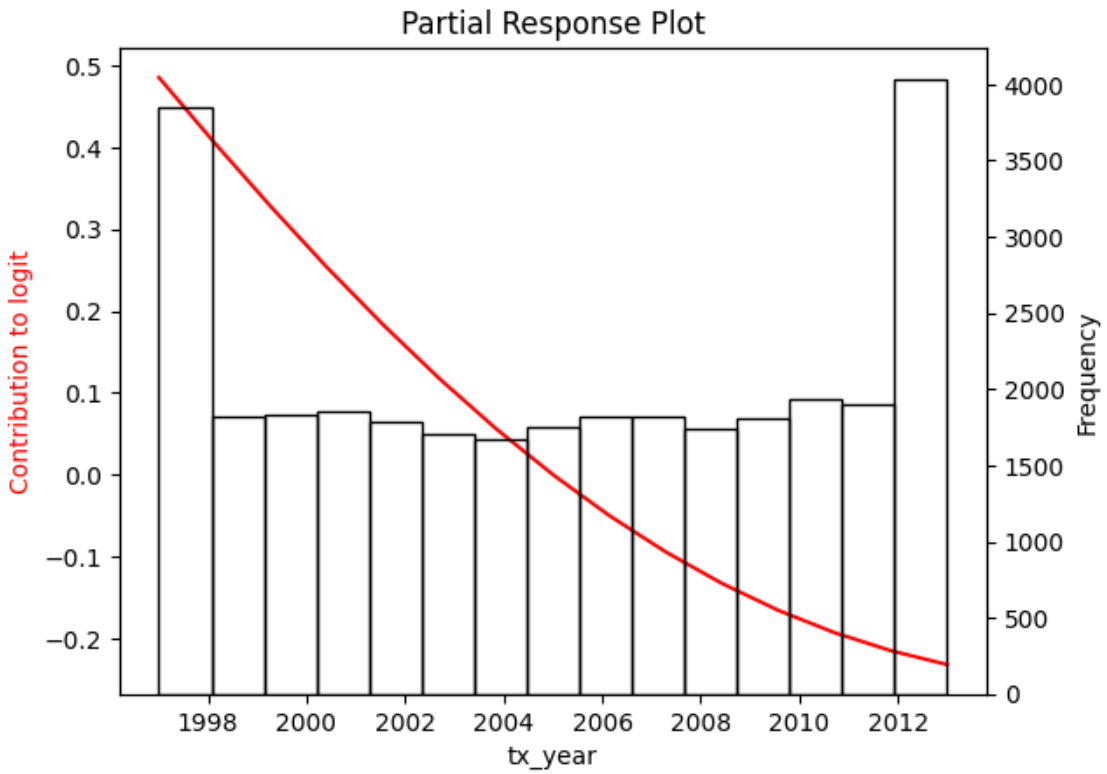
5 - univ


Partial Response Plot
Contribution to logit
Frequency
recventilator
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
30000
25000
20000
15000
10000
5000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

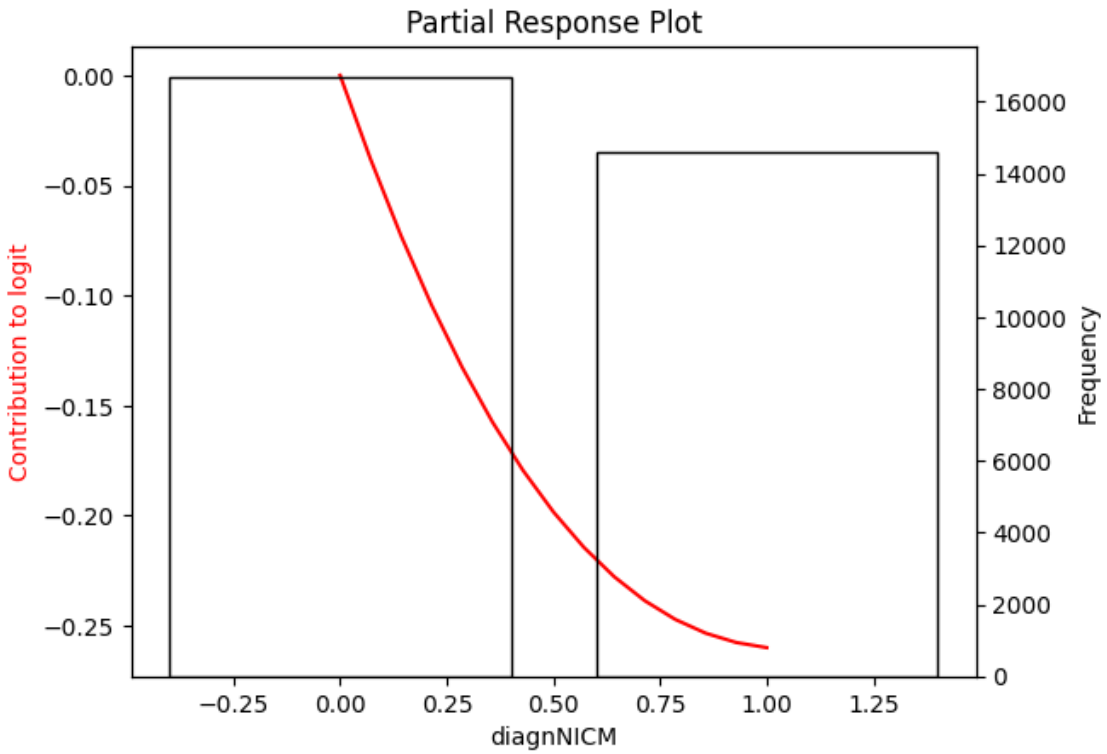
6 - univ

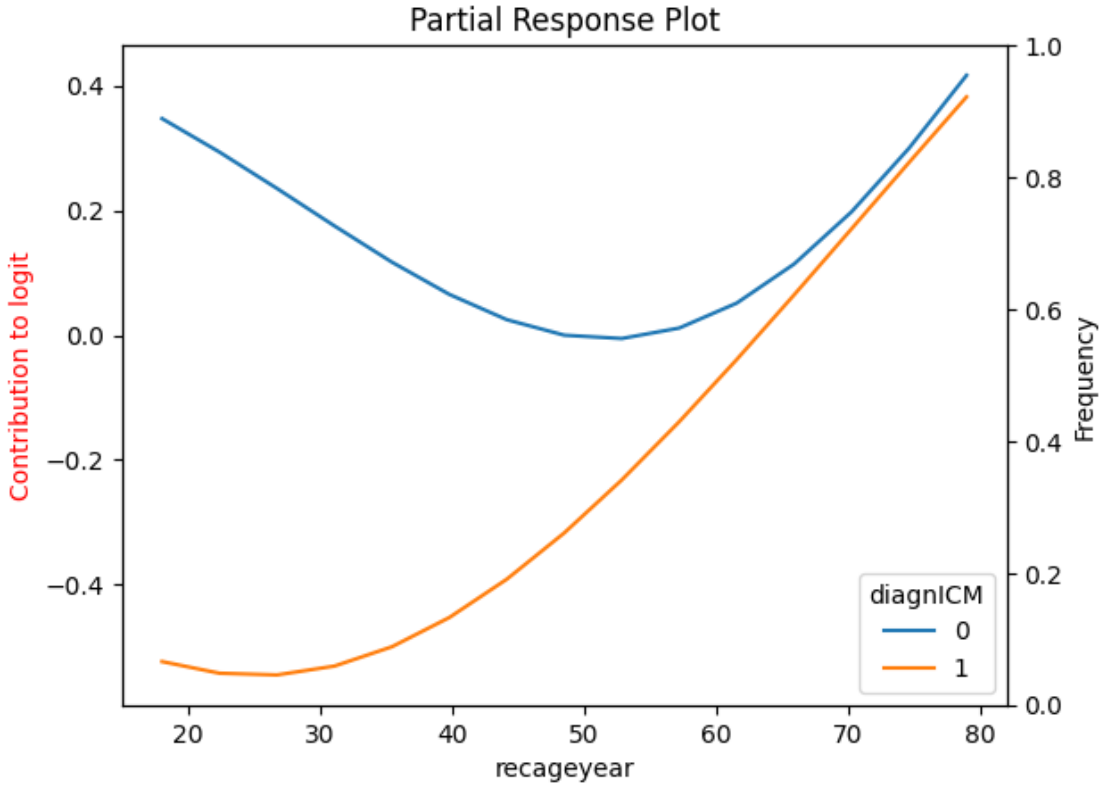

Partial Response Plot
Contribution to logit
Frequency
recsex
0.0
0.1
0.2
0.3
0.4
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

7 - univ


Partial Response Plot
Contribution to logit
Frequency
tx_year
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
4000
3500
3000
2500
2000
1500
1000
500
0
1998
2000
2002
2004
2006
2008
2010
2012

9 - univ

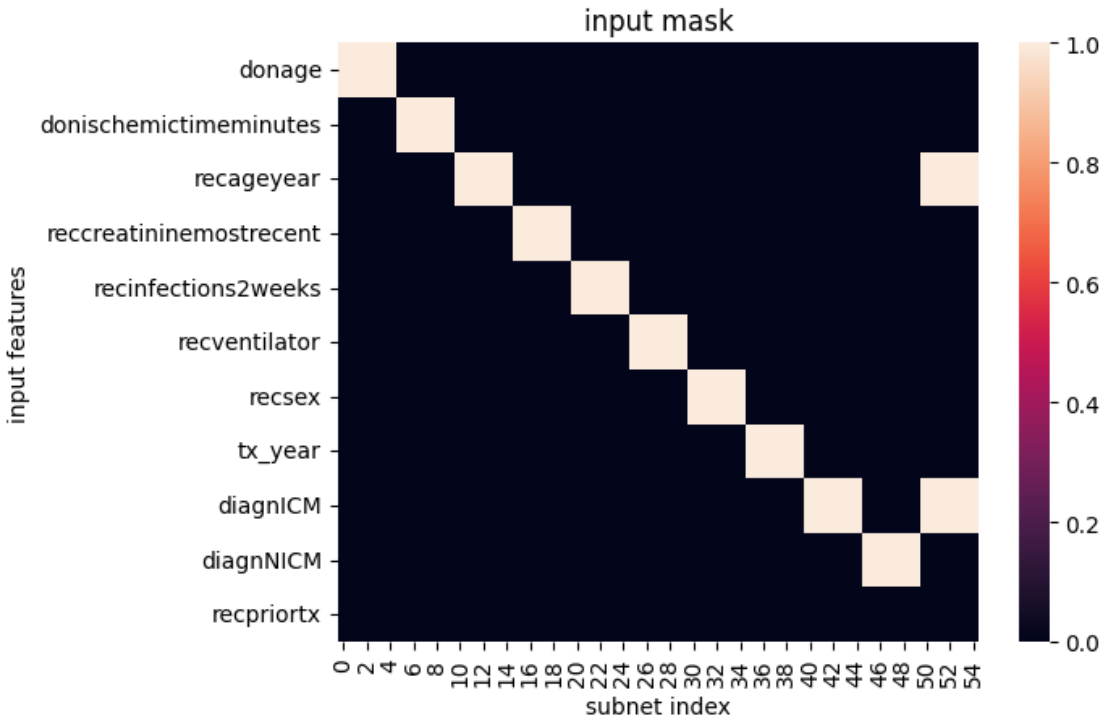

Partial Response Plot
Contribution to logit
Frequency
diagnNICM
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
16000
14000
12000
10000
8000
6000
4000
2000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25


Partial Response Plot
Contribution to logit
Frequency
recageyear
diagnICM
0
1
0.4
0.2
0.0
−0.2
−0.4
1.0
0.8
0.6
0.4
0.2
0.0
20
30
40
50
60
70
80

0.9 Train the Partial Response Network using the responses from the selected lambda

[16]:
```
mask, nPr = generate_mask(betas, userLambda, x_train,␣
↪bivariate_inputs,include_bivariate_as_univariate=True)
```

```
['donage', 'donischemictimeminutes', 'recageyear', 'reccreatininemostrecent',
'recinfections2weeks', 'recventilator', 'recsex', 'tx_year', 'diagnNICM',
'recageyear : diagnICM']
```



Note, that changing the patience from 10 to 100 caused the predicted probability histogram to more closely match previous iterations. However, inference seemed unusually slow.

After experimenting with hyperparameters, I changed from the default initialization (Kaiming, suitable for relu), to specifically Xavier (Glorot) with gains set according to the activation functions of each layer.

With this and the "default" params, training stopped after 227 (rather than 300) epochs.

[18]:
```
prn_params = {
    'n_hidden': nPr,
    'mask': mask,
    'subnet_nodes': 5,
```

```
    'iter': 10000,
    'lr': 0.05,
    'weight_decay': 0.00001,
    'tolerance': 0.0001,
    'patience': 100,
    'device': device,
    'seed': seed
}

prn = mlpmask_pytorch(x_train, y_train, x_test, y_test, **prn_params)
```

```
Epoch 0, Training loss 0.7491949796676636, Validation loss 0.5058018565177917
Epoch 1, Training loss 0.5208706855773926, Validation loss 0.3845042586326599
Epoch 2, Training loss 0.4087001085281372, Validation loss 0.34338873624801636
Epoch 3, Training loss 0.3777748942375183, Validation loss 0.3566133677959442
Epoch 4, Training loss 0.3964877128601074, Validation loss 0.38072434067726135
Epoch 5, Training loss 0.42082342505455017, Validation loss 0.39131689071655273
Epoch 6, Training loss 0.4279251992702484, Validation loss 0.38494932651519775
Epoch 7, Training loss 0.41638851165771484, Validation loss 0.36732038855552673
Epoch 8, Training loss 0.39412549138069153, Validation loss 0.34791669249534607
Epoch 9, Training loss 0.3729674220085144, Validation loss 0.33749744296073914
Epoch 10, Training loss 0.3656228184700012, Validation loss 0.34155094623565674
Epoch 11, Training loss 0.37698012590408325, Validation loss 0.3493284285068512
Epoch 12, Training loss 0.39084920287132263, Validation loss 0.34701046347618103
Epoch 13, Training loss 0.38845211267471313, Validation loss 0.3386439383029938
Epoch 14, Training loss 0.37404558062553406, Validation loss 0.33528995513916016
Epoch 15, Training loss 0.3619024157524109, Validation loss 0.3410446345806122
Epoch 16, Training loss 0.35944950580596924, Validation loss 0.35184338688850403
Epoch 17, Training loss 0.3643118143081665, Validation loss 0.36123892664909363
Epoch 18, Training loss 0.3701641261577606, Validation loss 0.3649636209011078
Epoch 19, Training loss 0.3723222315311432, Validation loss 0.362106591463089
Epoch 20, Training loss 0.3696381747722626, Validation loss 0.3544863164424896
Epoch 21, Training loss 0.3639782965183258, Validation loss 0.3455081284046173
Epoch 22, Training loss 0.3587774634361267, Validation loss 0.3386857509613037
Epoch 23, Training loss 0.35707536339759827, Validation loss 0.335803747177124
Epoch 24, Training loss 0.35936906933784485, Validation loss 0.3357851803302765
Epoch 25, Training loss 0.3628600239753723, Validation loss 0.33608564734458923
Epoch 26, Training loss 0.36388930678367615, Validation loss 0.33564338088035583
Epoch 27, Training loss 0.3615776300430298, Validation loss 0.335593044757843
Epoch 28, Training loss 0.3582203984260559, Validation loss 0.33725008368492126
Epoch 29, Training loss 0.3564668297767639, Validation loss 0.340413898229599
Epoch 30, Training loss 0.3570193946361542, Validation loss 0.3435826897621155
Epoch 31, Training loss 0.35869941115379333, Validation loss 0.3451804220676422
Epoch 32, Training loss 0.35986778140068054, Validation loss 0.3445168435573578
Epoch 33, Training loss 0.3596443831920624, Validation loss 0.3420269787311554
Epoch 34, Training loss 0.3582829236984253, Validation loss 0.3389121890068054
Epoch 35, Training loss 0.35680249333381653, Validation loss 0.33643555641174316
Epoch 36, Training loss 0.35620635747909546, Validation loss 0.33520403504371643
```

```
Epoch 37, Training loss 0.3567262887954712, Validation loss 0.33492323756217957
Epoch 38, Training loss 0.3576492965221405, Validation loss 0.33491653203964233
Epoch 39, Training loss 0.35799241065979004, Validation loss 0.3349255919456482
Epoch 40, Training loss 0.3574346601963043, Validation loss 0.3352886140346527
Epoch 41, Training loss 0.3565181791782379, Validation loss 0.33637866377830505
Epoch 42, Training loss 0.3559926152229309, Validation loss 0.3380572199821472
Epoch 43, Training loss 0.3561198115348816, Validation loss 0.3396962583065033
Epoch 44, Training loss 0.356579065322876, Validation loss 0.34061458706855774
Epoch 45, Training loss 0.35686594247817993, Validation loss 0.3404918313026428
Epoch 46, Training loss 0.3567216396331787, Validation loss 0.33949875831604004
Epoch 47, Training loss 0.35627955198287964, Validation loss 0.3381355106830597
Epoch 48, Training loss 0.35589656233787537, Validation loss 0.3369191288948059
Epoch 49, Training loss 0.3558361828327179, Validation loss 0.3361293077468872
Epoch 50, Training loss 0.3560543954372406, Validation loss 0.33577653765678406
Epoch 51, Training loss 0.356268048286438, Validation loss 0.3357725143432617
Epoch 52, Training loss 0.356241375207901, Validation loss 0.33608710765838623
Epoch 53, Training loss 0.35600021481513977, Validation loss 0.33671995997428894
Epoch 54, Training loss 0.35576486587524414, Validation loss 0.3375648260116577
Epoch 55, Training loss 0.35571014881134033, Validation loss 0.3383626639842987
Epoch 56, Training loss 0.3558168113231659, Validation loss 0.33881381154060364
Epoch 57, Training loss 0.3559291958808899, Validation loss 0.33874815702438354
Epoch 58, Training loss 0.3559182286262512, Validation loss 0.33821579813957214
Epoch 59, Training loss 0.35578784346580505, Validation loss 0.33744344115257263
Epoch 60, Training loss 0.35564911365509033, Validation loss 0.3366982340812683
Epoch 61, Training loss 0.35560449957847595, Validation loss 0.3361610174179077
Epoch 62, Training loss 0.35565441846847534, Validation loss 0.3358895778656006
Epoch 63, Training loss 0.3557095527648926, Validation loss 0.33587130904197693
Epoch 64, Training loss 0.35568973422050476, Validation loss 0.33608102798461914
Epoch 65, Training loss 0.35560187697410583, Validation loss 0.3364782929420471
Epoch 66, Training loss 0.35551899671554565, Validation loss 0.3369692265987396
Epoch 67, Training loss 0.35549667477607727, Validation loss 0.3374041020870209
Epoch 68, Training loss 0.3555222451686859, Validation loss 0.3376360833644867
Epoch 69, Training loss 0.35554012656211853, Validation loss 0.33759650588035583
Epoch 70, Training loss 0.35551339387893677, Validation loss 0.33732742071151733
Epoch 71, Training loss 0.35545578598976135, Validation loss 0.3369501531124115
Epoch 72, Training loss 0.3554099202156067, Validation loss 0.3365987241268158
Epoch 73, Training loss 0.3554005026817322, Validation loss 0.3363668918609619
Epoch 74, Training loss 0.35541170835494995, Validation loss 0.33629536628723145
Epoch 75, Training loss 0.3554096221923828, Validation loss 0.336385041475296
Epoch 76, Training loss 0.3553798794746399, Validation loss 0.3366064131259918
Epoch 77, Training loss 0.355339914560318, Validation loss 0.33689725399017334
Epoch 78, Training loss 0.35531502962112427, Validation loss 0.3371675908565521
Epoch 79, Training loss 0.35531070828437805, Validation loss 0.3373277187347412
Epoch 80, Training loss 0.35531044006347656, Validation loss 0.33732736110687256
Epoch 81, Training loss 0.35529693961143494, Validation loss 0.33717843890190125
Epoch 82, Training loss 0.3552703559398651, Validation loss 0.33694517612457275
Epoch 83, Training loss 0.35524487495422363, Validation loss 0.3367106020450592
Epoch 84, Training loss 0.3552315831184387, Validation loss 0.33654287457466125
```

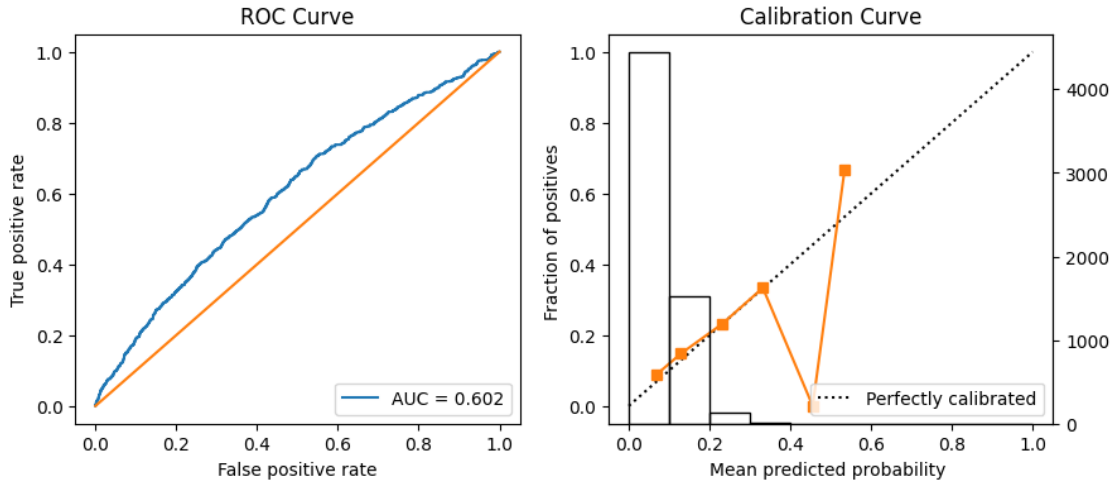
```
Epoch 85, Training loss 0.35522666573524475, Validation loss 0.3364787995815277
Epoch 86, Training loss 0.3552181124687195, Validation loss 0.3365233838558197
Epoch 87, Training loss 0.35520005226135254, Validation loss 0.3366541266441345
Epoch 88, Training loss 0.35517850518226624, Validation loss 0.33682578802108765
Epoch 89, Training loss 0.3551623821258545, Validation loss 0.3369801640510559
Epoch 90, Training loss 0.35515353083610535, Validation loss 0.3370654284954071
Epoch 91, Training loss 0.3551456332206726, Validation loss 0.3370566666126251
Epoch 92, Training loss 0.3551330864429474, Validation loss 0.336965411901474
Epoch 93, Training loss 0.3551167845726013, Validation loss 0.33683180809020996
Epoch 94, Training loss 0.3551023304462433, Validation loss 0.3367052376270294
Epoch 95, Training loss 0.3550926744937897, Validation loss 0.3366256356239319
Epoch 96, Training loss 0.35508519411087036, Validation loss 0.33661308884620667
Epoch 97, Training loss 0.3550756573677063, Validation loss 0.336665540933609
Epoch 98, Training loss 0.3550633192062378, Validation loss 0.3367617428302765
Epoch 99, Training loss 0.35505130887031555, Validation loss 0.3368672728538513
Epoch 100, Training loss 0.35504215955734253, Validation loss
0.33694565296173096
Epoch 101, Training loss 0.3550351560115814, Validation loss 0.3369717597961426
Epoch 102, Training loss 0.3550274968147278, Validation loss 0.3369414210319519
Epoch 103, Training loss 0.3550183176994324, Validation loss 0.3368719816207886
Epoch 104, Training loss 0.3550090193748474, Validation loss 0.3367931842803955
Epoch 105, Training loss 0.35500144958496094, Validation loss 0.3367346525192261
Epoch 106, Training loss 0.3549954891204834, Validation loss 0.3367157578468323
Epoch 107, Training loss 0.3549894392490387, Validation loss 0.3367408812046051
Epoch 108, Training loss 0.3549825549125671 4, Validation loss
0.33679965138435364
Epoch 109, Training loss 0.35497552156448364, Validation loss 0.336870938539505
Epoch 110, Training loss 0.35496950149536133, Validation loss 0.3369302451610565
Epoch 111, Training loss 0.35496464371681213, Validation loss 0.3369589149951935
Epoch 112, Training loss 0.3549599349498749, Validation loss 0.33695095777511597
Epoch 113, Training loss 0.3549547493457794, Validation loss 0.33691462874412537
Epoch 114, Training loss 0.3549495041370392, Validation loss 0.3368677496910095
Epoch 115, Training loss 0.35494503378868103, Validation loss
0.33682993054389954
Epoch 116, Training loss 0.3549412488937378, Validation loss 0.3368152976036072
Epoch 117, Training loss 0.3549376130104065, Validation loss 0.33682781457901
Epoch 118, Training loss 0.3549337685108185, Validation loss 0.3368615210056305
Epoch 119, Training loss 0.35492992401123047, Validation loss 0.3369028866291046
Epoch 120, Training loss 0.35492652654647827, Validation loss
0.33693641424179077
Epoch 121, Training loss 0.3549236059188843, Validation loss 0.33695051074028015
Epoch 122, Training loss 0.35492080450057983, Validation loss 0.3369424343109131
Epoch 123, Training loss 0.35491788387298584, Validation loss 0.3369179666042328
Epoch 124, Training loss 0.35491499304771423, Validation loss 0.3368891179561615
Epoch 125, Training loss 0.3549123704433441, Validation loss 0.33686843514442444
Epoch 126, Training loss 0.35491010546684265, Validation loss
0.33686426281929016
Epoch 127, Training loss 0.3549078106880188, Validation loss 0.3368780314922333
```

```
Epoch 128, Training loss 0.35490548610687256, Validation loss 0.3369044065475464
Epoch 129, Training loss 0.3549031615257263, Validation loss 0.3369336426258087
Epoch 130, Training loss 0.35490119457244873, Validation loss 0.3369556963443756
Epoch 131, Training loss 0.35489925742149353, Validation loss 0.3369642496109009
Epoch 132, Training loss 0.35489732027053833, Validation loss
0.33695870637893677
Epoch 133, Training loss 0.35489538311958313, Validation loss 0.3369443416595459
Epoch 134, Training loss 0.3548935353755951, Validation loss 0.3369292616844177
Epoch 135, Training loss 0.3548917770385742, Validation loss 0.33692100644111633
Epoch 136, Training loss 0.3548900783061981, Validation loss 0.3369234800338745
Epoch 137, Training loss 0.3548884093761444, Validation loss 0.33693575859069824
Early stopping! Epoch 137
Best model at epoch 37
```

0.10 Evaluate the Partial Response Network

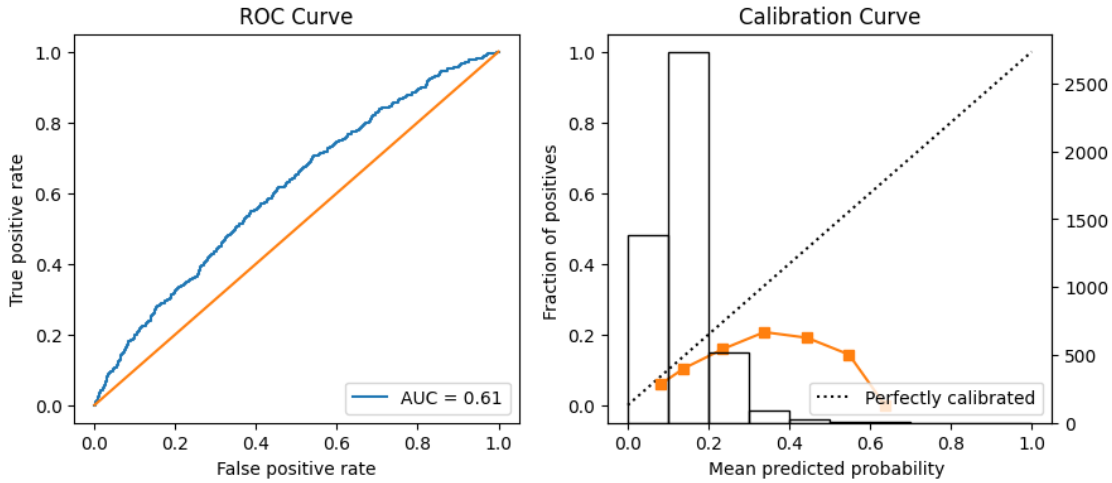
[19]:
```
# Get model inference
y_test_prn_pytorch = prn.predict(x_test_tensor, device=device).to('cpu').numpy()
y_val_prn_pytorch = prn.predict(x_val_tensor, device=device).to('cpu').numpy()
```

[20]:
```
prn_metrics_test = modelStats(y_test_prn_pytorch, y_test, y_train, auc_ci=True)
modelStats(y_val_prn_pytorch,y_val,y_train, auc_ci=True)
```




```
-------- Metrics --------
prevalence: 0.123
sensitivity: 0.235
specificity: 0.867
accuracy: 0.799
ppv: 0.174
auc score: 0.602
```

```
auc lower ci: 0.578
auc upper ci: 0.63
-------------------------
```




```
-------- Metrics --------
prevalence: 0.123
sensitivity: 0.639
specificity: 0.512
accuracy: 0.524
ppv: 0.126
auc score: 0.61
auc lower ci: 0.585
auc upper ci: 0.636
-------------------------
```

[20]:
```
{'prevalence': 0.123,
 'sensitivity': 0.639,
 'specificity': 0.512,
 'accuracy': 0.524,
 'ppv': 0.126,
 'auc score': 0.61,
 'auc lower ci': '0.585',
 'auc upper ci': '0.636'}
```

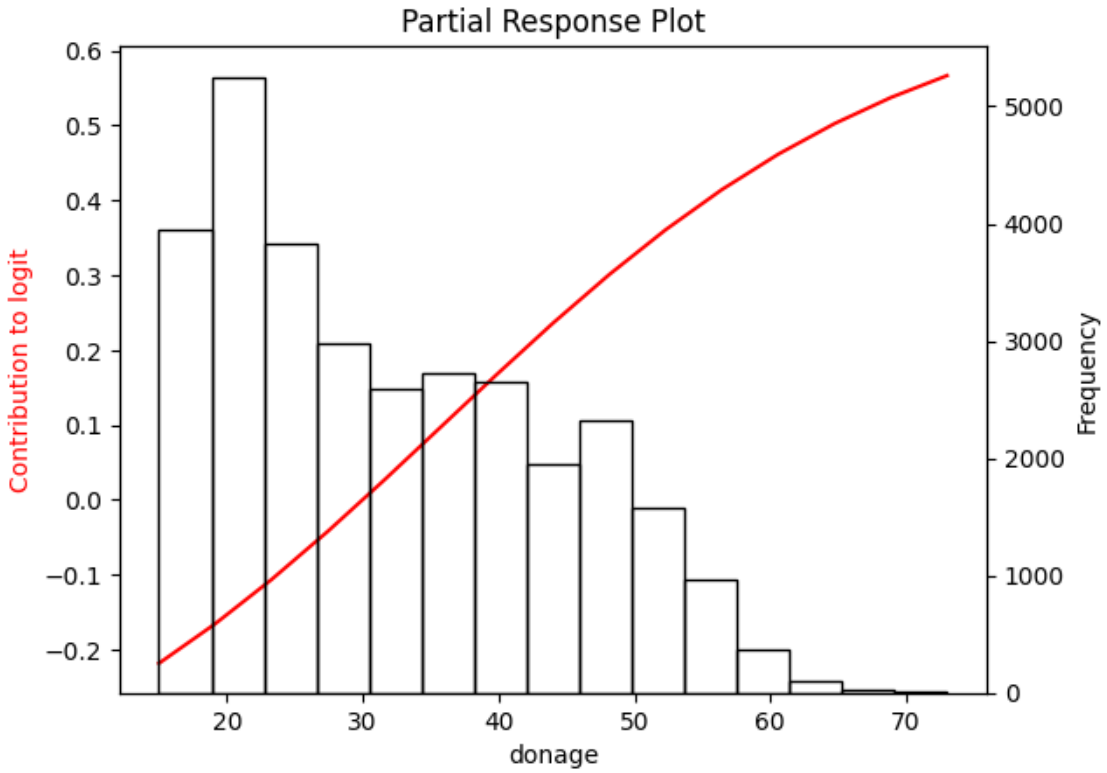
[21]:
```
if SAVE_MODELS:
    save_prn(prn, prn_params, prn_metrics_test, MODELS_DIR)
```

```
PRN model saved as prn_model_20240705_140928
```

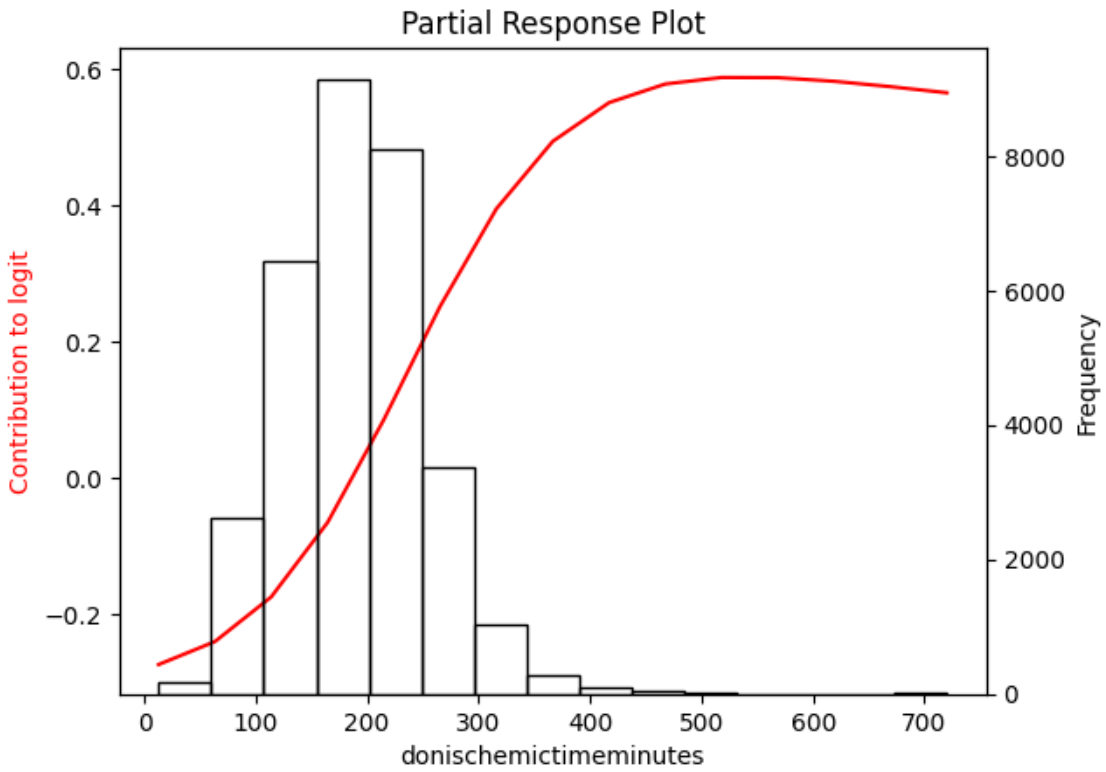
Quick prPlots check of all univariate features using , before proceeding with LASSO on the Partial Response Network.

[22]:
```
prPlots(betas_univariate, 0, x_train0, x_train, data_train_test, prn,␣
↪bivariate_inputs, n_steps = 15, sd_scale=2, method=method,device=device)
```

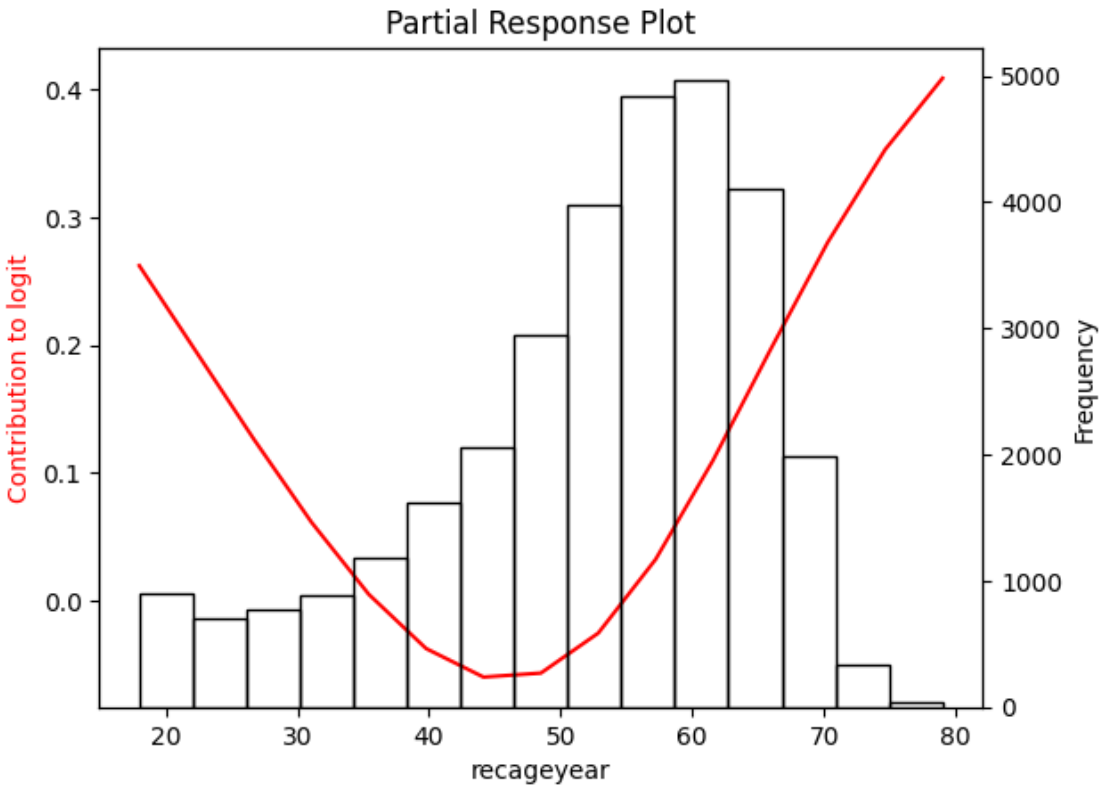
```
0 - univ
```



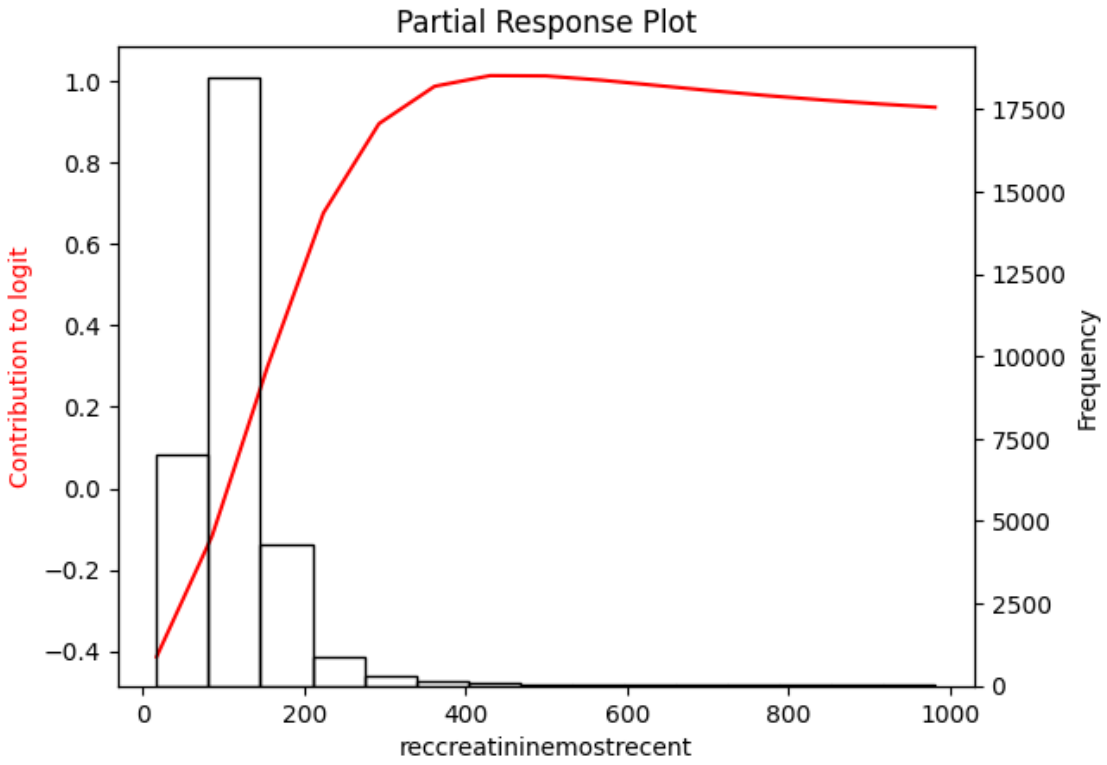

```
1 - univ
```


Partial Response Plot
Contribution to logit
Frequency
donischemictimeminutes
0.6
0.4
0.2
0.0
−0.2
8000
6000
4000
2000
0
0
100
200
300
400
500
600
700

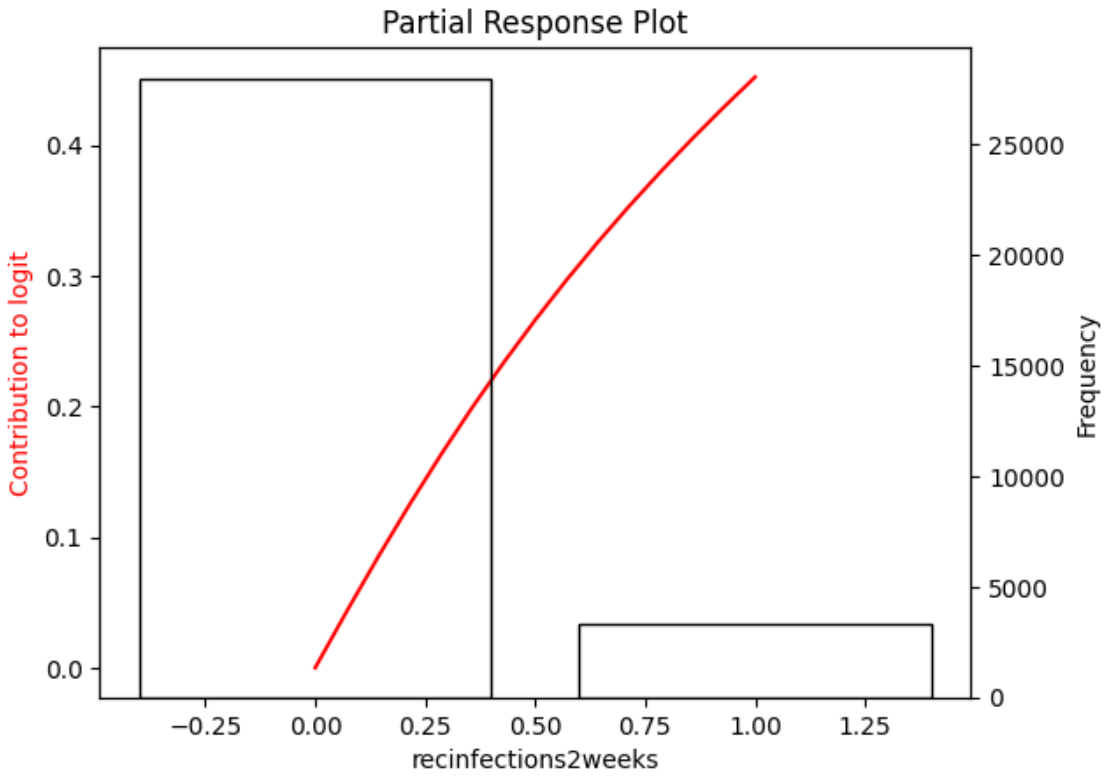
2 - univ


Partial Response Plot
Contribution to logit
Frequency
recageyear
0.4
0.3
0.2
0.1
0.0
5000
4000
3000
2000
1000
0
20
30
40
50
60
70
80

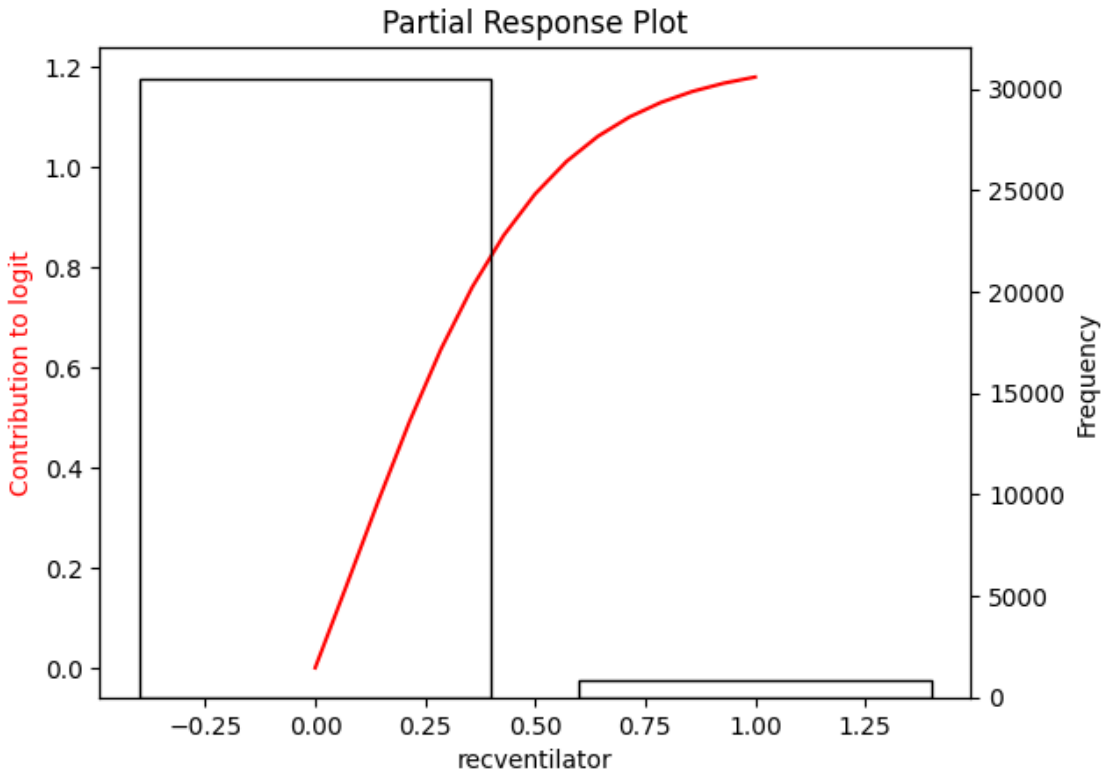
3 - univ


Partial Response Plot
Contribution to logit
Frequency
reccreatininemostrecent
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
17500
15000
12500
10000
7500
5000
2500
0
0
200
400
600
800
1000

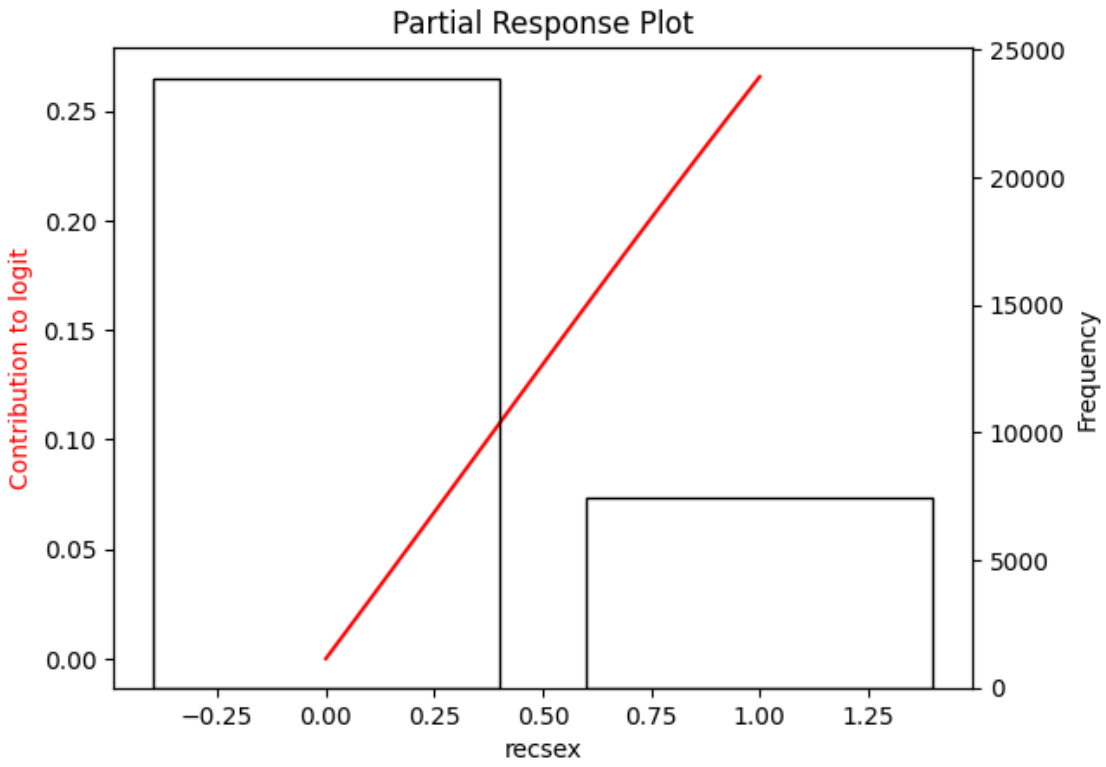
4 - univ


Partial Response Plot
Contribution to logit
Frequency
recinfections2weeks
0.0
0.1
0.2
0.3
0.4
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

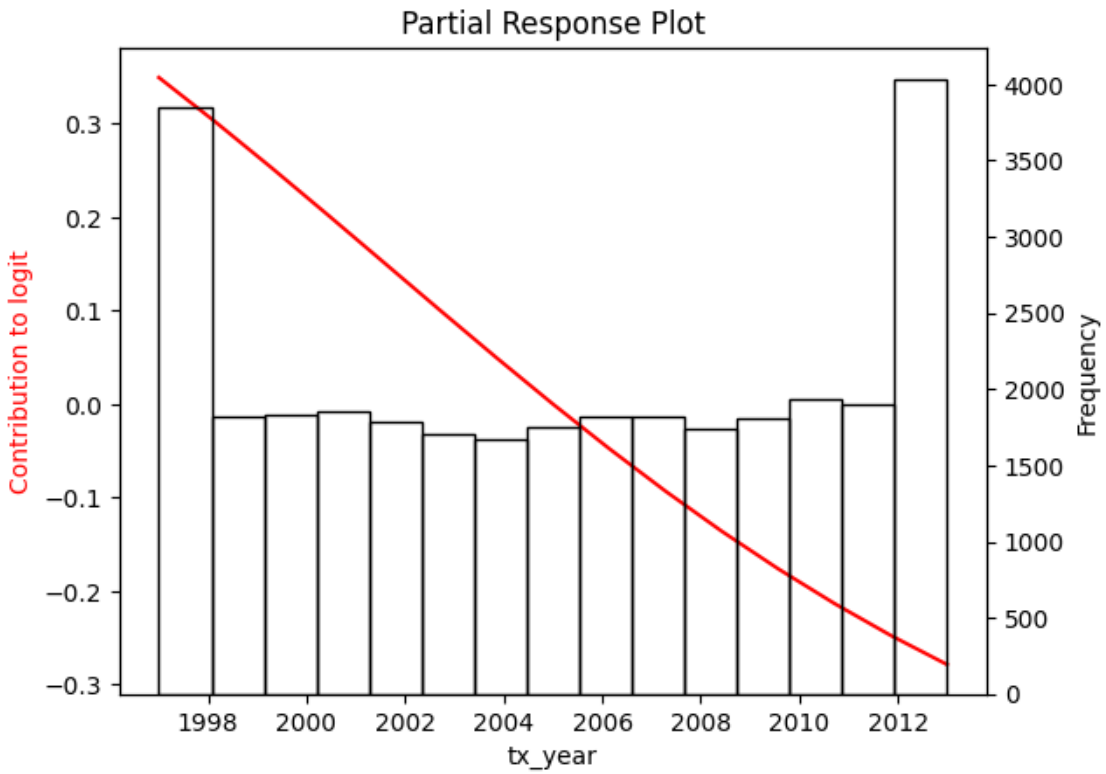
5 - univ


Partial Response Plot
Contribution to logit
Frequency
recventilator
1.2
1.0
0.8
0.6
0.4
0.2
0.0
30000
25000
20000
15000
10000
5000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

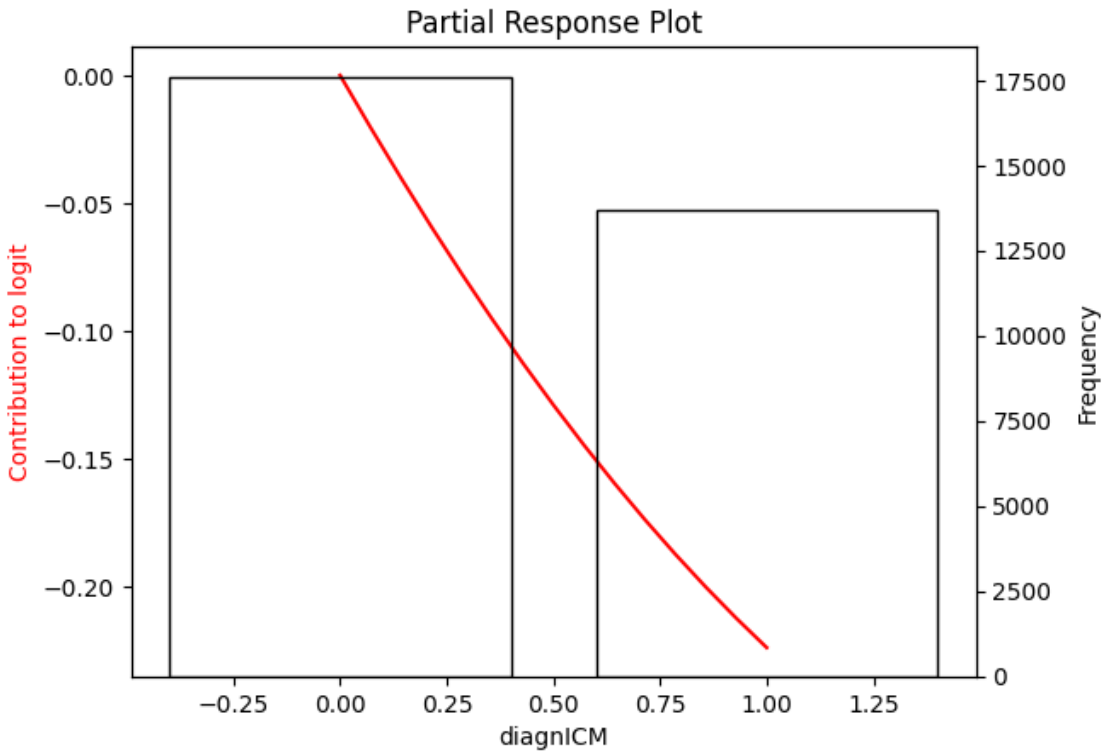
6 - univ


Partial Response Plot
Contribution to logit
Frequency
recsex
0.00
0.05
0.10
0.15
0.20
0.25
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

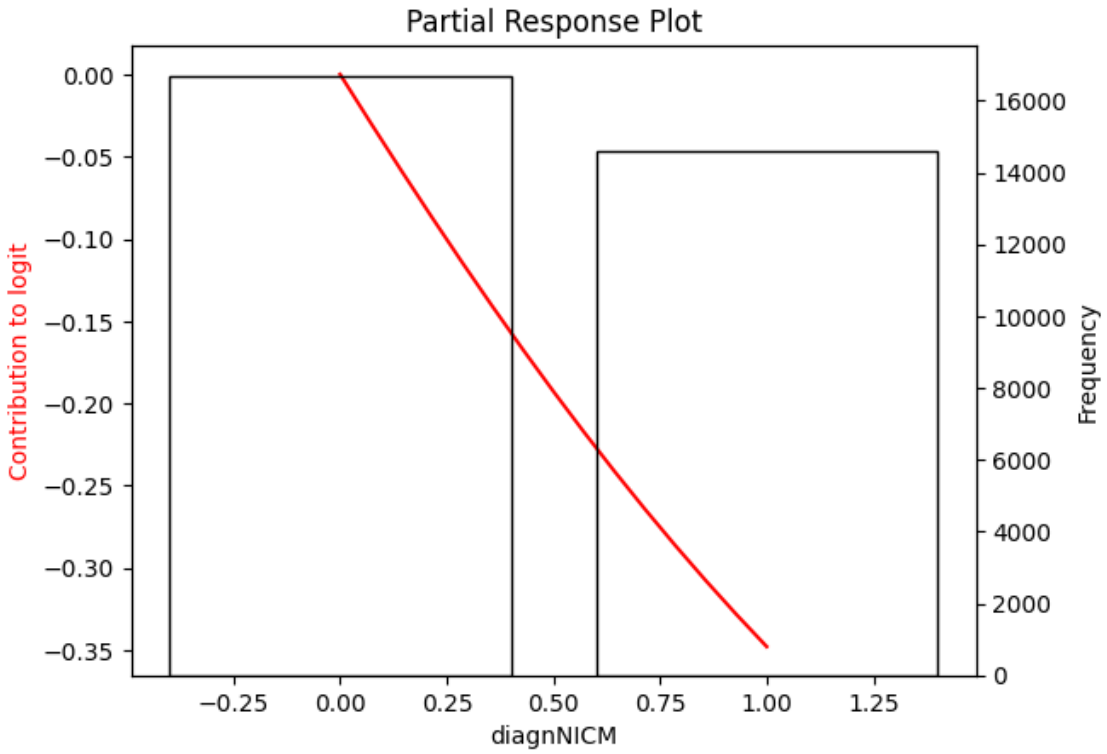
```
7 - univ
```


Partial Response Plot
Contribution to logit
Frequency
tx_year
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.3
4000
3500
3000
2500
2000
1500
1000
500
0
1998
2000
2002
2004
2006
2008
2010
2012

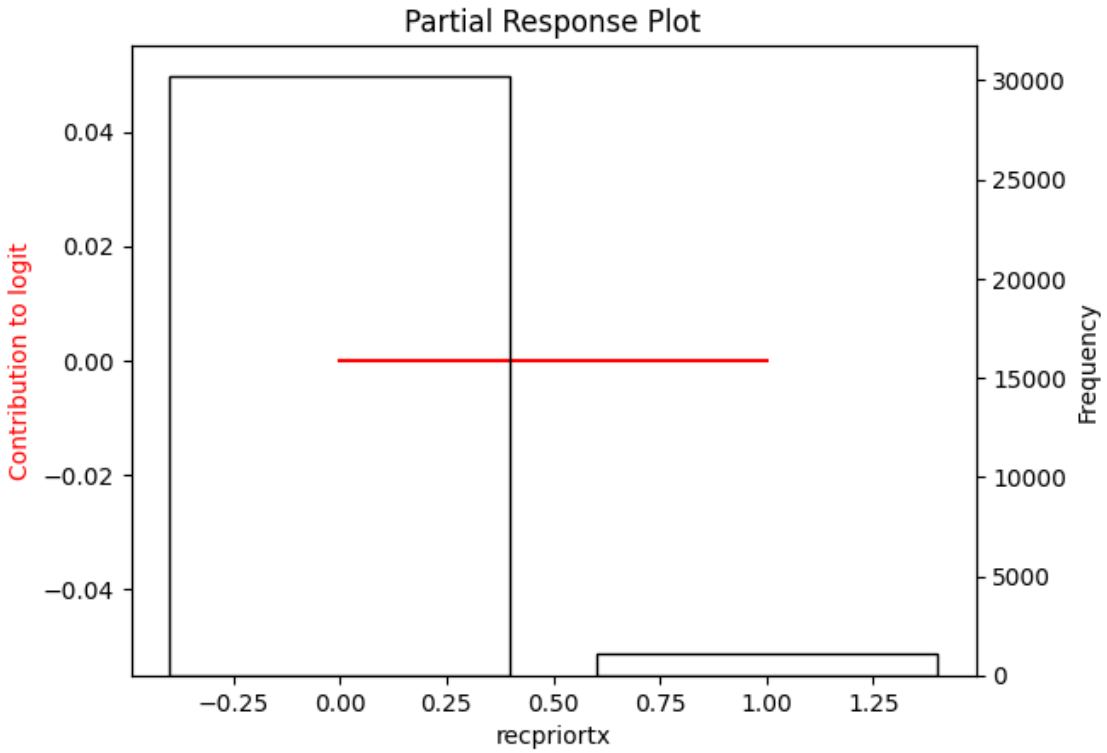
8 - univ


Partial Response Plot
Contribution to logit
Frequency
diagnICM
0.00
−0.05
−0.10
−0.15
−0.20
17500
15000
12500
10000
7500
5000
2500
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

9 - univ


Partial Response Plot
Contribution to logit
Frequency
diagnNICM
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
−0.30
−0.35
16000
14000
12000
10000
8000
6000
4000
2000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

10 - univ



0.11 LASSO on the Partial Response Network

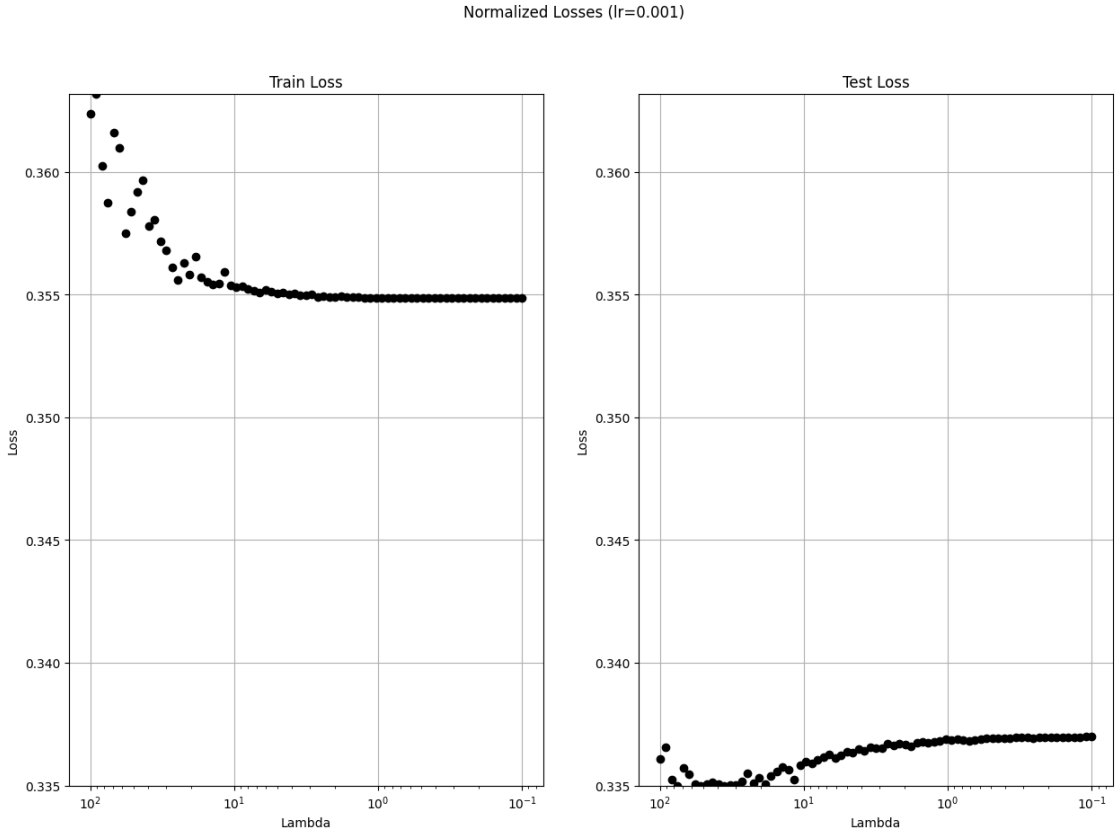
[23]:
```
prn_weights = get_model_weights_with_biases(prn)

F_covariates_train_prn, F_covariates_test_prn, bivariate_inputs_prn =␣
 ↪partialResponses(x_train, x_test, prn_weights, residual=False, method =␣
 ↪method,device=device);
```

```
Univariate Responses:
Column 0:
Column 1:
Column 2:
Column 3:
Column 4:
Column 5:
Column 6:
Column 7:
Column 8:
Column 9:
```

```
Column 10:
Bivariate Responses:
Columns 0 & 1:
Columns 0 & 2:
Columns 0 & 3:
Columns 0 & 4:
Columns 0 & 5:
Columns 0 & 6:
Columns 0 & 7:
Columns 0 & 8:
Columns 0 & 9:
Columns 0 & 10:
Columns 1 & 2:
Columns 1 & 3:
Columns 1 & 4:
Columns 1 & 5:
Columns 1 & 6:
Columns 1 & 7:
Columns 1 & 8:
Columns 1 & 9:
Columns 1 & 10:
Columns 2 & 3:
Columns 2 & 4:
Columns 2 & 5:
Columns 2 & 6:
Columns 2 & 7:
Columns 2 & 8:
Columns 2 & 9:
Columns 2 & 10:
Columns 3 & 4:
Columns 3 & 5:
Columns 3 & 6:
Columns 3 & 7:
Columns 3 & 8:
Columns 3 & 9:
Columns 3 & 10:
Columns 4 & 5:
Columns 4 & 6:
Columns 4 & 7:
Columns 4 & 8:
Columns 4 & 9:
Columns 4 & 10:
Columns 5 & 6:
Columns 5 & 7:
Columns 5 & 8:
Columns 5 & 9:
Columns 5 & 10:
Columns 6 & 7:
```

```
Columns 6 & 8:
Columns 6 & 9:
Columns 6 & 10:
Columns 7 & 8:
Columns 7 & 9:
Columns 7 & 10:
Columns 8 & 9:
Columns 8 & 10:
Columns 9 & 10:
Residual
```

[24]:
```
lambdas_prn, betas_prn, trainAUC_prn, testAUC_prn, trainDev_prn, testDev_prn,␣
↪glmPred_prn = prLASSO(F_covariates_train_prn, F_covariates_test_prn,y_train,␣
↪y_test, num_lambdas=75, log_max_lambda=2,log_min_lambda=-1,verbose=True)
```



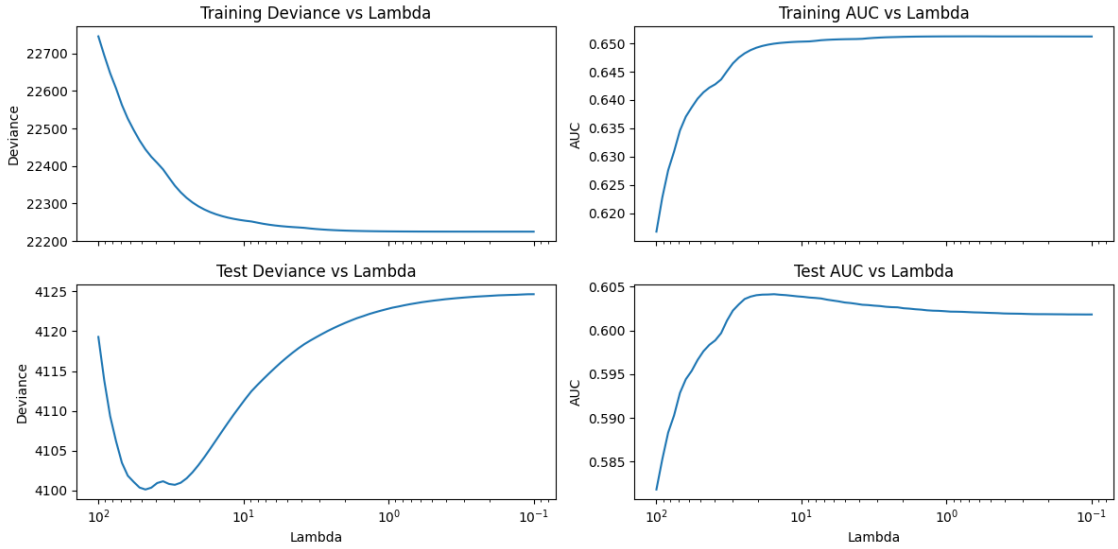

```
73 - Lambda 0.10978: Test AUC 0.6018, n_univ: 10, n_biv: 1
 0 - Lambda 100.00000: Train AUC 0.6168, Train Deviance 22745.3330, Test AUC
0.5818, Test Deviance 4119.2754, n_univ: 4, n_biv: 0
    -> Within 1 SD of maximum Test AUC
 1 - Lambda 91.08764: Train AUC 0.6228, Train Deviance 22694.6960, Test AUC
0.5853, Test Deviance 4113.8151, n_univ: 4, n_biv: 0
```

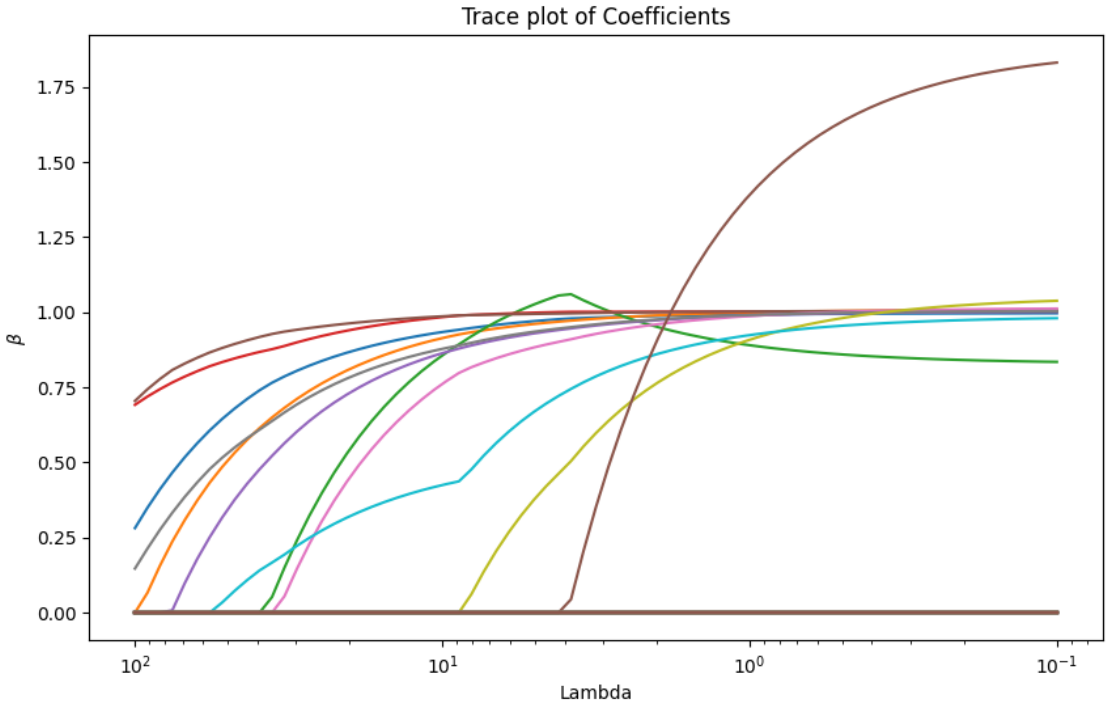
```
 2 - Lambda 82.96959: Train AUC 0.6276, Train Deviance 22647.3210, Test AUC
0.5883, Test Deviance 4109.3436, n_univ: 5, n_biv: 0
    -> Within 1 SD of minimum Test Deviance
 3 - Lambda 75.57504: Train AUC 0.6309, Train Deviance 22606.9829, Test AUC
0.5903, Test Deviance 4106.2062, n_univ: 5, n_biv: 0
 4 - Lambda 68.83952: Train AUC 0.6346, Train Deviance 22562.6724, Test AUC
0.5929, Test Deviance 4103.4870, n_univ: 5, n_biv: 0
 5 - Lambda 62.70430: Train AUC 0.6371, Train Deviance 22526.1738, Test AUC
0.5944, Test Deviance 4101.8651, n_univ: 6, n_biv: 0
 6 - Lambda 57.11586: Train AUC 0.6387, Train Deviance 22495.9789, Test AUC
0.5954, Test Deviance 4101.0876, n_univ: 6, n_biv: 0
 7 - Lambda 52.02549: Train AUC 0.6402, Train Deviance 22468.0526, Test AUC
0.5966, Test Deviance 4100.3778, n_univ: 6, n_biv: 0
 8 - Lambda 47.38880: Train AUC 0.6414, Train Deviance 22444.2386, Test AUC
0.5976, Test Deviance 4100.1325, n_univ: 6, n_biv: 0
    -> Minimum Test Deviance at this lambda
 9 - Lambda 43.16534: Train AUC 0.6422, Train Deviance 22424.4982, Test AUC
0.5984, Test Deviance 4100.3634, n_univ: 7, n_biv: 0
10 - Lambda 39.31829: Train AUC 0.6428, Train Deviance 22408.1175, Test AUC
0.5989, Test Deviance 4100.9447, n_univ: 7, n_biv: 0
11 - Lambda 35.81410: Train AUC 0.6436, Train Deviance 22390.8938, Test AUC
0.5997, Test Deviance 4101.1642, n_univ: 7, n_biv: 0
12 - Lambda 32.62222: Train AUC 0.6451, Train Deviance 22368.9682, Test AUC
0.6011, Test Deviance 4100.8306, n_univ: 8, n_biv: 0
13 - Lambda 29.71481: Train AUC 0.6465, Train Deviance 22347.5819, Test AUC
0.6023, Test Deviance 4100.7193, n_univ: 9, n_biv: 0
14 - Lambda 27.06652: Train AUC 0.6475, Train Deviance 22329.8349, Test AUC
0.6030, Test Deviance 4100.9810, n_univ: 9, n_biv: 0
15 - Lambda 24.65426: Train AUC 0.6482, Train Deviance 22315.0866, Test AUC
0.6036, Test Deviance 4101.5308, n_univ: 9, n_biv: 0
16 - Lambda 22.45698: Train AUC 0.6488, Train Deviance 22302.8423, Test AUC
0.6039, Test Deviance 4102.2867, n_univ: 9, n_biv: 0
17 - Lambda 20.45553: Train AUC 0.6492, Train Deviance 22292.6756, Test AUC
0.6040, Test Deviance 4103.1828, n_univ: 9, n_biv: 0
18 - Lambda 18.63246: Train AUC 0.6495, Train Deviance 22284.2308, Test AUC
0.6041, Test Deviance 4104.1719, n_univ: 9, n_biv: 0
19 - Lambda 16.97187: Train AUC 0.6498, Train Deviance 22277.2047, Test AUC
0.6041, Test Deviance 4105.2321, n_univ: 9, n_biv: 0
20 - Lambda 15.45928: Train AUC 0.6500, Train Deviance 22271.3807, Test AUC
0.6041, Test Deviance 4106.3141, n_univ: 9, n_biv: 0
    -> Maximum Test AUC at this lambda
21 - Lambda 14.08149: Train AUC 0.6501, Train Deviance 22266.5388, Test AUC
0.6041, Test Deviance 4107.4012, n_univ: 9, n_biv: 0
22 - Lambda 12.82650: Train AUC 0.6502, Train Deviance 22262.5168, Test AUC
0.6040, Test Deviance 4108.4748, n_univ: 9, n_biv: 0
23 - Lambda 11.68335: Train AUC 0.6502, Train Deviance 22259.1760, Test AUC
0.6040, Test Deviance 4109.5224, n_univ: 9, n_biv: 0
24 - Lambda 10.64209: Train AUC 0.6503, Train Deviance 22256.4051, Test AUC
```

```
0.6039, Test Deviance 4110.5197, n_univ: 9, n_biv: 0
25 - Lambda 9.69363: Train AUC 0.6503, Train Deviance 22254.0981, Test AUC
0.6038, Test Deviance 4111.5088, n_univ: 9, n_biv: 0
26 - Lambda 8.82970: Train AUC 0.6504, Train Deviance 22252.1845, Test AUC
0.6038, Test Deviance 4112.4417, n_univ: 9, n_biv: 0
27 - Lambda 8.04277: Train AUC 0.6504, Train Deviance 22249.1219, Test AUC
0.6037, Test Deviance 4113.2229, n_univ: 9, n_biv: 0
28 - Lambda 7.32597: Train AUC 0.6506, Train Deviance 22246.1131, Test AUC
0.6037, Test Deviance 4113.9627, n_univ: 10, n_biv: 0
29 - Lambda 6.67305: Train AUC 0.6506, Train Deviance 22243.6278, Test AUC
0.6035, Test Deviance 4114.6815, n_univ: 10, n_biv: 0
30 - Lambda 6.07832: Train AUC 0.6507, Train Deviance 22241.5731, Test AUC
0.6034, Test Deviance 4115.3773, n_univ: 10, n_biv: 0
31 - Lambda 5.53660: Train AUC 0.6507, Train Deviance 22239.8749, Test AUC
0.6033, Test Deviance 4116.0518, n_univ: 10, n_biv: 0
32 - Lambda 5.04316: Train AUC 0.6507, Train Deviance 22238.4675, Test AUC
0.6032, Test Deviance 4116.6808, n_univ: 10, n_biv: 0
33 - Lambda 4.59370: Train AUC 0.6508, Train Deviance 22237.3041, Test AUC
0.6031, Test Deviance 4117.2820, n_univ: 10, n_biv: 0
34 - Lambda 4.18429: Train AUC 0.6508, Train Deviance 22236.3412, Test AUC
0.6030, Test Deviance 4117.8443, n_univ: 10, n_biv: 0
35 - Lambda 3.81137: Train AUC 0.6508, Train Deviance 22235.2343, Test AUC
0.6029, Test Deviance 4118.3611, n_univ: 10, n_biv: 0
36 - Lambda 3.47169: Train AUC 0.6509, Train Deviance 22233.5205, Test AUC
0.6029, Test Deviance 4118.8076, n_univ: 10, n_biv: 1
37 - Lambda 3.16228: Train AUC 0.6510, Train Deviance 22232.0951, Test AUC
0.6028, Test Deviance 4119.2273, n_univ: 10, n_biv: 1
38 - Lambda 2.88044: Train AUC 0.6510, Train Deviance 22230.9092, Test AUC
0.6028, Test Deviance 4119.6347, n_univ: 10, n_biv: 1
39 - Lambda 2.62373: Train AUC 0.6511, Train Deviance 22229.9240, Test AUC
0.6027, Test Deviance 4120.0266, n_univ: 10, n_biv: 1
40 - Lambda 2.38989: Train AUC 0.6511, Train Deviance 22229.1052, Test AUC
0.6027, Test Deviance 4120.3891, n_univ: 10, n_biv: 1
41 - Lambda 2.17690: Train AUC 0.6511, Train Deviance 22228.4241, Test AUC
0.6026, Test Deviance 4120.7280, n_univ: 10, n_biv: 1
42 - Lambda 1.98288: Train AUC 0.6512, Train Deviance 22227.8585, Test AUC
0.6025, Test Deviance 4121.0483, n_univ: 10, n_biv: 1
43 - Lambda 1.80616: Train AUC 0.6512, Train Deviance 22227.3884, Test AUC
0.6025, Test Deviance 4121.3509, n_univ: 10, n_biv: 1
44 - Lambda 1.64519: Train AUC 0.6512, Train Deviance 22226.9987, Test AUC
0.6024, Test Deviance 4121.6399, n_univ: 10, n_biv: 1
45 - Lambda 1.49857: Train AUC 0.6512, Train Deviance 22226.6744, Test AUC
0.6024, Test Deviance 4121.8806, n_univ: 10, n_biv: 1
46 - Lambda 1.36501: Train AUC 0.6512, Train Deviance 22226.4046, Test AUC
0.6023, Test Deviance 4122.1400, n_univ: 10, n_biv: 1
47 - Lambda 1.24335: Train AUC 0.6512, Train Deviance 22226.1808, Test AUC
0.6023, Test Deviance 4122.3601, n_univ: 10, n_biv: 1
48 - Lambda 1.13254: Train AUC 0.6512, Train Deviance 22225.9949, Test AUC
```

```
0.6022, Test Deviance 4122.5656, n_univ: 10, n_biv: 1
49 - Lambda 1.03161: Train AUC 0.6512, Train Deviance 22225.8406, Test AUC
0.6022, Test Deviance 4122.7584, n_univ: 10, n_biv: 1
50 - Lambda 0.93966: Train AUC 0.6512, Train Deviance 22225.7126, Test AUC
0.6022, Test Deviance 4122.9456, n_univ: 10, n_biv: 1
51 - Lambda 0.85592: Train AUC 0.6512, Train Deviance 22225.6060, Test AUC
0.6021, Test Deviance 4123.0867, n_univ: 10, n_biv: 1
52 - Lambda 0.77964: Train AUC 0.6512, Train Deviance 22225.5181, Test AUC
0.6021, Test Deviance 4123.2436, n_univ: 10, n_biv: 1
53 - Lambda 0.71015: Train AUC 0.6512, Train Deviance 22225.4447, Test AUC
0.6021, Test Deviance 4123.3852, n_univ: 10, n_biv: 1
54 - Lambda 0.64686: Train AUC 0.6512, Train Deviance 22225.3840, Test AUC
0.6021, Test Deviance 4123.5066, n_univ: 10, n_biv: 1
55 - Lambda 0.58921: Train AUC 0.6512, Train Deviance 22225.3335, Test AUC
0.6021, Test Deviance 4123.6318, n_univ: 10, n_biv: 1
56 - Lambda 0.53670: Train AUC 0.6512, Train Deviance 22225.2915, Test AUC
0.6020, Test Deviance 4123.7331, n_univ: 10, n_biv: 1
57 - Lambda 0.48887: Train AUC 0.6512, Train Deviance 22225.2565, Test AUC
0.6020, Test Deviance 4123.8315, n_univ: 10, n_biv: 1
58 - Lambda 0.44530: Train AUC 0.6512, Train Deviance 22225.2279, Test AUC
0.6020, Test Deviance 4123.9166, n_univ: 10, n_biv: 1
59 - Lambda 0.40561: Train AUC 0.6512, Train Deviance 22225.2038, Test AUC
0.6019, Test Deviance 4124.0062, n_univ: 10, n_biv: 1
60 - Lambda 0.36946: Train AUC 0.6512, Train Deviance 22225.1839, Test AUC
0.6019, Test Deviance 4124.0766, n_univ: 10, n_biv: 1
61 - Lambda 0.33653: Train AUC 0.6512, Train Deviance 22225.1674, Test AUC
0.6019, Test Deviance 4124.1461, n_univ: 10, n_biv: 1
62 - Lambda 0.30654: Train AUC 0.6512, Train Deviance 22225.1536, Test AUC
0.6019, Test Deviance 4124.2069, n_univ: 10, n_biv: 1
63 - Lambda 0.27922: Train AUC 0.6512, Train Deviance 22225.1423, Test AUC
0.6019, Test Deviance 4124.2639, n_univ: 10, n_biv: 1
64 - Lambda 0.25433: Train AUC 0.6512, Train Deviance 22225.1329, Test AUC
0.6019, Test Deviance 4124.3206, n_univ: 10, n_biv: 1
65 - Lambda 0.23167: Train AUC 0.6512, Train Deviance 22225.1250, Test AUC
0.6019, Test Deviance 4124.3629, n_univ: 10, n_biv: 1
66 - Lambda 0.21102: Train AUC 0.6512, Train Deviance 22225.1185, Test AUC
0.6019, Test Deviance 4124.4032, n_univ: 10, n_biv: 1
67 - Lambda 0.19221: Train AUC 0.6512, Train Deviance 22225.1132, Test AUC
0.6019, Test Deviance 4124.4462, n_univ: 10, n_biv: 1
68 - Lambda 0.17508: Train AUC 0.6512, Train Deviance 22225.1086, Test AUC
0.6019, Test Deviance 4124.4920, n_univ: 10, n_biv: 1
69 - Lambda 0.15948: Train AUC 0.6512, Train Deviance 22225.1049, Test AUC
0.6019, Test Deviance 4124.5162, n_univ: 10, n_biv: 1
70 - Lambda 0.14527: Train AUC 0.6512, Train Deviance 22225.1019, Test AUC
0.6018, Test Deviance 4124.5440, n_univ: 10, n_biv: 1
71 - Lambda 0.13232: Train AUC 0.6512, Train Deviance 22225.0993, Test AUC
0.6018, Test Deviance 4124.5619, n_univ: 10, n_biv: 1
72 - Lambda 0.12053: Train AUC 0.6512, Train Deviance 22225.0972, Test AUC
```

```
0.6018, Test Deviance 4124.5983, n_univ: 10, n_biv: 1
73 - Lambda 0.10978: Train AUC 0.6512, Train Deviance 22225.0954, Test AUC
0.6018, Test Deviance 4124.6295, n_univ: 10, n_biv: 1
74 - Lambda 0.10000: Train AUC 0.6512, Train Deviance 22225.0940, Test AUC
0.6018, Test Deviance 4124.6350, n_univ: 10, n_biv: 1
```

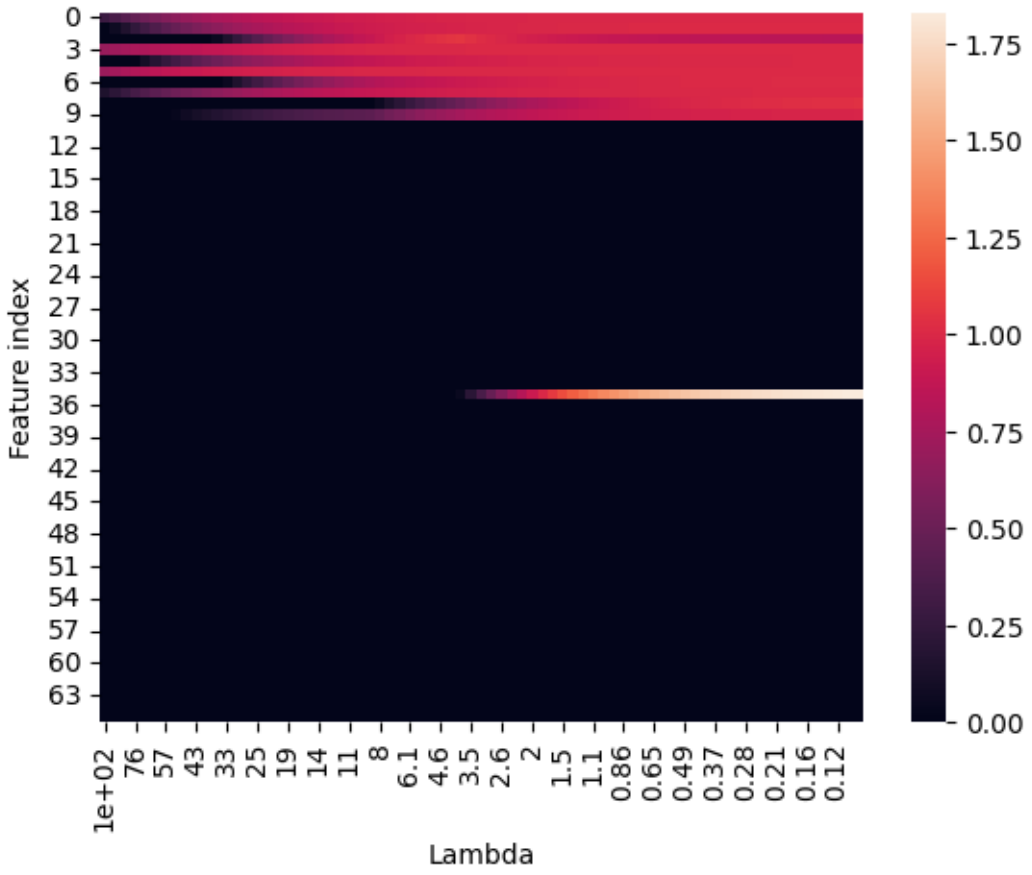




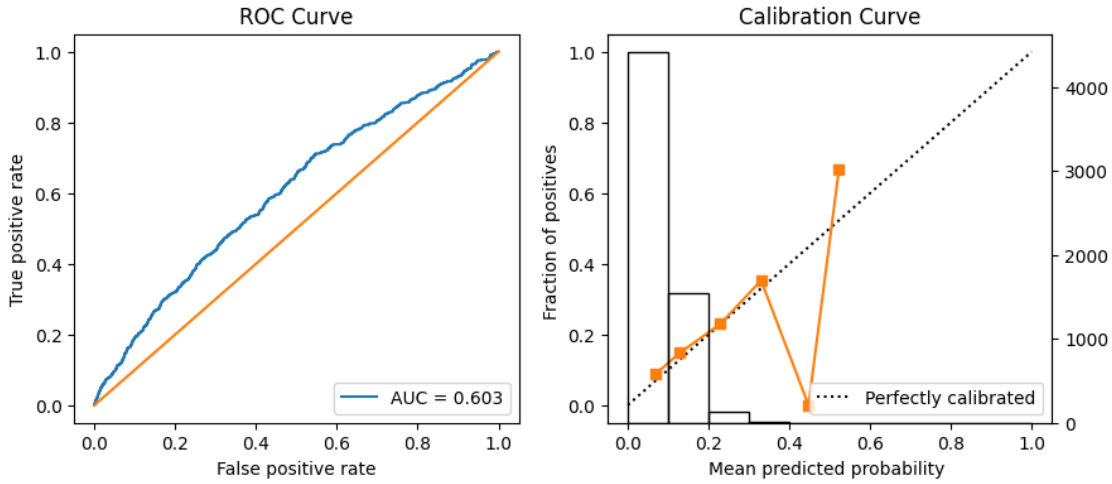
[25]:
```
# userLambda_prn = selectLambda(lambdas_prn, betas_prn, testAUC_prn,␣
 ↪testDev_prn, x_train, bivariate_inputs)
userLambda_prn = 40
# userLambda_prn = 20
lambdas_prn[userLambda_prn]
```

[25]: 2.389892566231049

[26]:
```
ax = sns.heatmap(betas_prn, xticklabels=lambdas_prn)
ax.set_xticklabels(["{:.2g}".format(x) for x in lambdas_prn], rotation=90)
ax.set_xlabel('Lambda')
ax.set_ylabel('Feature index')

# reduce number of xtick labels
ax.set_xticks(ax.get_xticks()[::3]);
```



[27]:
```
modelStats(glmPred_prn[:,userLambda_prn], y_test, y_train, auc_ci=True)
```




```
-------- Metrics --------
prevalence: 0.123
sensitivity: 0.238
specificity: 0.867
accuracy: 0.8
ppv: 0.177
auc score: 0.603
auc lower ci: 0.581
auc upper ci: 0.627
-------------------------
```

[27]:
```
{'prevalence': 0.123,
 'sensitivity': 0.238,
 'specificity': 0.867,
 'accuracy': 0.8,
 'ppv': 0.177,
 'auc score': 0.603,
 'auc lower ci': '0.581',
 'auc upper ci': '0.627'}
```

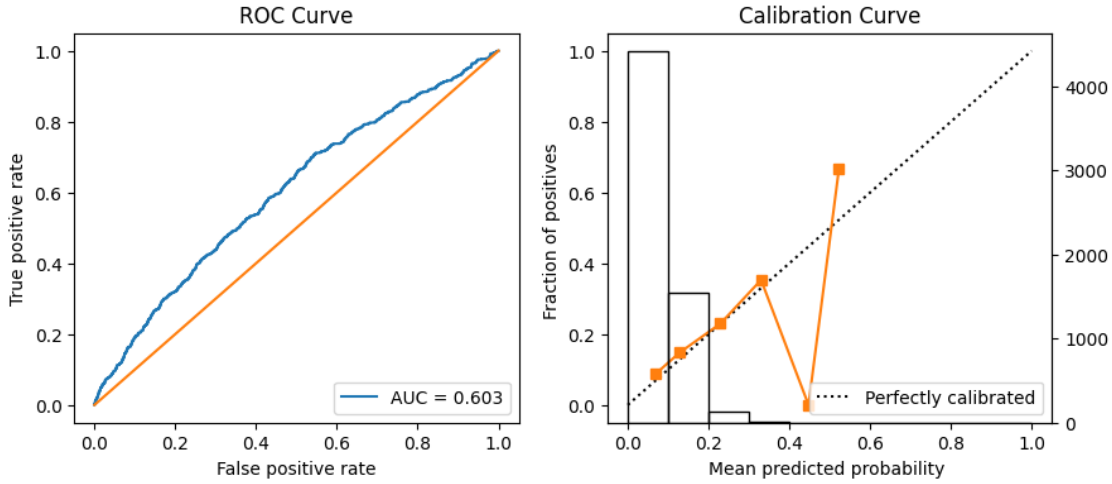
0.12 Validation inference

[28]:
```
F_covariates_train_prn, F_covariates_val_prn, bivariate_inputs_prn =␣
 ↪partialResponses(x_train, x_val, prn_weights, residual=False, method =␣
 ↪method,device=device);

lasso_params = {
    'C': 1/lambdas_prn[userLambda_prn],
    'penalty': 'l1',
    'solver': 'saga',
```

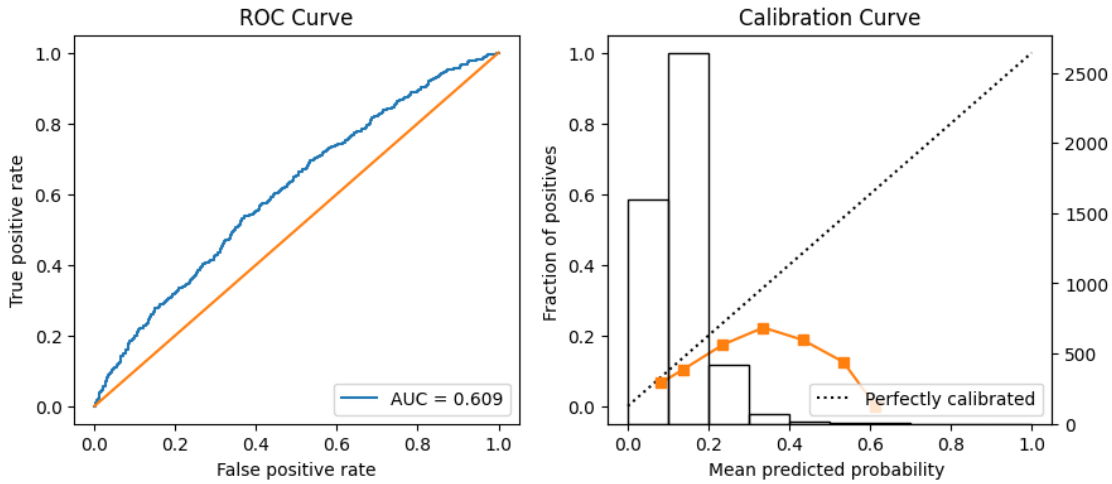
```
    'max_iter': 10000
}

prn_lasso = LogisticRegression(**lasso_params)
prn_lasso.fit(F_covariates_train_prn, y_train.to_numpy().ravel())

y_pred_test_prn_lasso = prn_lasso.predict_proba(F_covariates_test_prn)[:, 1]
y_pred_val_prn_lasso = prn_lasso.predict_proba(F_covariates_val_prn)[:, 1]
lasso_metrics_test = modelStats(y_pred_test_prn_lasso, y_test, y_train,␣
 ↪auc_ci=True)
modelStats(y_pred_val_prn_lasso, y_val, y_train, auc_ci=True)
```

```
Univariate Responses:
Column 0:
Column 1:
Column 2:
Column 3:
Column 4:
Column 5:
Column 6:
Column 7:
Column 8:
Column 9:
Column 10:
Bivariate Responses:
Columns 0 & 1:
Columns 0 & 2:
Columns 0 & 3:
Columns 0 & 4:
Columns 0 & 5:
Columns 0 & 6:
Columns 0 & 7:
Columns 0 & 8:
Columns 0 & 9:
Columns 0 & 10:
Columns 1 & 2:
Columns 1 & 3:
Columns 1 & 4:
Columns 1 & 5:
Columns 1 & 6:
Columns 1 & 7:
Columns 1 & 8:
Columns 1 & 9:
Columns 1 & 10:
Columns 2 & 3:
Columns 2 & 4:
Columns 2 & 5:
Columns 2 & 6:
```

```
Columns 2 & 7:
Columns 2 & 8:
Columns 2 & 9:
Columns 2 & 10:
Columns 3 & 4:
Columns 3 & 5:
Columns 3 & 6:
Columns 3 & 7:
Columns 3 & 8:
Columns 3 & 9:
Columns 3 & 10:
Columns 4 & 5:
Columns 4 & 6:
Columns 4 & 7:
Columns 4 & 8:
Columns 4 & 9:
Columns 4 & 10:
Columns 5 & 6:
Columns 5 & 7:
Columns 5 & 8:
Columns 5 & 9:
Columns 5 & 10:
Columns 6 & 7:
Columns 6 & 8:
Columns 6 & 9:
Columns 6 & 10:
Columns 7 & 8:
Columns 7 & 9:
Columns 7 & 10:
Columns 8 & 9:
Columns 8 & 10:
Columns 9 & 10:
Residual
```



```
-------- Metrics --------
prevalence: 0.123
sensitivity: 0.238
specificity: 0.867
accuracy: 0.8
ppv: 0.177
auc score: 0.603
auc lower ci: 0.575
auc upper ci: 0.63
-------------------------
```




```
-------- Metrics --------
prevalence: 0.123
sensitivity: 0.594
specificity: 0.558
accuracy: 0.561
ppv: 0.129
auc score: 0.609
auc lower ci: 0.582
auc upper ci: 0.629
-------------------------
```

[28]:
```
{'prevalence': 0.123,
 'sensitivity': 0.594,
 'specificity': 0.558,
 'accuracy': 0.561,
```

```
 'ppv': 0.129,
 'auc score': 0.609,
 'auc lower ci': '0.582',
 'auc upper ci': '0.629'}
```

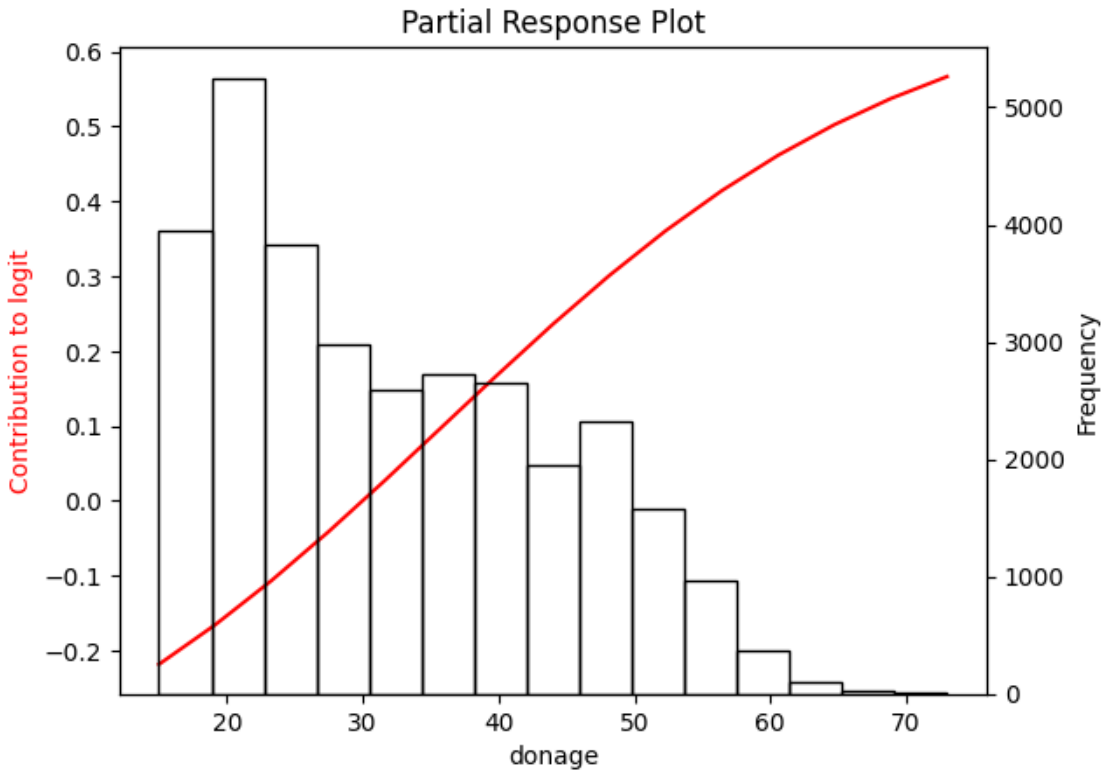
[29]:
```
if SAVE_MODELS:
    save_lasso(prn_lasso, lasso_params, lasso_metrics_test, MODELS_DIR)
```

```
LASSO model saved as lasso_model_20240705_141214
```

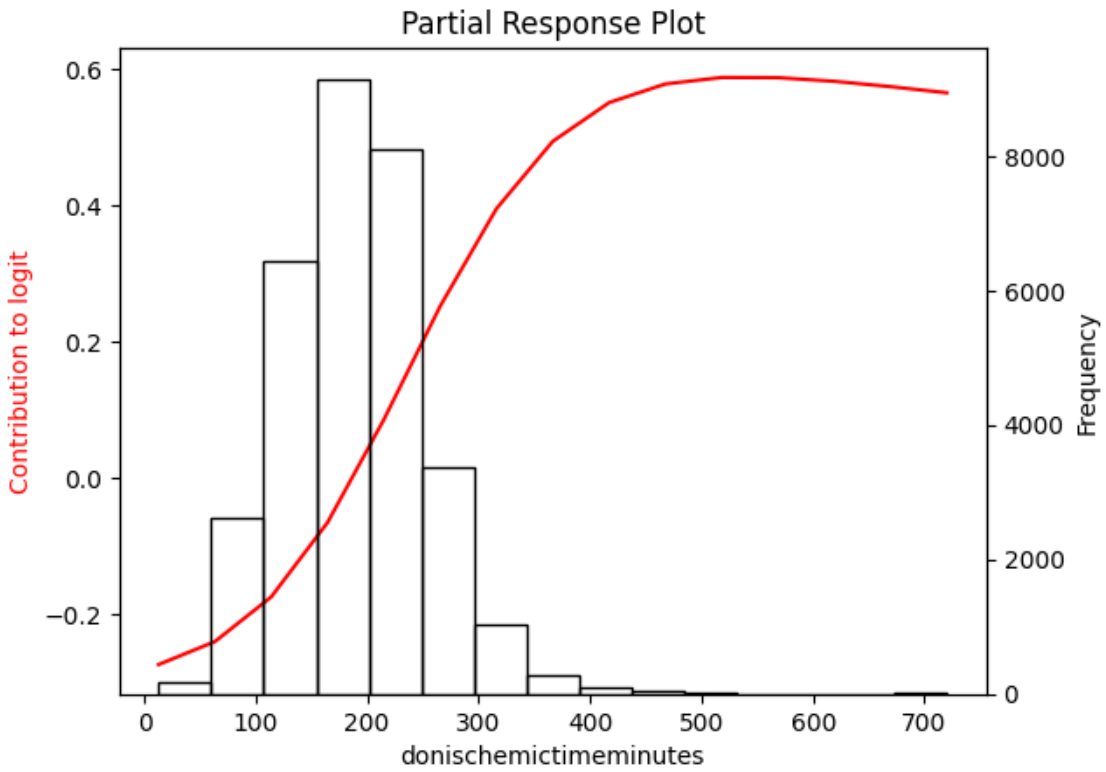
0.13 User selected Partial Response plots from the Partial Response Network

[30]:
```
prPlots(betas_prn, userLambda_prn, x_train0, x_train, data_train_test, prn,␣
↪bivariate_inputs_prn, n_steps = 15, sd_scale=2, method=method,device=device)
```

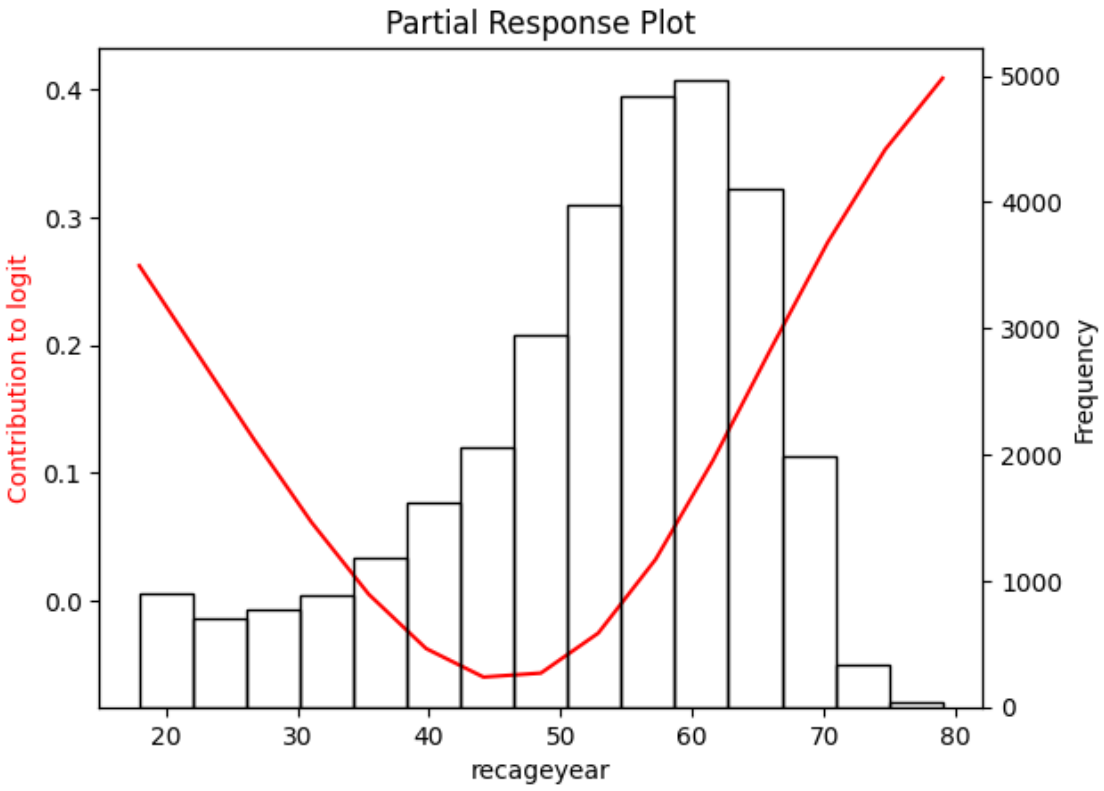
```
0 - univ
```



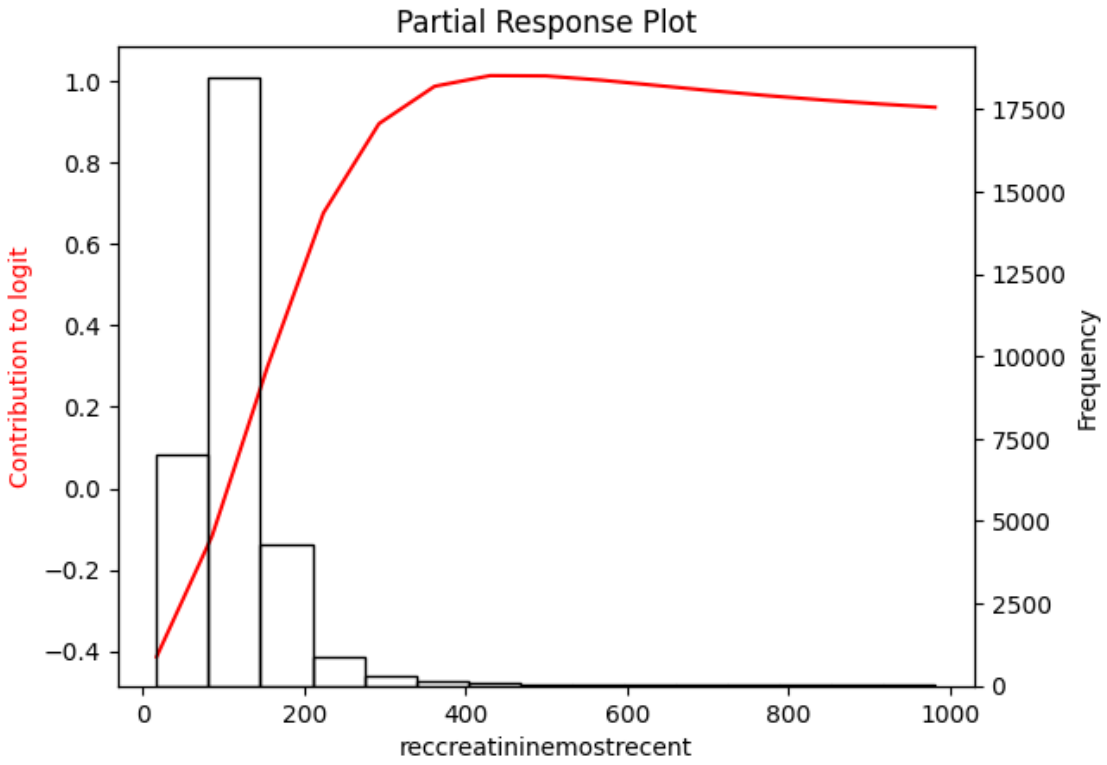

```
1 - univ
```


Partial Response Plot
Contribution to logit
Frequency
donischemictimeminutes
0.6
0.4
0.2
0.0
−0.2
8000
6000
4000
2000
0
0
100
200
300
400
500
600
700

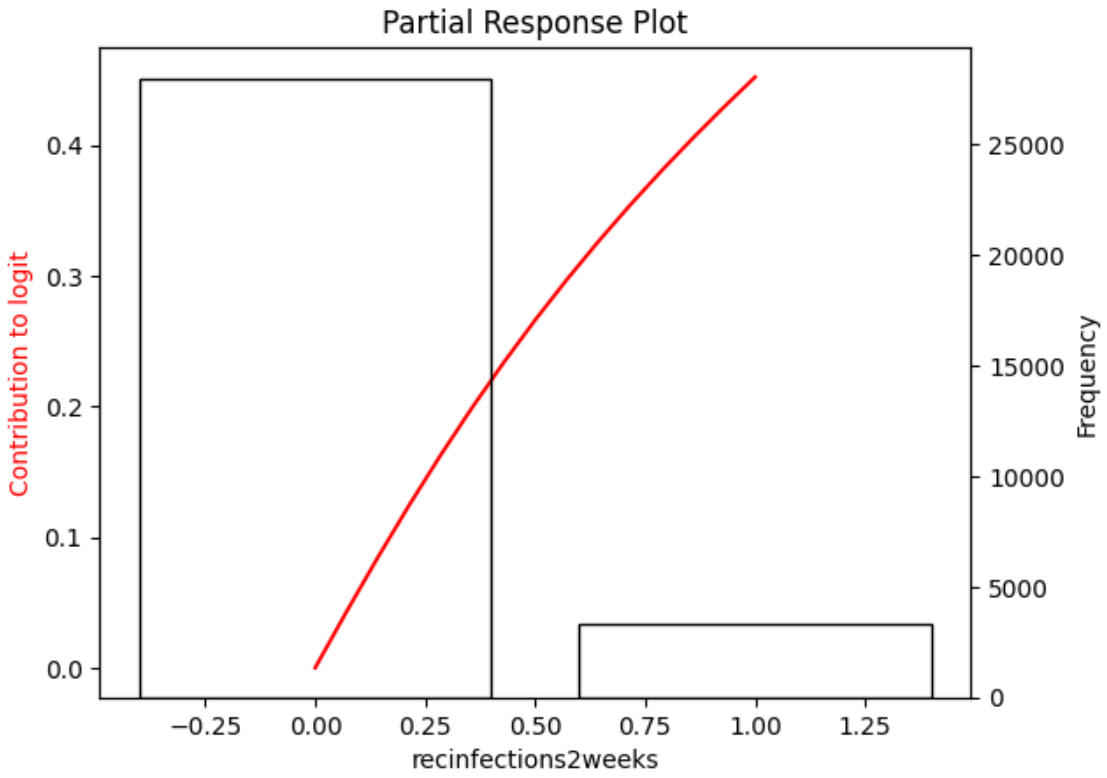
2 - univ


Partial Response Plot
Contribution to logit
Frequency
recageyear
0.4
0.3
0.2
0.1
0.0
5000
4000
3000
2000
1000
0
20
30
40
50
60
70
80

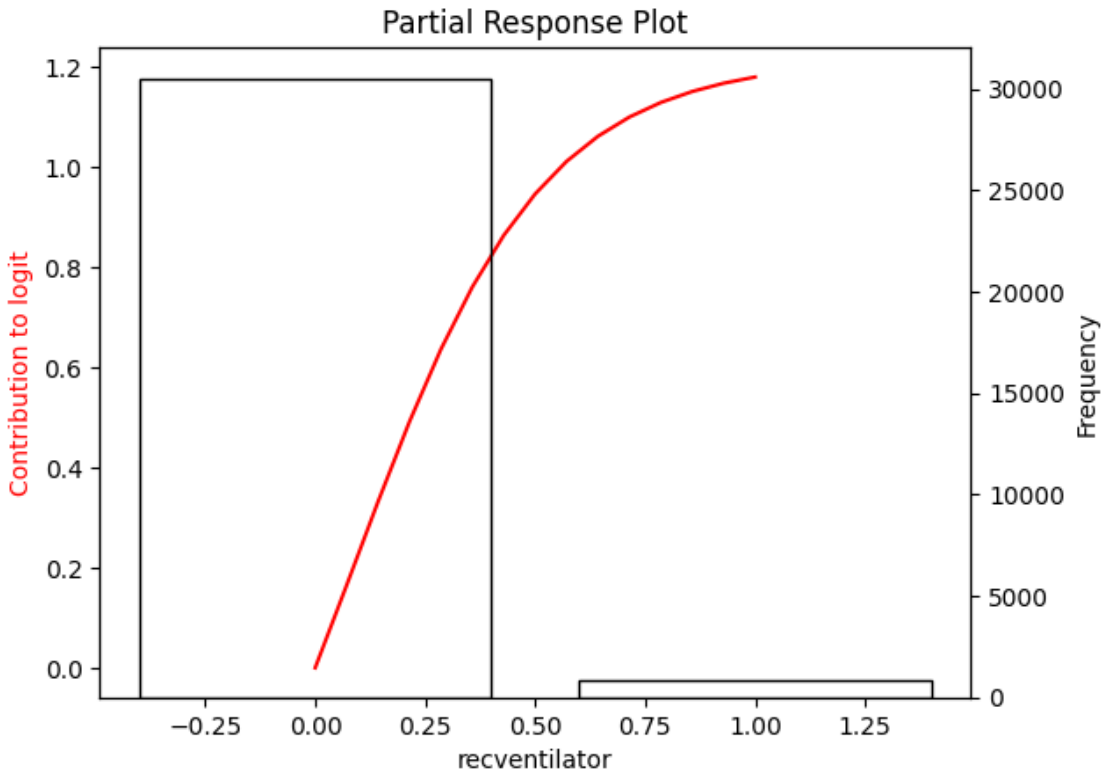
3 - univ


Partial Response Plot
Contribution to logit
Frequency
reccreatininemostrecent
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
17500
15000
12500
10000
7500
5000
2500
0
0
200
400
600
800
1000

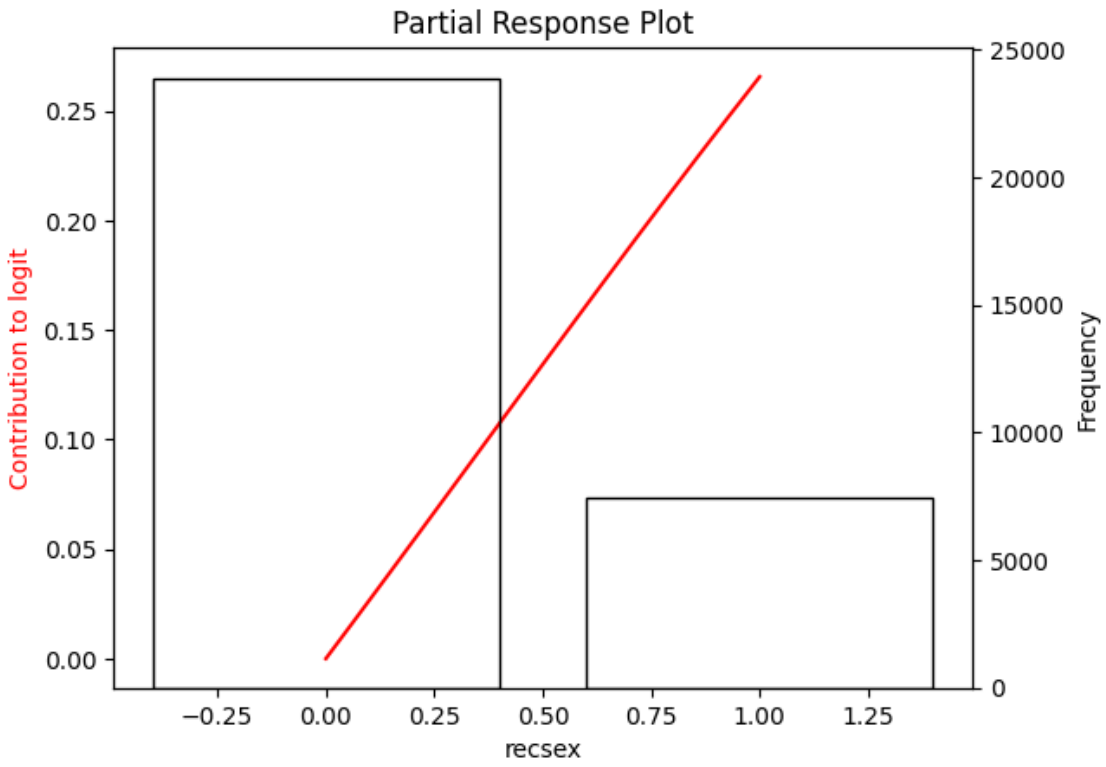
```
4 - univ
```


Partial Response Plot
Contribution to logit
Frequency
recinfections2weeks
0.0
0.1
0.2
0.3
0.4
0
5000
10000
15000
20000
25000
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

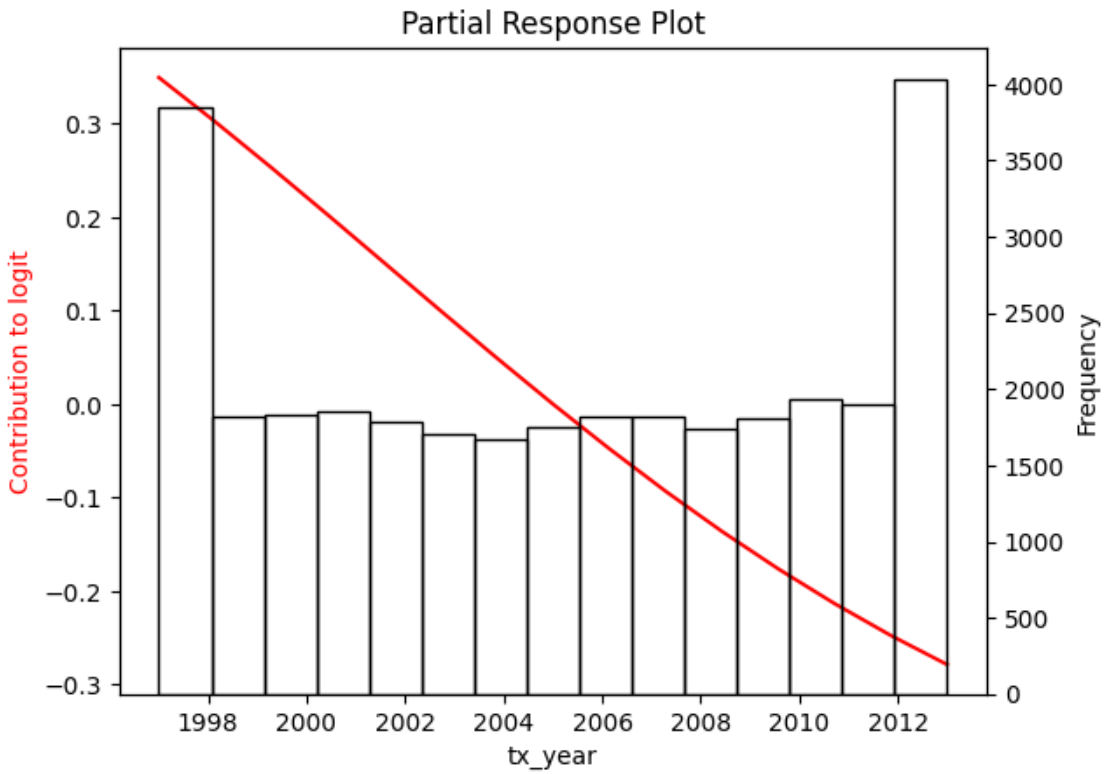
```
5 - univ
```


Partial Response Plot
Contribution to logit
Frequency
recventilator
1.2
1.0
0.8
0.6
0.4
0.2
0.0
30000
25000
20000
15000
10000
5000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

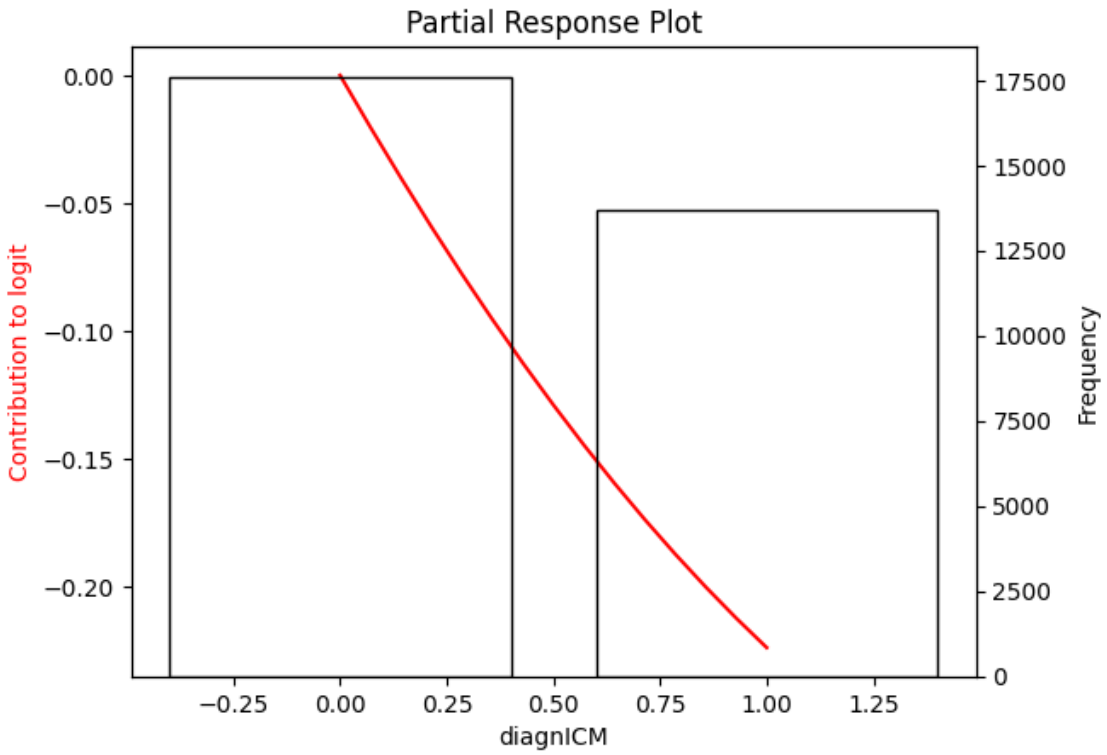
```
6 - univ
```


Partial Response Plot
Contribution to logit
Frequency
recsex
0.25
0.20
0.15
0.10
0.05
0.00
25000
20000
15000
10000
5000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

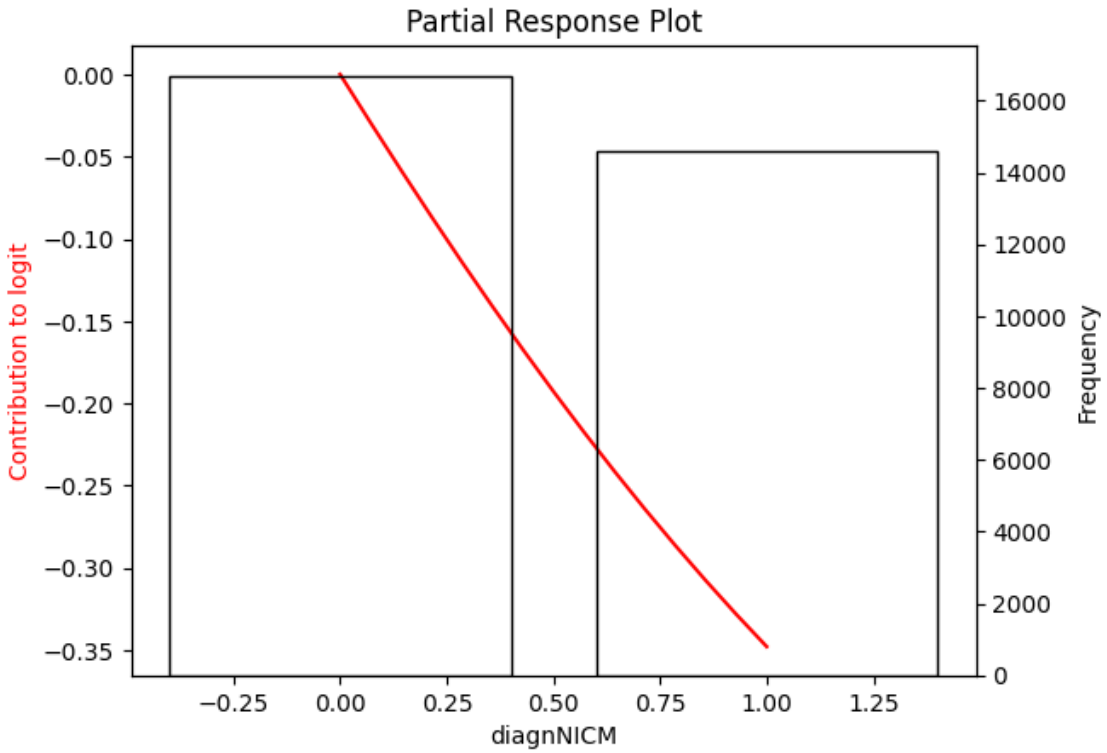
7 - univ

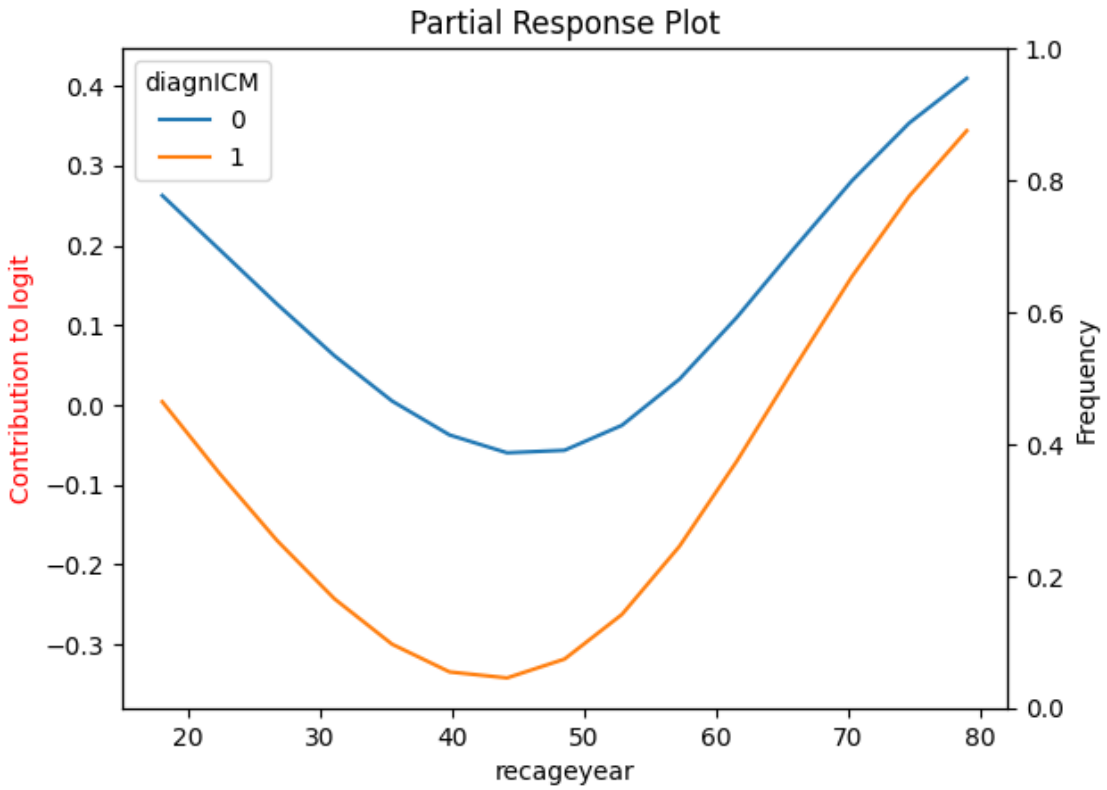

Partial Response Plot
Contribution to logit
Frequency
tx_year
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.3
4000
3500
3000
2500
2000
1500
1000
500
0
1998
2000
2002
2004
2006
2008
2010
2012

```
8 - univ
```


Partial Response Plot
Contribution to logit
Frequency
diagnICM
0.00
−0.05
−0.10
−0.15
−0.20
17500
15000
12500
10000
7500
5000
2500
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25

9 - univ

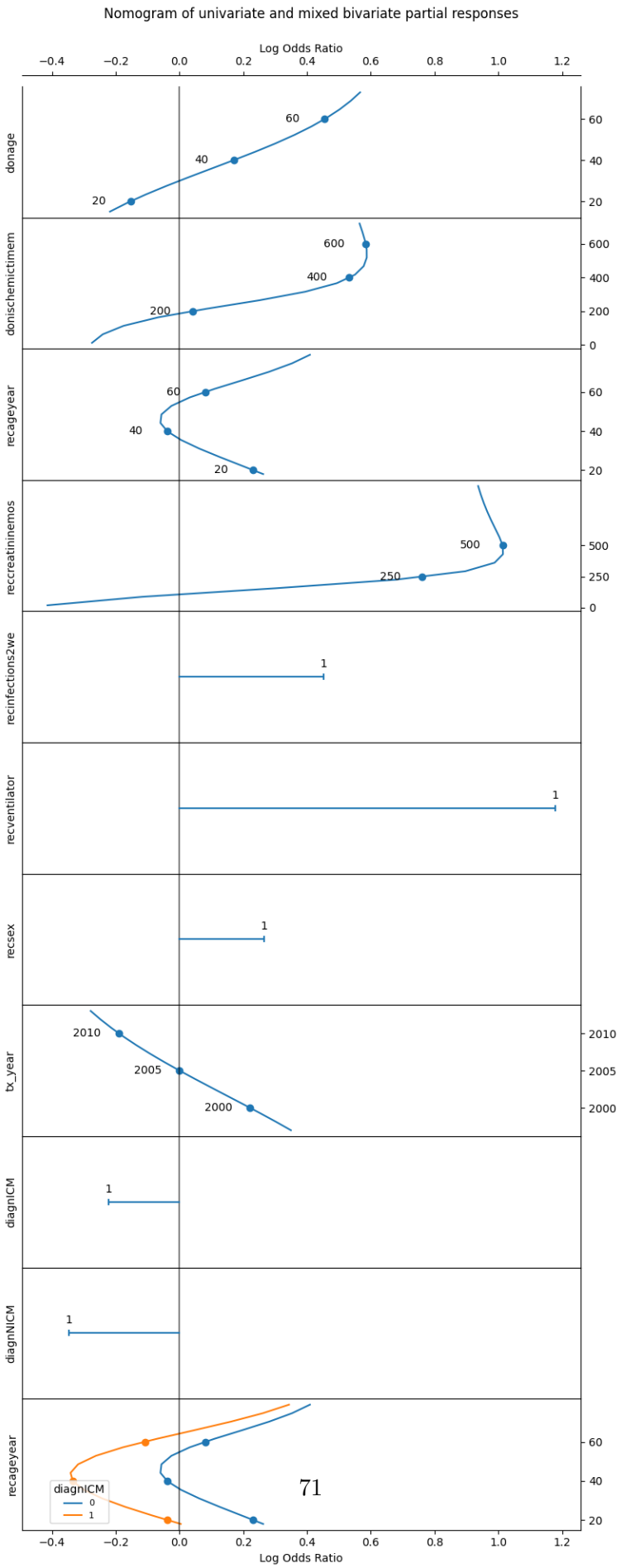

Partial Response Plot
Contribution to logit
Frequency
diagnNICM
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
−0.30
−0.35
16000
14000
12000
10000
8000
6000
4000
2000
0
−0.25
0.00
0.25
0.50
0.75
1.00
1.25



0.14 Partial Response Network Nomogram

[31]:
```
prnomogram_refactor(betas_prn, userLambda_prn, x_train0, x_train,␣
↪data_train_test, prn_weights,␣
↪bivariate_inputs_prn,device=device,method=method)
```

```
C:\Users\Henry Pigot\Documents\MEGAsync\2024 post doc\explainable survival
prediction model\prism_github\prism\prnomogram.py:463: UserWarning:
FigureCanvasAgg is non-interactive, and thus cannot be shown
  nomo.show()
```


Nomogram of univariate and mixed bivariate partial responses
Log Odds Ratio
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8
1.0
1.2
donage
60
40
20
donischemictimem
600
400
200
0
recageyear
60
40
20
reccreatininemos
500
250
0
recinfections2we
1
recventilator
1
recsex
1
tx_year
2010
2005
2000
diagnICM
1
diagnICM
1
recageyear
60
40
20
diagnICM
0
1
Log Odds Ratio